
DATA AND EVALUATION CLOSED-LOOP FOR MODEL CAPABILITY ENHANCEMENT

A PREPRINT

Zhixuan Li*

Baidu Inc.

lizhixuan.2017@tsinghua.org.cn

Jiangan Yuan*

Baidu Inc.

jianganyuan@link.cuhk.edu.cn

Han Xu

Baidu Inc.

xhbj66@gmail.com

June 29, 2026

ABSTRACT

Model capability is the central variable in LLM pre-training, yet is never observed directly: data shapes it prospectively, while evaluation reveals it only retrospectively, compressing samples, prompts, decoding, and scoring rules into one noisy score. Practical optimization runs this backward: a failure is observed first, and the engineer must infer the corpus fix. The two sides speak incompatible vocabularies—benchmark names and per-sample correctness versus data sources, domains, and quality labels—so this inference is usually intuition, not method. We close this gap with the *capability slice*: a group of evaluation samples sharing background condition, task type, solving operation, and output constraint—precise enough to localize a single weakness yet stable enough to survive aggregation, unlike a benchmark name, too coarse, or a single sample, too noisy. Built around this unit, an evaluation taxonomy, a non-instruction data taxonomy, and mapping rules form a closed loop turning a benchmark-level failure into a targeted, testable data intervention. We test this loop on two case studies pulling in opposite directions. First, the loop rules the data out: continued pre-training drives BBH down by -46.82% , but diagnosis traces this to a single masked `<EOS>` loss rather than weakened reasoning; restoring it recovers BBH to 66.44, above the original checkpoint, without changing the data. Second, the loop rules the data in: a persistent math-reasoning weakness is decomposed by solving operation into specific failing combinations, and a weakness-targeted sampling procedure built from it lifts AIME2025/AIME2026 Pass@128 from 6.67/0.00 to 26.67 each. The same unmodified loop reaches opposite, correct verdicts in both cases, showing the evaluation-to-data inference can be routine, auditable, and experimentally validated rather than intuitive.

1 Introduction

In large language model (LLM) pre-training, the quantity that ultimately matters is model capability, but capability itself is never observed directly: it is shaped, prospectively, by the training data the model consumes, and it surfaces only after the fact, retrospectively, as a benchmark score. That score is itself an aggregate, compressing heterogeneous evaluation samples, prompt templates, decoding configurations, answer-extraction rules, and scoring protocols into a single number attached to a benchmark name. State-of-the-art systems are now evaluated on suites that aggregate dozens of such benchmarks, spanning language understanding, factual knowledge, mathematical reasoning, and code generation [Hendrycks et al., 2021a, Zellers et al., 2019, Bisk et al., 2020, Suzgun et al., 2023, Cobbe et al., 2021, Hendrycks et al., 2021b, Chen et al., 2021, Austin et al., 2021, Joshi et al., 2017, Lai et al., 2017, Dua et al., 2019, Wang et al., 2024, Huang et al., 2023, Li et al., 2024a, Zhong et al., 2024], with headline performance reported as a single score per benchmark name. This compression obscures more than it reveals: two models with the same MMLU score may fail on disjoint kinds of problems; a gain on a broad benchmark may stem from improved closed-book factual recall rather than from stronger reasoning; a regression on BBH may originate from output-format violations rather than from any change in task-solving ability. As long as a benchmark name is treated as a direct readout of model capability,

*Co-first authors with equal contributions.

the question that matters for the next training run—what data signal, if added, reweighted, or filtered, would address the failure—remains under-determined.

Practical optimization, however, must traverse this relationship in reverse: the failure shows up first, in an evaluation result, and only afterward can an engineer work backward to the change in the training corpus that might address it. This is consistent with a simple operational claim that motivates the rest of the paper: under a fixed model architecture, training objective, and optimization recipe, training data is the principal controllable factor shaping model capability, while evaluation supplies only a noisy observation of that capability. The data-side problem has accordingly received considerable attention, with corpus-composition methods that search for or optimize over data mixtures [Xie et al., 2023, Liu et al., 2024, Ye et al., 2024], large-scale quality filtering [Penedo et al., 2024, Li et al., 2024b], and synthetic instruction generation [Wang et al., 2023a, Xu et al., 2024, Taori et al., 2023, Wang et al., 2023b]. On the evaluation side, an equally large body of work has refined what is measured, through finer-grained benchmarks [Rein et al., 2024, Glazer et al., 2024] and sample-level analyses of failure modes. Both literatures are mature, and we draw on both; what remains comparatively under-developed is the *interface* between them. The obstacle is that evaluation and training data are described in different languages: evaluation results arrive as benchmark scores or per-sample correctness, whereas training data is catalogued by source, domain, and quality label, and bridging the two has so far been a matter of intuition more than of method. Given an evaluation failure, what kind of training-data signal is the failure actually evidence of, and how should one decide among, for instance, adding more mathematical content, reweighting an existing source, or repairing supervision density?

This paper develops that interface. Mapping between the two sides at the level of benchmark names is too coarse to isolate a single weakness, while mapping at the level of individual samples is too noisy to be trusted; neither granularity supports a reliable inference from failure to fix. We therefore introduce an intermediate representation, the *capability slice*: a group of evaluation samples that share structured conditions along four dimensions—background condition, task type, solving operation, and output constraint. A capability slice is more specific than a benchmark and more stable than a single sample, and it is the unit on which our diagnosis, data-side mapping, and intervention validation all operate.

Around this representation, we build an analysis toolkit with three components (Section 3), instantiating the conceptual framework developed in Section 2. First, an **evaluation sample taxonomy** (Section 3.2) annotates each sample along the four dimensions above, applied to sixteen benchmarks commonly used during pre-training; this taxonomy supports benchmark-composition analysis, the localization of weak capability slices below the benchmark level, and a structured attribution of post-intervention score changes. Second, a **non-instruction data taxonomy** (Section 3.3) characterizes raw corpus segments along four data-side dimensions—content world, discourse structure, operation opportunity, and supervision density—chosen so that, where meaningful, they correspond to the evaluation-side dimensions. Third, a set of **evaluation-to-data mapping rules** (Section 3.4) connects the two taxonomies: a small set of taxonomy-aligned correspondences covers the dimensions that were jointly designed across the two sides, and a complementary LLM-assisted mechanism handles slices that fall outside them. Together, the three components define a closed loop,

Weak Capability Slice → Data Affordance Profile → Data Action → Experimental Validation → Result Analysis,
that converts a benchmark-level observation into an auditable, iterable optimization procedure. We treat the mapping rules as structured heuristics for generating data-intervention hypotheses rather than as causal claims; any resulting intervention remains subject to downstream experimental validation.

We put this loop under stress with two case studies built on a continued pre-training pipeline, deliberately chosen so that they push the diagnosis in opposite directions. In the first (Section 4.1), the loop is asked to explain a regression, and the right answer turns out to rule the data out entirely: continued pre-training improves nearly every benchmark but produces a large regression on BBH (−46.82% relative). Reading the regression through the output-constraint dimension shows that the model frequently emits the correct gold answer and then continues generating beyond it, that the resulting filtered responses become abnormally long, and—tracing the failure into the training pipeline itself—that the loss on the <EOS> token has been masked during continued pre-training. Restoring this single piece of supervision raises BBH from 25.14 to 66.44, above the warm-start checkpoint, without degrading, and in several cases modestly improving, the other benchmarks; the training data itself is never touched. The diagnostic loop thereby converts what initially appeared to be a reasoning regression into a localized, fully correctable termination-supervision bug.

The second case study (Section 4.2) addresses a more characteristic failure mode of pre-training, weak mathematical problem solving, and here the loop runs the other way: the diagnosis rules the data in. The warm-start checkpoint attains only 0.05 at AIME2025 Pass@1 and 0.00 at AIME2026 Pass@1; on AIME2026 the model fails to surface a correct trajectory even at Pass@128, indicating that test-time scaling alone cannot resolve the gap. Decomposing the evaluation set by solving operation shows that the weakness is not isolated arithmetic but specific operation combinations involving constraint tracking, boundary-case reasoning, symbolic transformation, counting, and comparison; a pairwise- and composition-effect analysis further shows that several of these combinations are non-additive, performing worse than

the sum of their pairwise effects would predict. From this diagnosis we derive an importance-sampling procedure (Section 4.2.3) that ranks operation combinations by a diagnosis-aligned weakness score, matches synthetic instruction examples to the highest-scoring combinations, and draws from the resulting pool using Efraimidis–Spirakis weighted reservoir sampling without replacement [Efraimidis and Spirakis, 2006] under a bounded duplication factor. Holding the training recipe and instruction-data budget fixed, the resulting checkpoint reaches 26.67 at AIME2025 Pass@128 and 26.67 at AIME2026 Pass@128, while leaving general-domain performance essentially unchanged.

We close this introduction with two clarifications about the kind of contribution this paper makes, before turning to the body of the paper. First, the contribution is methodological rather than a claim about any specific model or benchmark: the taxonomies and mapping rules supply a vocabulary and a set of heuristics for converting a benchmark-level observation into a testable data-intervention hypothesis, and every such hypothesis is still required to clear a controlled, single-variable experimental design before being treated as a finding. Second, what unifies the two case studies into a single demonstration, rather than two unrelated debugging anecdotes, is that the same closed loop reaches the correct diagnosis in both directions: in the first, it rules the data out, redirecting what looked like a data or capability problem toward the training objective, where the fix actually belonged; in the second, it rules the data in, localizing a genuine capability gap to specific operation combinations and resolving it through a targeted data intervention. The loop is therefore informative not only when it points toward a data fix, but also when it correctly rules one out. Taken together, the two case studies make the same point from opposite sides: turning an evaluation failure into a data fix is something the same procedure can do reliably, with each inferential step open to inspection and each resulting hypothesis checked experimentally, rather than something that depends on an engineer’s intuition. The remainder of this paper follows the structure outlined above: Section 2 develops the conceptual framework in full, Section 3 presents the analysis toolkit and the closed loop it instantiates, Section 4 reports the two case studies, and Section 5 discusses the scope and limitations of the resulting methodology.

2 Conceptual Framework

We organize our framework around three central objects: *evaluation*, *model capability*, and *training data*. Evaluation yields observations of model behavior, but these observations are entangled with implementation choices such as the prompt template, decoding strategy, answer-extraction rule, and scoring protocol. Model capability refers to the model’s stable behavioral pattern under structured conditions, rather than to a score attached to a benchmark name. Training data provides the primary capability-forming signal: it exposes the model to knowledge, discourse structures, reasoning opportunities, and supervised input–output patterns from which capabilities emerge and through which they are expressed.

The core premise of this work is that evaluation is a noisy observation of model capability, while model capability is, in turn, shaped by training data under a fixed model architecture, training objective, and optimization recipe. Thus, a benchmark score should not be interpreted as a direct readout of a single model capability; rather, it is an aggregate outcome produced by the interaction between model capability, sample composition, evaluation protocol, and measurement noise. This motivates a more structured form of analysis, in which benchmark results are decomposed into sample-level observation conditions, aggregated into capability-level behavioral patterns, and then mapped back to the training data that may strengthen or weaken these patterns.

This section follows an engineering analysis order:

Evaluation → Model Capability → Training Data.

We begin with evaluation because practical optimization is typically triggered by an observed failure: a model underperforms on a benchmark, on a subset of samples, or on a specific task family. To make such failures actionable, we first examine what an evaluation sample is actually observing. We then define capability as the model’s stable behavioral pattern over groups of structurally similar evaluation samples, which we later formalize as capability slices. Finally, we connect these capability slices to training data, which provides the intervention surface for improving future models.

This analysis order differs from the underlying causal and optimization loop. In the training process, data first shapes model capability, which is then exposed through evaluation:

Training Data → Model Capability → Evaluation.

Once evaluation reveals a weakness, the diagnosis is mapped back onto a data intervention, closing the engineering loop:

Evaluation Failure → Capability Diagnosis → Data Intervention.

The purpose of the conceptual framework is therefore not merely to describe evaluation, capability, and training data as separate components, but to define an actionable interface among them: evaluation tells us where the model fails,

capability analysis tells us what kind of behavior is unstable or missing, and training-data analysis tells us what signals can be adjusted to remedy the failure.

2.1 Understanding Evaluation

Evaluation serves as the entry point of the proposed analysis framework, since practical model optimization is typically triggered by an observed failure: a model underperforms on a benchmark, on a subset of samples, or on a specific task family. However, a benchmark score should not be interpreted as a direct readout of a single model capability. Instead, it is a noisy aggregate produced by several interacting factors, including the benchmark’s sample distribution, the prompt template, the decoding strategy, the answer-extraction rule, the scoring protocol, sample ambiguity, potential contamination, and the model’s underlying capability. We formalize this by writing the observed benchmark score as

$$s_b = \mathcal{M}(\mathcal{D}_b, \pi, \delta, \rho, f_\theta) + \epsilon_b,$$

where $\mathcal{M}$ denotes the evaluation procedure that aggregates these factors into a score, $\mathcal{D}_b$ denotes the benchmark sample set, π denotes the prompting protocol, δ denotes the decoding strategy, ρ denotes the parsing and scoring rule, f_θ denotes the evaluated model, and ϵ_b summarizes measurement noise such as annotation noise, ambiguous samples, unstable generation, and contamination effects. This formulation is not meant to serve as a statistical estimator; rather, it highlights that the observed score conflates model behavior with evaluation-specific implementation choices.

This distinction matters in practice because many widely used benchmarks are internally heterogeneous. A benchmark may contain samples that require factual recall, passage understanding, commonsense plausibility judgment, arithmetic computation, symbolic manipulation, or output-format following. Conversely, samples from different benchmarks may probe similar underlying behaviors. As a result, benchmark-level comparison can obscure the source of improvement or degradation. For example, an average gain on a broad benchmark may stem from improved closed-book factual recall rather than from stronger reasoning, whereas a drop on a math-oriented benchmark may be caused by answer-format errors rather than by failures in symbolic manipulation.

Because benchmark scores are noisy aggregates, benchmark-level analysis alone is insufficient for data-driven optimization. To make evaluation actionable, we move from benchmark-level scores to sample-level observation conditions. The relevant question is therefore not only whether the model answered a sample correctly, but what the sample was designed to observe: what information the model needed to condition on, what task it was asked to perform, what operations were required to transform the input into the answer, and what output form was required by the scorer. We therefore treat each evaluation sample as a structured observation of model behavior rather than as an isolated binary outcome.

We decompose each evaluation sample along four dimensions: *background condition*, *task type*, *solving operation*, and *output constraint*. These dimensions are designed to answer four complementary questions:

- background condition : What information does the model need to condition on?
- task type : What task is the model asked to perform?
- solving operation : What operations are required to transform the input into the answer?
- output constraint : In what form must the answer be produced and scored?

This decomposition is intentionally independent of benchmark names. Its purpose is to expose the structure of the observation itself, so that heterogeneous samples within the same benchmark can be separated and structurally similar samples across benchmarks can be aggregated.

The first dimension, *background condition*, describes the information environment required to solve the sample. It includes whether the sample is closed-book or context-dependent, whether the relevant evidence is local or distributed across a long passage, and whether the input takes the form of a dialogue, table, code snippet, or expository paragraph. It also includes whether the sample contains ambiguity, noisy formatting, or underspecified references. This dimension is motivated by the observation that many model failures are caused not by the target task alone, but by the information condition under which the task is presented. For instance, factual question answering under a closed-book setting differs from factual question answering where the answer is explicitly stated in a long passage; similarly, arithmetic over a clean word problem differs from arithmetic over a noisy table. By explicitly labeling the background condition, we can separate failures caused by missing knowledge, long-context dependence, discourse complexity, or input noise, etc.

The second dimension, *task type*, identifies the primary task requested by the sample. Examples include factual question answering, passage understanding, logical reasoning, arithmetic word problem solving, code generation, etc. This dimension provides a coarse semantic grouping of evaluation samples, and its main role is to prevent over-reliance on benchmark names: a single benchmark may contain multiple task types, while the same task type may appear across

several benchmarks. Task-type labeling therefore supports cross-benchmark aggregation, enabling comparison of model behavior on similar samples across different evaluation suites.

The third dimension, *solving operation*, describes the operations required to derive the answer from the available information. These operations may include fact recall, span extraction, multi-hop composition, arithmetic computation, program synthesis, etc. This dimension is motivated by the observation that task names are often too coarse to explain failures: two samples may both be categorized as mathematical problem solving, yet one may primarily require numerical calculation while another requires symbolic transformation. Similarly, two reading-comprehension samples may differ substantially depending on whether they require local span extraction, cross-paragraph evidence aggregation, or comparison among options. Solving-operation labels thus provide a finer-grained decomposition than task type alone, making it possible to localize a failure to the specific operation the model fails to execute.

The fourth dimension, *output constraint*, captures the required answer form and the scoring sensitivity. This includes whether the output is a multiple-choice option, a short entity, a number or executable code, as well as constraints such as exact-match scoring, numerical tolerance, option-letter formatting, code executability, and instruction-following requirements. This dimension is necessary because a model can fail a sample even when it possesses the relevant knowledge and performs the main reasoning step correctly: in such cases, the failure may instead originate from answer extraction, formatting, verbosity, invalid units, or non-executable code. Separating output constraints from task type and solving operation allows us to distinguish capability failures from expression and scoring failures.

This four-dimensional decomposition provides several practical benefits. First, it reveals heterogeneity inside benchmark-level scores: instead of treating a benchmark as a single capability probe, we can identify which subsets of samples contribute to an observed gain or drop. Second, it enables cross-benchmark aggregation by grouping samples that share similar observation conditions. Third, it helps localize the source of failure, distinguishing whether a model lacks background knowledge, cannot process the required context, fails to perform the necessary operation, or violates the output constraint. Fourth, and most importantly for this work, it creates a structured interface between evaluation and data: once a failure is localized to a particular combination of background condition, task type, solving operation, and output constraint, we can search for or construct training data that supplies the corresponding learning signal.

2.2 Understanding Model Capability

The sample-level decomposition introduced in Section 2.1 provides the basis for defining model capability. Since a benchmark score is a noisy aggregate over heterogeneous samples and evaluation-specific implementation choices, we do not treat model capability as a property of a benchmark name. Instead, we operationalize model capability as the model’s stable performance pattern across structured evaluation conditions, which we formalize below as capability slices.

Concretely, an evaluation sample defines a single observation point that specifies a particular background condition, task type, solving operation, and output constraint under which the model is observed. A capability slice is a group of evaluation samples that share similar structured conditions along these dimensions, and model capability is defined as the model’s consistent behavioral pattern over one or more such slices. Under this view, capability is neither a benchmark label nor the outcome of a single sample; it is a regularity in model behavior that becomes meaningful only after aggregating observations over structurally similar conditions.

Correctly identifying a weak capability slice requires ruling out two sources of false signal. The first is the isolated sample: a single failed item should not be directly interpreted as a capability failure, since a model may fail because of accidental generation instability, ambiguous wording, or an answer-parsing issue, and such a failure is an observation rather than a diagnosis. For example, an isolated error on a formal mathematics problem does not by itself establish a weakness in mathematical reasoning; only repeated failures on samples that require symbolic transformation under strict numerical output constraints indicate an unstable capability slice. A weak capability slice is therefore established by consistency of failure across structurally similar samples, not by any single outcome.

The second source of false signal is the benchmark name. A benchmark-level score should not be directly interpreted as a capability measurement, since a benchmark often mixes samples with different background conditions, task types, solving operations, and output constraints, such that its aggregate score may conceal multiple distinct behavioral patterns. Consequently, capability slices do not align with benchmark boundaries in either direction: samples from different benchmarks may belong to the same capability slice if they require similar conditions and operations, while samples from the same benchmark may belong to different capability slices if they rely on different operations or output constraints. For instance, a broad knowledge benchmark may contain closed-book entity recall, concept discrimination, commonsense plausibility judgment, and multi-step reasoning samples; treating the benchmark as a single capability would collapse these distinctions. Likewise, a model that performs well on local span extraction but fails on cross-paragraph evidence aggregation should not be described simply as weak at reading comprehension—the failure is more

specifically associated with long-context evidence integration. A weak capability slice, in other words, is defined by shared structured conditions, not by benchmark identity.

Once a weakness has been correctly localized in this way, the natural next step is data-driven intervention: identifying what kind of training data can supply the missing signal. A weakness in closed-book factual recall points to insufficient or poorly retained knowledge coverage, whereas a weakness in long-context evidence aggregation reflects insufficient exposure to documents with distributed dependencies and recoverable evidence chains. Limited symbolic transformation ability suggests insufficient exposure to mathematical derivation or proof-like supervision, and weak executable code generation indicates insufficient alignment between natural-language specifications, API constraints, and runnable programs. Thus, capability slices serve as the intermediate representation between evaluation failures and training data interventions.

2.3 Understanding Training Data

The preceding subsections describe evaluation as a noisy aggregate of model capability and evaluation-specific implementation choices, and model capability as a stable behavioral pattern over capability slices. This subsection turns to the remaining component of the framework: training data, which we treat as the primary source of intervention. Within a fixed model architecture, training objective, and optimization recipe, training data is the principal controllable factor shaping model capability. Consequently, once an evaluation failure has been localized to a specific capability slice, the natural next step is to identify what kind of training signal may strengthen the corresponding behavior.

The relationship between training data and model capability is indirect, since training data rarely specifies capability labels explicitly. Instead, it exposes the model to knowledge, discourse structures, reasoning opportunities, and supervised input–output patterns, from which capabilities are learned. Some of these signals are implicit: a textbook passage, for example, may contain definitions, derivations, and worked examples without explicitly asking the model to solve a task. Other signals are explicit: an instruction–response pair may directly ask the model to answer a question, generate code, follow a specified format, or produce a reasoning trace. This distinction motivates a first-order separation between instruction data and non-instruction data.

2.3.1 Instruction vs. Non-instruction Data

We define *instruction data* as training examples that pair an explicit, task-oriented input with a target response demonstrating the desired behavior under that input. The input specifies what the model is asked to do, through a natural-language instruction, a question, a tool-use request, a code specification, or another task-oriented prompt, while the target response realizes the corresponding output, such as a short answer, an explanation, a derivation, a formatted result, or a tool-calling trajectory. In this sense, instruction data makes part of the evaluation condition explicit at training time: it typically specifies the task type, the relevant background context, and the required output constraint, echoing the dimensions used to decompose evaluation samples in Section 2.1.

We define *non-instruction data* as the complement of instruction data within the training corpus: text that is not primarily organized as paired task input and target response. This category includes naturally occurring or weakly structured text such as web pages, books, encyclopedic articles, academic papers, news articles, forum discussions, code repositories and tables, etc. Such data is typically consumed through the standard language modeling objective, under which the supervision signal is distributed across the sequence rather than concentrated in an explicit answer span.

This distinction matters because instruction and non-instruction data shape model capability through different mechanisms. Non-instruction data primarily supplies the substrate from which capabilities are formed: knowledge coverage, conceptual associations, discourse structure, latent reasoning patterns, and the operational opportunities embedded in raw text. Instruction data, in contrast, more directly shapes how these latent capabilities are activated and expressed under task-like conditions: it aligns inputs with objectives, constrains output formats, demonstrates reasoning or tool-use trajectories, and teaches the model how to respond when an evaluation-like request is presented.

This separation is not intended to establish a strict hierarchy in which one data type is universally more important than the other; rather, it provides an analytical lens for examining each type on its own terms. For non-instruction data, the central question is what learnable signals are latent in the text and how densely they occur. For instruction data, the central question is how raw content, solving operations, and output constraints are converted into explicit, supervised behavior. The following subsections analyze these two data types from their respective perspectives, so that capability weaknesses diagnosed through evaluation can be mapped to concrete training-data interventions, completing the data-intervention surface introduced at the start of this section.

2.3.2 Understanding Non-instruction Data

As noted in Section 2.3.1, non-instruction data does not present itself as explicit task supervision: rather than pairing a clearly separated instruction with a target response, it exposes the model to naturally occurring text, code, tables, documents, and discourse patterns. The learning signal is therefore latent, embedded in what the text talks about, how the information is organized, what transformations are implicitly demonstrated, and how recoverable the next-token prediction target is from its surrounding context. To analyze such data in a way that is useful for capability-oriented optimization, we characterize non-instruction data along four complementary perspectives: *content world*, *discourse structure*, *operation opportunity*, and *supervision density*.

The first perspective, *content world*, describes the knowledge, entities, concepts, domains, and symbolic systems covered by the data, i.e., what part of the world the data exposes the model to. For example, encyclopedic text provides entity-attribute knowledge, scientific articles provide domain concepts and causal mechanisms, mathematical textbooks provide formal objects and transformations, and code repositories provide APIs, programming idioms, and implementation patterns. This perspective matters because many capability failures originate from insufficient coverage of the relevant content space: a model cannot reliably answer questions about an entity, concept, theorem, API, or domain if the corresponding content is absent, sparse, low-quality, or poorly connected within the training corpus.

The second perspective, *discourse structure*, describes how information is organized within the data. It includes whether the text is narrative, expository, conversational, tabular, list-like, proof-like, or code-like; whether the relevant information is local or distributed across a long span; and whether the text contains explicit references, temporal structure, contrastive organization, examples, definitions, or derivation steps. This perspective is motivated by the observation that the same content world can support different learnable behaviors depending on how it is structured: a short encyclopedia paragraph mainly supports entity recall, whereas a long report with distributed evidence supports cross-paragraph aggregation; a list of mathematical formulas exposes symbolic objects, whereas a worked proof exposes transformation trajectories. Discourse structure thus determines how content is made available to the model, independently of what that content is.

The third perspective, *operation opportunity*, describes the implicit operations that the text allows the model to learn, such as fact association, definition matching, comparison, counting, temporal ordering, causal attribution, evidence aggregation, symbolic transformation, algorithmic decomposition, code completion, and specification-to-implementation alignment. Operation opportunity is distinct from explicit task supervision: non-instruction data may never ask the model to perform an operation, yet the operation can still be embedded in the text. A financial report containing multiple comparable quantities creates opportunities for comparison and aggregation; a proof demonstrates symbolic transformation; a code file paired with comments exposes specification-to-code alignment. Among the four perspectives, this one provides the most direct bridge from raw text to the solving operations required by evaluation samples.

The fourth perspective, *supervision density*, describes how strong, explicit, and recoverable the learning signal is under the language modeling objective. Some text offers dense, low-noise supervision, such as definitions followed by examples, worked solutions, aligned code comments, tables with clear headers, or derivations in which each step is locally supported. Other text offers weaker supervision, such as fragmented web pages, noisy forum discussions, ambiguous references, duplicated boilerplate, or passages in which the relevant relation is implicit and difficult to recover from context. Supervision density therefore captures not only whether useful content or operations are present, but also how efficiently they can be learned from next-token prediction; two documents may carry the same nominal information yet differ substantially in how learnable that information is.

Together, these four perspectives form a progression from raw content to learnable signal:

Content World $\rightarrow$ Discourse Structure $\rightarrow$ Operation Opportunity $\rightarrow$ Supervision Density.

The content world determines what knowledge and symbolic objects are available; the discourse structure determines how these objects are arranged and connected; the operation opportunities determine what transformations, comparisons, derivations, or alignments are implicitly demonstrated by that arrangement; and the supervision density determines how strongly these signals are exposed to the model during training. This progression is useful in practice because data optimization should not only ask whether a corpus covers a given domain, but also whether the corpus presents that domain in a structure that supports the operations required by the target capability slice.

This progression also clarifies how the analysis of non-instruction data connects to the sample-level decomposition introduced in Section 2.1. The four perspectives used here are not identical to the four dimensions used to decompose evaluation samples; rather, in designing the two schemes, we sought to keep them broadly aligned wherever possible, though the correspondence is approximate rather than exact. The *content world* of training data corresponds to the knowledge and domain component of the *background condition* of evaluation samples, while the *discourse structure* of training data corresponds to its structural component, namely the context scope, input format, and information

organization that the same dimension also captures. The *operation opportunity* in training data maps directly onto the *solving operation* required to answer an evaluation sample. This approximate compatibility allows later sections to map evaluation failures to candidate data interventions: once a model failure is localized to a capability slice, we can search for non-instruction data that covers the relevant content world, presents it in the required discourse structure, embeds the necessary operation opportunities, and provides sufficiently dense supervision.

2.3.3 Understanding Instruction Data

Section 2.3.1 characterizes instruction data as the component of the training corpus that converts raw content, solving operations, and output constraints into explicit, supervised behavior. We make this characterization concrete by analyzing instruction data along two complementary perspectives: an *evaluation-side* perspective, which treats an instruction example as a training-time analogue of an evaluation sample, and a *non-instruction-data-side* perspective, which treats it as a supervised reorganization of the latent signals already present in non-instruction data. These two perspectives correspond to the dual role instruction data plays in the framework: it specifies an explicit, task-like condition under which the model is expected to respond, while simultaneously repackaging existing content, structure, and operations into paired input–output supervision.

From the evaluation-side perspective, an instruction example exposes the same four dimensions used in Section 2.1 to decompose evaluation samples. The input specifies the *background condition*, such as a provided passage, dialogue history, table, code context, or closed-book setting, and the *task type*, such as factual question answering, mathematical problem solving, code generation, information extraction, or tool-use invocation. The target response, in turn, demonstrates the required *solving operation*—evidence aggregation, arithmetic computation, symbolic transformation, program synthesis, or step-by-step reasoning—and instantiates the corresponding *output constraint*, including answer format, numerical normalization, JSON schema conformance, code executability, or citation style.

This perspective is useful because it connects instruction data directly to capability slices. Once an evaluation failure is localized to a specific combination of background condition, task type, solving operation, and output constraint, instruction data can be constructed to provide explicit supervision under the same or a neighboring condition. For instance, a failure on long-context passage QA that requires cross-paragraph evidence aggregation under short-form exact-match scoring motivates instruction data that pairs long documents with questions whose answers require distributed evidence, together with concise, normalized target outputs. In this sense, instruction data directly shapes how capabilities are activated and expressed under evaluation-like conditions.

From the non-instruction-data-side perspective, instruction data can instead be understood as a supervised reorganization of the latent signals characterized in Section 2.3.2: *content world*, *discourse structure*, *operation opportunity*, and *supervision density*. Instruction construction selects and reformats these latent signals into explicit task supervision: an encyclopedic passage can be converted into factual or multi-hop question answering, a mathematical derivation into a problem–solution pair, API documentation into a code-generation task and a table into extraction, comparison, or arithmetic questions. Through this process, instruction construction increases the explicitness and density of the learning signal by aligning latent content and operations with an explicit objective and a target response.

This second perspective matters because instruction data is not defined merely by the surface presence of an imperative sentence or a question mark; its value depends on what latent signal it captures and how faithfully that signal is converted into supervision. Low-quality instruction data may specify a task while relying on shallow pattern matching, noisy answers, underspecified context, or weakly grounded reasoning. High-quality instruction data, by contrast, preserves the relevant content world, exposes the necessary discourse structure, renders the target operation observable, and produces a response whose format is aligned with the desired behavior. Instruction-data quality should therefore be assessed not only by task category, but also by the quality of the underlying signal from which the instruction is derived.

The two perspectives are complementary rather than competing. The evaluation-side view specifies what behavior an instruction example is intended to train: under what condition, for what task, using which operation, and under what output constraint. The non-instruction-data-side view specifies where the corresponding learning signal originates: what content is covered, how it is organized, what operation is latent or demonstrated, and how dense and reliable the supervision is. Together, the two views allow instruction data to serve as a bridge between evaluation failures and data construction; later sections use this bridge to map diagnosed capability weaknesses to both non-instruction-data retrieval criteria and instruction-data synthesis criteria.

2.3.4 How Data Shapes Capability

The preceding analysis of non-instruction and instruction data (Sections 2.3.2 and 2.3.3) suggests two complementary paths through which training data shapes model capability. The first is the *non-instruction data path*, in which capability-forming signals are acquired indirectly from raw or weakly structured text under the language modeling

objective. The second is the *instruction data path*, in which task-like supervision more explicitly specifies the condition, objective, response pattern, and output constraint. These two paths should be understood as analytical abstractions rather than mutually exclusive mechanisms: in practice, instruction data often repackages signals that originate from non-instruction data, and non-instruction pretraining provides the representational substrate on which instruction tuning operates.

The non-instruction data path is indirect. Non-instruction data primarily shapes the substrate of capability by exposing the model, at scale, to its content world, discourse structure, operation opportunities, and supervision density:

Non-instruction Data → Content World → Discourse Structure
→ Operation Opportunity → Supervision Density → Capability Substrate.

For example, consider a quarterly financial report. Its content world consists of company-specific entities, financial metrics, and accounting concepts; these are organized through a tabular discourse structure that aligns the same quantities across consecutive periods and segments. This structure in turn creates an operation opportunity for comparison and aggregation, and because each table carries clearly labeled headers and explanatory captions, these comparisons are also exposed with high supervision density.

However, even in this example, the report never explicitly instructs the model to compute a growth rate or compare two figures; the underlying operation is demonstrated, not assigned as an explicit task. More broadly, such signals are rarely presented as explicit task supervision: the model acquires them through next-token prediction alone, so the resulting capability may remain latent, incomplete, or difficult to activate under evaluation-like prompts. A model may be repeatedly exposed to comparisons of exactly this kind during pretraining and still fail to compute one reliably when explicitly queried at evaluation time. Non-instruction data is therefore best understood as providing broad capability-forming conditions rather than direct supervision for a specific evaluation slice.

Because this path is indirect, data–capability attribution is inherently noisy. A failure on a given capability slice cannot usually be traced to a single missing document or absent pattern; instead, the relevant question is whether the training corpus provides sufficient coverage, structural exposure, operation opportunities, and learnable signal density for the behavior that slice requires. For instance, improving symbolic reasoning may require not only mathematical content, but derivation-like text in which intermediate transformations are sufficiently dense and consistent. In this sense, non-instruction data shapes capability through cumulative distributional exposure rather than through any single, identifiable intervention.

The instruction data path is more direct. Instruction examples specify task-like conditions and desired responses, and therefore more closely resemble the evaluation samples used to observe model behavior:

Instruction Data → Background Condition and Task Type → Solving Operation Demonstration
→ Output Constraint Alignment → Behavior on Target Capability Slices.

From the evaluation-side perspective developed in Section 2.3.3, an instruction example exposes the model, at training time, to background conditions, task types, solving operations, and output constraints analogous to those used during evaluation. This correspondence makes instruction data particularly effective for shaping how a capability is activated and expressed: whether the model follows the requested task, selects an appropriate reasoning mode, produces the required answer format, and satisfies the relevant scoring constraints.

This relationship is closely related to domain adaptation. If instruction data is closer to the evaluation distribution in task format, input structure, reasoning pattern, and output constraint, it may provide a more direct training signal for the corresponding capability slice. For example, instruction data constructed from long documents paired with short-answer questions may better match long-context QA evaluation than raw documents alone. Yet this similarity should not be interpreted as a sufficient causal explanation: performance may also depend on data quality, diversity, contamination control, optimization stage, loss weighting, and the capability substrate already established through non-instruction pretraining.

This caveat is not incidental: it follows from the dual nature of instruction data established in Section 2.3.3. Instruction data is not an independent source of capability running parallel to non-instruction data, but a transformation applied on top of it. Returning to the financial-report example, an instruction pair built from that report, such as a question asking the model to compute quarter-over-quarter revenue growth, does not introduce a new comparison ability; it converts the comparison opportunity already latent in the table into an explicit, scored task. The instruction format makes the operation harder to ignore and easier to evaluate, but the comparison itself was already there to be learned, or not, from the underlying non-instruction signal. Consequently, instruction data inherits a ceiling from the corpus it draws on: if the source content is shallow, noisy, or weakly grounded, no amount of instruction-format refinement can manufacture a capability that was never available to learn. What such data produces instead is a model that exhibits the surface form

of the target behavior without having reliably acquired the underlying operation. This gap often remains invisible until the model is evaluated under conditions slightly outside the instruction distribution.

Taken together, the two paths play complementary but interacting roles in shaping model capability. Non-instruction data primarily builds the broad substrate of knowledge, representations, discourse familiarity, and latent operations; instruction data more directly shapes how this substrate is activated, composed, and expressed under task-like conditions. This division of roles closes the engineering loop introduced in Section 2: once an evaluation failure has been localized to a capability slice (Section 2.2), diagnosing it requires considering both paths jointly: whether the non-instruction corpus provides sufficient latent signal for the target behavior, and whether the instruction corpus offers explicit supervision that matches the target evaluation slice without overfitting to benchmark-specific artifacts.

3 Analysis Toolkit

3.1 Overview of the Toolkit

The analysis toolkit operationalizes the conceptual framework introduced in Section 2, under which evaluation provides a noisy observation of model capability, while model capability is, in turn, shaped by training data. Its purpose is to convert benchmark-level observations into structured capability diagnoses and, from these diagnoses, into concrete data-intervention hypotheses that can be examined under controlled experimental conditions. As illustrated in Figure 1, the toolkit comprises three components: an evaluation sample taxonomy (Section 3.2), a non-instruction data taxonomy (Section 3.3), and a set of evaluation-to-data mapping rules (Section 3.4) that connect the two.

The first component, the **evaluation sample taxonomy**, decomposes each benchmark sample into a structured description of its capability demand. Rather than treating a benchmark as a homogeneous measurement unit, we annotate each sample along four dimensions: background condition, task type, solving operation, and output constraint. This taxonomy answers the question: *what capability does each evaluation sample require?* It allows benchmark results to be re-aggregated into capability slices below the level of the benchmark name, making it possible to attribute a model weakness to, for example, closed-book factual recall, cross-paragraph evidence aggregation, symbolic transformation, or strict output-formatting requirements, rather than to the benchmark as a whole.

The second component, the **non-instruction data taxonomy**, characterizes raw corpus segments by the learnable signals they may supply during pre-training. Non-instruction data is described along four complementary dimensions: content world, discourse structure, operation opportunity, and supervision density. This taxonomy answers the question: *what learnable signal does each corpus segment provide?* It yields a data-side affordance profile that supports corpus composition analysis, quality-aware filtering, targeted data retrieval, and reweighting of existing data sources.

The third component, the **evaluation-to-data mapping rules**, connects the evaluation-side and data-side descriptions. Given a weak capability slice identified from evaluation results, the mapping rules translate its capability demand into a target data affordance profile. Because instruction data shares the same four-dimensional vocabulary used to describe evaluation samples, the corresponding mapping is comparatively direct, amounting to restating the slice specification as a synthesis target; non-instruction data, in contrast, is characterized by a structurally different taxonomy, so translating a weak capability slice into a target content world, discourse structure, operation opportunity, and supervision-density profile requires an explicit translation step. The mapping rules accordingly concentrate on this more indirect, non-instruction-data side of the bridge, combining a small set of taxonomy-aligned dimension correspondences with a complementary LLM-assisted mechanism for slices that fall outside them. This component answers the question: *what data intervention may address a given evaluation failure?* Consistent with their role as hypothesis generators rather than as guarantees of improvement, the rules are treated as structured heuristics that guide data retrieval, reweighting, targeted synthesis, and filtering, with any resulting intervention remaining subject to downstream experimental validation.

Together, these three components instantiate a single closed-loop workflow: a weak capability slice is first localized through the evaluation sample taxonomy; the mapping rules translate it into a target data affordance profile; a corresponding data action is selected by inspecting the non-instruction data taxonomy’s annotations over the existing corpus; and the resulting intervention is applied and evaluated under controlled experimental conditions before being mapped back onto the evaluation sample taxonomy to assess whether the targeted slice has improved. This design allows benchmark analysis to move beyond aggregate score comparison and turns the evaluation–data relationship into an actionable, auditable, and iterable optimization loop.

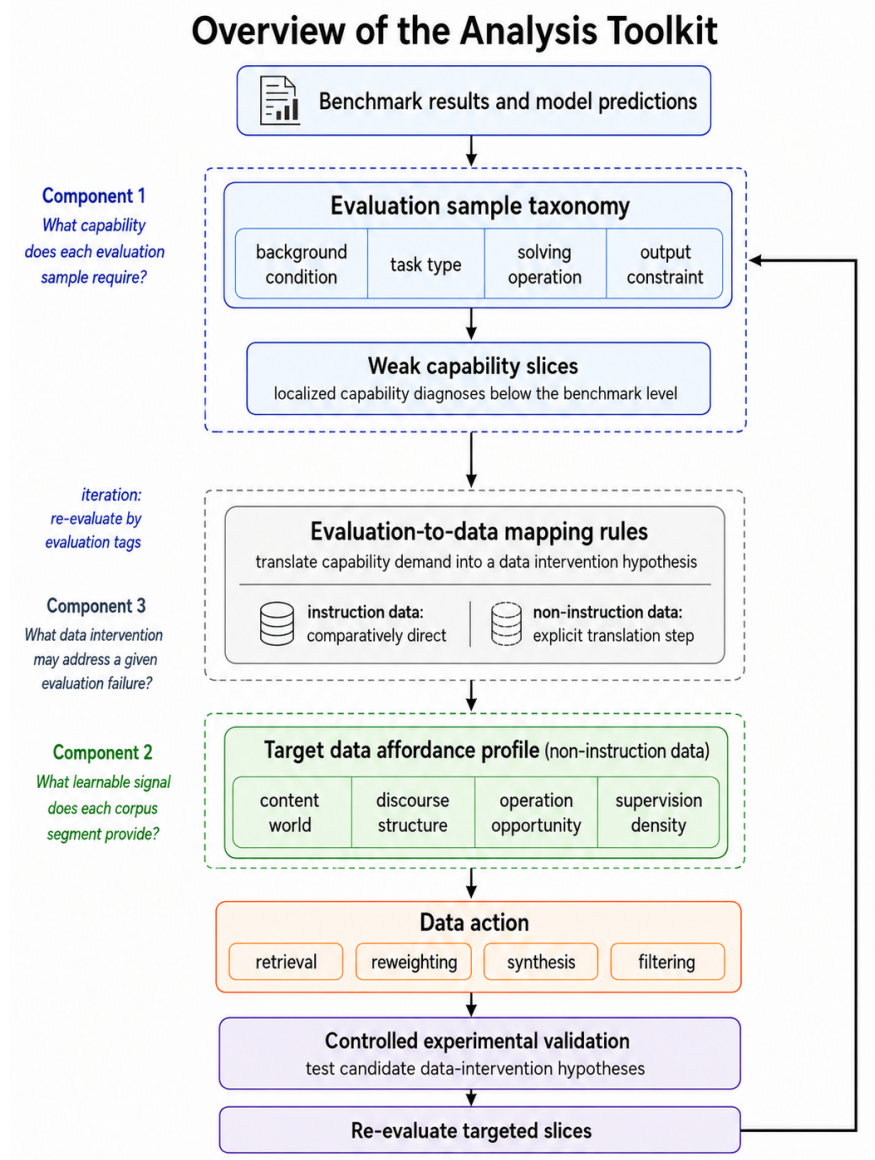


Figure 1: Overview of the analysis toolkit. The toolkit translates noisy benchmark-level observations into structured capability diagnoses, and these diagnoses, in turn, into candidate data-intervention hypotheses that can be validated experimentally.

3.2 Evaluation Sample Taxonomy

3.2.1 Construction and Annotation

We construct the taxonomy from samples drawn from sixteen benchmarks commonly used for evaluating pre-training base models: PIQA, HellaSwag, MMLU, TriviaQA, RACE, DROP, MMLU-Pro, BBH, C-Eval, CMMLU, AGIEval-CN, MBPP, HumanEval, GSM8K, MathQA, and MATH (Minerva). Together, these benchmarks cover commonsense plausibility judgment, factual question answering, passage understanding, mathematical problem solving, logical reasoning, and code generation. Sampling from this heterogeneous suite, rather than from any single benchmark, prevents the taxonomy from being tied to a particular benchmark format and allows it to capture capability demands that recur across benchmark boundaries.

The construction process begins with manual inspection of sampled items. For each item, annotators examine the question, the candidate options when available, the reference answer, and the scoring rule, with the goal of identifying

the minimal set of conditions and operations required for a correct prediction. We organize the resulting capability requirements into the four dimensions introduced in Section 2.1: *background condition*, *task type*, *solving operation*, and *output constraint*.

We refine the taxonomy through multiple rounds of human–LLM co-design. In each round, we draft an annotation prompt, use LLMs to annotate a held-out set of benchmark samples, manually review the resulting annotations, and collect cases of disagreement at category boundaries. The prompt and the label definitions are then revised to reduce the recurrent confusions identified in this review. Typical revisions include clarifying the boundary between task type and solving operation, distinguishing parametric knowledge from contextual evidence, preventing over-labeling of reasoning operators, and aligning output-constraint labels with the benchmark’s actual scoring rule rather than an idealized human grading criterion.

For large-scale annotation, each sample is independently labeled by two strong LLMs, DeepSeek-V3.2 and Qwen3.5 Plus, under the same prompt and the same input fields: the question or task instruction, the candidate options when available, the reference answer, and the scoring rule when it can be recovered from the benchmark protocol. We compare the two models’ annotations category by category. Annotations on which the two models agree are accepted after a lightweight sanity check, while disagreements are routed to human review. Human reviewers inspect the original sample, the two model annotations, and their accompanying rationales, and then adjudicate the final labels according to the taxonomy’s definitions and decision rules. The resulting adjudicated labels support benchmark composition analysis, weak-slice diagnosis, and the downstream mapping from evaluation to training data.

3.2.2 Background Condition

The background condition category describes the information environment under which the model is evaluated, answering the question: *what information must the model condition on to solve the sample?* This category is necessary because samples that share the same task type may still differ substantially in their information source and presentation. For example, a factual question may be answerable from parametric knowledge alone, whereas a passage-QA item may require evidence drawn from a long context; similarly, a multiple-choice commonsense item may be solvable only by comparing the options themselves when the question stem provides little useful information on its own. To capture these differences, we decompose background condition into six subcategories: language, domain, parametric–contextual need, context scope, discourse form, and ambiguity noise. Each subcategory targets a distinct aspect of the information environment, ranging from the linguistic and domain setting of the sample to the scope, structure, and reliability of the evidence it provides. Table 1 summarizes the label space and diagnostic role of each subcategory, and Table 2 lists representative annotated examples.

3.2.3 Task Type

The task type category describes the primary user-facing objective of the sample, answering the question: *what task is the model asked to perform?* It provides a coarse semantic grouping of evaluation samples, independent of benchmark identity, that captures what the sample is asking for rather than how the answer is to be derived. Task type is intentionally separated from solving operation: task type describes the final objective presented to the model, whereas solving operation describes the intermediate operations required to reach the answer. The same task type may therefore recur across different solving operations, and conversely, samples with superficially different task types may rely on the same underlying operation. Table 3 lists the task type labels used in our taxonomy, and Table 4 lists representative annotated examples.

3.2.4 Solving Operation

The solving operation category describes the intermediate operations required to transform the input into the answer, answering the question: *how is the answer derived?* Unlike task type, solving operation is multi-label, since a single sample may require a composition of operations; however, annotators are instructed to mark only the operations that lie on the minimal sufficient solving path, rather than every operation that could conceivably be applied. This category constitutes the main bridge between evaluation diagnosis and data intervention: performance on a benchmark may degrade not only because the model lacks the relevant knowledge, but also because it fails to aggregate evidence, track constraints, formulate equations, or handle algorithmic edge cases. By making these distinct failure sources observable at the sample level, operation labels allow a low score to be attributed to the specific operation underlying it, rather than to the task as a whole. Table 5 summarizes the operation labels, and Table 6 lists representative annotated examples.

Table 1: Background condition labels. Domain labels are illustrative rather than exhaustive; the full domain taxonomy follows standard subject categorizations (e.g., the MMLU/C-Eval/CMMLU subject lists)

Subcategory	Label space	Diagnostic role
Language	zh, en, mixed, code_mixed	Identifies the linguistic environment of the sample and supports language-specific slice analysis.
Domain	mathematics, physics, law, history, finance, others	Separates content coverage from task format and supports domain-conditioned aggregation across benchmarks.
Parametric-contextual need	0, 1, 2, 3	Distinguishes whether the answer depends primarily on parametric knowledge, on provided context, or on a combination of both, and helps separate knowledge gaps from context-utilization failures.
Context scope	standalone, local_span, multi_sentence, whole_passage, cross_option	Measures the span of information that must be integrated to solve the sample, distinguishing local-evidence reliance from long-context dependence.
Discourse form	standalone_question, dialogue, narrative, expository, report_news, list_or_table, code_spec	Describes the structural form in which the required information is presented, supporting slice analysis by input format such as dialogue, narrative, or tabular text.
Ambiguity noise	low, medium, high	Captures surface ambiguity, unclear references, or irregular formatting that may interfere with solving, helping separate noise-induced errors from genuine capability failures.

3.2.5 Output Constraint

The output constraint category describes the form in which the answer must be produced and the rule by which it is judged, answering the question: *how must the answer be expressed and scored?* As the fourth and final dimension of the decomposition, it isolates a source of failure that is orthogonal to the preceding three: a model may correctly identify the relevant background condition, perform the intended task, and execute the required solving operation, yet still be scored incorrect because the generated output is unparseable, incorrectly normalized, numerically inequivalent to the reference, or non-executable. This category therefore separates failures of expression and scoring from failures of underlying capability. We decompose output constraint into six subcategories: answer form, answer space, scoring rule, format rigidity, exactness requirement, and explanation required, each capturing a distinct aspect of how the benchmark protocol constrains and evaluates the model’s response. Table 7 summarizes the label space and diagnostic role of each subcategory, and Table 8 lists representative annotated examples.

3.2.6 Usage

The four-dimensional taxonomy developed above is not an end in itself; its value lies in how it converts the sample-level decomposition introduced in Section 2.1 into concrete analysis operations. In this paper, we use the taxonomy in three complementary ways: to characterize what a benchmark actually measures, to localize where a model’s behavior is unstable, and to explain why a benchmark score changes after an intervention. The three uses share the same underlying mechanism—aggregating sample-level labels into structured groups—but differ in what is held fixed and what is allowed to vary.

The first use is to analyze the capability composition of the sixteen benchmarks introduced in Section 3.2.1. Rather than treating each benchmark as a homogeneous capability probe, we compute, for each benchmark, the distribution of background conditions, task types, solving operations, and output constraints over its constituent samples. This profile makes explicit, for example, whether a benchmark is dominated by closed-book factual recall, by passage-grounded evidence extraction, by arithmetic computation, by code generation, or by strict, parser-sensitive answer formats. Because this analysis operates below the benchmark name, it also exposes cases in which a single benchmark mixes several distinct capability demands, or in which two differently named benchmarks turn out to probe a similar underlying slice.

Table 2: Examples of background condition annotations.

Benchmark	Sample	Labels
MMLU	Question: According to the Law of Effect, behaviors followed by negative consequences. A. occur more frequently B. occur less frequently C. will never be performed again D. will be performed more forcefully	Language: en Domain: psychology, education_and_training Parametric need: 3 Contextual need: 0 Context scope: standalone Discourse form: standalone_question Ambiguity noise: low
DROP	Passage: There were 1,882 households of which 21.4% had children under the age of 18 living with them, 36.4% were Marriage living together, 9.7% had a female householder with no husband present, 3.6% had a male householder with no wife present, and 50.4% were non-families. 37.7% of all households were made up of individuals and 13% had someone living alone who was 65 years of age or older. The average household size was 2.20 and the average family size was 2.90. Question: Which household was third most common?	Language: en Domain: family, mathematics Parametric need: 0 Contextual need: 3 Context scope: whole_passage Discourse form: expository Ambiguity noise: low
HumanEval	Sample: <pre>def make_a_pile(n): """ Given a positive integer n, you have to make a pile of n levels of stones. The first level has n stones. The number of stones in the next level is: - the next odd number if n is odd. - the next even number if n is even. Return the number of stones in each level in a list, where element at index i represents the number of stones in the level (i+1). Examples: >>> make_a_pile(3) [3, 5, 7] """</pre>	Language: code_mixed Domain: information_technology, mathematics Parametric need: 3 Contextual need: 3 Context scope: whole_passage Discourse form: code_spec, standalone_question Ambiguity noise: low

The second use is to support the discovery and analysis of weak capability slices, instantiating the notion of a capability slice introduced in Section 2.2. Given model predictions and sample-level correctness, we aggregate performance over groups of samples that share similar labels along the four dimensions, rather than over an entire benchmark or an isolated item, and examine these groups to identify slices on which performance is consistently and non-trivially low. A weak capability slice can then be represented as a structured tuple over the four dimensions, for example:

(whole_passage, passage_understanding_qa, evidence_aggregation, exact_match).

Such a slice is more actionable than a benchmark name because it specifies the background condition, task type, solving operation, and output constraint under which the model consistently fails, rather than a single aggregate score that conflates many such conditions. This localization step is the analytical core of the second use: it converts a diffuse, benchmark-level weakness into a small number of well-specified, structurally homogeneous failure patterns that can be examined and prioritized individually. Once localized, these weak slices serve as the direct input to the evaluation-to-data mapping rules introduced in Section 3.4, which translate each slice into a concrete optimization

Table 3: Task type labels.

Task type	Description
factual_qa	Answering closed-book questions about objective facts, entities, relations, or events.
concept_discrimination	Distinguishing, applying, or reasoning about concepts, definitions, categories, or principles.
passage_understanding_qa	Answering questions whose evidence must be located or integrated from an accompanying passage or context.
commonsense_plausibility_judgment	Selecting the option or continuation most plausible under commonsense, physical, or social constraints.
next_event_or_next_step_prediction	Predicting the most likely next event, action, or continuation given a preceding context.
logical_reasoning	Reasoning about truth conditions, implications, consistency, necessity, or other abstract relations.
arithmetic_word_problem	Solving natural-language problems that require arithmetic or numerical computation.
formal_math_problem_solving	Solving formal mathematical problems involving algebra, geometry, symbolic derivation, or proof-like reasoning.
code_generation_from_spec	Generating executable code from a natural-language or structured specification.
code_comprehension_or_debugging	Understanding code behavior, identifying bugs, or reasoning about edge cases.
information_extraction_or_transformation	Extracting, normalizing, mapping, or transforming information from a given input.

Table 4: Examples of task type annotations.

Benchmark	Question	Task type
MMLU	Question: Say’s Law A. is the basis of Keynesian economic analysis. B. is the basis of Classical economic analysis. C. states that demand creates its own supply. D. indicates that prices will be stable in capitalist economies.	factual_qa
BBH	Question: Tamika tells the truth. Ka says Tamika tells the truth. Fidel says Ka tells the truth. Elanor says Fidel tells the truth. Amberly says Elanor tells the truth. Does Amberly tell the truth?	logical_reasoning
GSM8K	Question: 100 people apply for a job at Google. Of the people that apply, only 30% receive interviews. Of those who receive interviews, 20% receive a job offer. Of those who receive a job offer, a third of the people accept the position. How many people accept the position?	arithmetic_word_problem

direction together with a corresponding data or supervision strategy. In this way, the taxonomy not only localizes where the model fails but also constrains what kind of data intervention is likely to address that failure.

The third use is to explain benchmark changes after optimization. When a model improves or regresses on a benchmark, we decompose the resulting score delta along the same four dimensions rather than reading it as a single undifferentiated signal. This decomposition allows us to attribute an observed gain or drop to a specific source, for example, improved factual recall, more reliable arithmetic computation, better code generation, or a reduction in format-sensitive output failures, and to distinguish such capability-level changes from changes driven mainly by output-constraint compliance. In this way, the taxonomy closes the loop between evaluation and data: it is first used to diagnose weak capability slices, then to map these slices to candidate data or supervision interventions, and finally to verify, after intervention, whether

Table 5: Solving operation labels.

Solving Operation	Description
fact_recall	Recalling a fact from parametric knowledge, without relying on provided context.
concept_alignment	Mapping an observed phenomenon, expression, or description onto the correct concept, definition, or principle.
span_extraction	Locating and extracting a relevant evidence span directly from the provided context.
evidence_aggregation	Combining multiple pieces of evidence, typically drawn from different sentences or regions of a passage.
compare_or_rank	Comparing, ranking, or identifying differences among entities, quantities, or options.
option_elimination	Deriving the answer primarily by ruling out incorrect options rather than by direct computation.
multi_hop_composition	Chaining two or more reasoning steps or intermediate relations into a single derivation.
causal_inference	Inferring or evaluating causal relations between events, actions, or variables.
temporal_reasoning	Reasoning about event order, duration, or other temporal relations.
counting	Counting entities, occurrences, or events to obtain a numerical answer.
arithmetic_computation	Performing arithmetic operations such as addition, subtraction, ratio computation, differencing, or summation.
equation_formulation	Translating a natural-language description into mathematical relations or equations.
symbolic_transformation	Performing algebraic, logical, or other symbolic transformations.
constraint_tracking	Tracking multiple conditions, constraints, states, or intermediate results throughout a derivation.
physical_state_simulation	Simulating physical, spatial, object, or tool-use states to determine an outcome.
social_script_simulation	Simulating social situations, behavioral scripts, or plausible event continuations.
algorithm_design	Designing program logic, data structures, or algorithmic procedures to satisfy a specification.
code_execution_simulation	Mentally executing code to reason about its behavior or output.
boundary_case_reasoning	Handling edge cases, exceptional inputs, or boundary conditions that the main solution path may not cover.

the resulting benchmark improvement originates from the targeted capability slice or from an unrelated source. The case studies in Section 4 demonstrate these three uses.

3.3 Non-instruction Data Taxonomy

3.3.1 Construction and Annotation

The non-instruction data taxonomy is designed to characterize the learnable signal exposed by raw pre-training text. Unlike evaluation samples, which are organized around an explicit task and a scoring rule, non-instruction samples carry no separated task input or target response; they supervise the model only through the language modeling objective. Annotating such samples therefore requires more than a topical label: the taxonomy must describe what a corpus segment is about, how its information is organized, what operation-like patterns it exposes, and whether the resulting signal is sufficiently clean and informative to serve as effective next-token supervision.

We design the taxonomy under three constraints. First, it should reflect properties intrinsic to non-instruction data rather than properties imported from a downstream task. We accordingly organize the data-side description into four dimensions, *content world*, *discourse structure*, *operation opportunity*, and *supervision density*, which respectively characterize the topical and referential content of a sample, the way its information is arranged and connected, the capability-forming operations explicitly present in the text, and the recoverability and informativeness of the resulting supervision signal. Second, the taxonomy should remain compatible with the evaluation-side decomposition introduced in Section 2.1 and instantiated in Section 3.2. Since the overall framework aims to close the loop between evaluation failures and data interventions, the four data-side dimensions are deliberately designed to correspond, where such

Table 6: Examples of solving operation annotations.

Benchmark	Question	Solving operation
MMLU	Question: Which of the following styles of fuzzer is more likely to explore paths covering every line of code in the following program? A. Generational B. Blackbox C. Whitebox D. Mutation-based	concept_alignment fact_recall
GSM8K	Question: Bubbles collects stuffed animals. She has three stuffed puppies, five stuffed koalas, two stuffed zebras and four stuffed frogs. If she wants to buy enough stuffed goats, such that the percentage of stuffed goats is 30% of all of her stuffed animals, how many stuffed goats should she buy?	arithmetic_computation equation_formulation multi_hop_composition symbolic_transformation
HumanEval	Question: <pre>def iscube(a): """ Write a function that takes an integer a and returns True if this integer is a cube of some integer number. Note: you may assume the input is always valid. Examples: iscube(1) ==> True iscube(2) ==> False iscube(-1) ==> True iscube(64) ==> True iscube(0) ==> True iscube(180) ==> False """</pre>	algorithm_design boundary_case_reasoning concept_alignment

correspondence is meaningful, to the evaluation-side dimensions of *background condition*, *task type*, *solving operation*, and *output constraint*. Third, the taxonomy should remain compatible with the data annotations already used in our data pipeline, including existing domain labels and quality-related labels. Rather than replacing these annotations, we incorporate them into a broader structure in which they serve as reusable signals for corpus composition analysis, filtering, and targeted data construction.

For large-scale annotation, we primarily use compact LLM annotators such as Qwen3 30B-A3B or Gemma4 26B-A4B, rather than conventional text classifiers such as fastText- or BERT-style models. This choice follows from the structure of the annotation problem itself: several dimensions, most notably discourse structure and operation opportunity, require the annotator to reason over multi-sentence dependencies, separate topic from operation, and apply boundary rules that shallow or short-context classifiers struggle to capture. In preliminary prompt-development iterations, compact LLMs produced more faithful boundary decisions and generalized more reliably across domains and discourse forms, whereas smaller discriminative classifiers were more prone to overfitting the annotation examples, missing long-range dependencies, and collapsing structurally complex samples into coarse topic-level labels. At the same time, recent improvements in LLM inference systems have made compact-LLM annotation practical at the scale required for large-scale corpus processing. We therefore use compact LLMs as the main annotation engine, with rule-based checks, sampled human audits, and distribution-level sanity checks used to monitor annotation quality.

The annotation prompts are developed through multiple rounds of human-LLM co-design, following the same general protocol used for the evaluation sample taxonomy in Section 3.2.1. In each round, we draft or revise the prompt, annotate a held-out set of corpus segments, manually inspect the resulting labels, and collect recurring boundary errors; the label definitions, decision rules, and few-shot examples are then revised to reduce these errors. Typical revisions include requiring discourse-structure annotation to first identify the sample’s main learnable signal before assigning context-scope labels, selecting few-shot examples that more clearly distinguish local from passage-level dependency, and refining the operation-opportunity taxonomy so that its first-level types have sharper boundaries and its second-level sub-types capture the most common operational forms without encouraging over-labeling.

Table 7: Output constraint labels.

Subcategory	Label space	Diagnostic role
Answer form	mcq_single, mcq_multi, extractive_span, generated_text, exact_numeric, math_expression, code, boolean	Describes the expected surface form of the final answer, and helps separate failures caused by mismatched output type from failures caused by incorrect content.
Answer space	closed_set, semi_open, open	Indicates whether the answer is selected from a finite set, admits equivalent surface forms, or lies in an open generation space, and distinguishes selection-style failures from generation-style failures.
Scoring rule	option_match, exact_match, normalized_text_match, semantic_equivalence, numeric_equivalence, unit_test_pass	Identifies the criterion the evaluator applies to judge correctness, and clarifies whether an observed error reflects incorrect content or a mismatch with the scoring procedure itself.
Format rigidity	0, 1, 2, 3	Measures how sensitive the evaluator is to surface-level formatting deviations, and helps isolate formatting-induced failures from genuine reasoning errors.
Exactness requirement	0, 1, 2, 3	Measures how precisely the generated answer must match the reference under the benchmark scorer, and separates failures of precision from failures of reasoning.
Explanation required	no, optional, yes	Specifies whether the scoring protocol requires an accompanying explanation or rationale, and clarifies whether a missing explanation should itself be counted as an error.

3.3.2 Content World

The *content world* dimension characterizes what a non-instruction sample is about, abstracting the topical, referential, and distributional semantics of raw text into a small set of complementary labels. Unlike instruction-side annotations, which primarily characterize task form and response behavior, content-world annotations are designed to support corpus composition analysis, data mixture design, and targeted data supplementation. We operationalize this dimension through three complementary subcategories: domain labels, entity–concept labels, and text embeddings. Table 9 summarizes these subcategories together with their diagnostic role within the content-world dimension. Table 10 complements this summary with representative annotated examples.

The two-level domain taxonomy is clearly structured, with the coarse-grained and fine-grained tiers deliberately serving distinct purposes. The fine-grained labels are identical to the domain labels used in the background condition dimension of the evaluation sample taxonomy (Section 3.2.2), so that data-side and evaluation-side annotations share a single label space and directly support the evaluation-to-data mapping rule (Section 3.4). The coarse-grained and fine-grained labels are not assigned sequentially; rather, both are produced jointly under a fixed coarse-to-fine mapping that constrains which fine-grained label is admissible under each coarse-grained label, ensuring the two tiers remain mutually consistent by construction rather than one being derived from the other after the fact. This coarse-grained tier nonetheless yields a smaller, more manageable category set that is convenient for data staging, tabulation, and mixture-ratio experimentation. Taken together, the domain subcategory provides the corpus’s primary topical-coverage signal, linking evaluation-defined capability slices to controllable units of data composition and mixture design.

The entity–concept annotation extracts salient referential and conceptual units from the text. Entities correspond to concrete, referential objects or named items, such as persons, organizations, locations, products, events, laws, technologies, or scientific objects, whereas concepts correspond to abstract, reusable knowledge units, such as theories, methods, processes, properties, phenomena, skills, or social issues. Compared with the domain subcategory, which situates a sample within a small set of coarse and fine topical categories, entity–concept annotation operates at a more granular level, surfacing the specific instance-level units that populate a given domain and providing more actionable signal for entity- and concept-level grouping and clustering. Taken together, the entity–concept subcategory provides the corpus’s primary fine-grained referential and conceptual signal, complementing the topical-coverage signal of the

Table 8: Examples of output constraint annotations.

Benchmark	Question	Golden answer	Labels
MMLU	Question: What is the optimal span of control? A. 2 B. 5 C. 7 D. None of the above	D	Answer form: mcq_single Answer space: closed_set Scoring rule: option_match Format rigidity: 0 Exactness requirement: 0 Explanation required: no
GSM8K	Question: While playing with her friends in their school playground, Katelyn saw 50 fairies flying above the nearby forest. After about twenty minutes, one of her friends saw half as many fairies as Katelyn saw come from the east and join the fairies that were there. Ten minutes later, 30 fairies flew away. How many fairies are remaining?	45	Answer form: exact_numeric Answer space: semi_open Scoring rule: numeric_equivalence Format rigidity: 1 Exactness requirement: 3 Explanation required: no
HumanEval	Question: <pre>def add(x: int, y: int): """Add two numbers x and y >>> add(2, 3) 5 >>> add(5, 7) 12 """</pre>	return x + y	Answer form: code Answer space: semi_open Scoring rule: unit_test_pass Format rigidity: 0 Exactness requirement: 3 Explanation required: no

domain subcategory and supporting targeted coverage analysis and data construction at the level of individual entities and concepts.

Beyond these discrete labels, each sample is also encoded by a lightweight embedding model into a fixed-dimensional vector representation. Although continuous rather than categorical, this embedding can be viewed as a complementary, latent content-world signal: samples with similar embeddings tend to share topical or referential content even when they are not assigned identical domain or entity–concept labels. Taken together, the embedding subcategory provides the corpus’s primary continuous content-similarity signal, complementing the discrete domain and entity–concept annotations and supporting similarity-based clustering, near-duplicate detection, and retrieval of topically or referentially related samples.

3.3.3 Discourse Structure

The *discourse structure* dimension characterizes how a non-instruction sample is organized as a discourse object: whereas *content world* (Section 3.3.2) specifies what a sample is about, discourse structure specifies how that content is presented, connected, and made accessible to the learner, instantiating the second stage of the content-world → discourse-structure → operation-opportunity → supervision-density progression introduced in Section 2.3.2. We operationalize this dimension through two complementary subcategories: *discourse form*, which characterizes the sample’s external surface format, and *context scope*, which characterizes the internal dependency structure required to recover the signal carried by that form. Table 11 summarizes the fields and diagnostic roles of this dimension, and Table 12 provides representative annotated examples.

The discourse-form subcategory identifies the dominant discourse format of a visible text chunk, such as an article, dialogue, table, code fragment, or noisy mixed-format record. Different discourse forms expose the model to different surface conventions, structural regularities, and generation regimes, including turn-taking structure in dialogue, hierarchical organization in tables, or symbolic regularity in code. Because a single chunk may exhibit more than one coherent form, for example a worked example embedded in an expository article, we record one primary form together with up to two secondary forms. Taken together, the discourse-form subcategory provides the corpus’s primary signal for surface-level format composition, supporting form-stratified corpus analysis and form-targeted data supplementation.

Table 9: Content World annotation subcategories for non-instruction data.

Subcategory	Field	Diagnostic role
Domain	Primary coarse domain	Assigns the dominant high-level semantic domain of the sample from a fixed set of coarse domains. This field provides a low-dimensional signal for data mixture design, broad corpus stratification, and high-level distribution monitoring.
	Primary fine domain	Assigns the dominant fine-grained topical domain, nested under the primary coarse domain. This field supports detailed corpus analysis, domain-specific error attribution, diversity diagnostics, and targeted data supplementation.
	Secondary fine domains	Records up to three additional fine-grained domains when substantial secondary topics are present, capturing mixed-domain composition that a single dominant label would otherwise obscure, without weakening the primary topical assignment.
Entity–Concept	Entities	Extracts up to five salient, concrete referential items that are explicitly mentioned or strongly implied in the text. Entity labels support entity-centered clustering and help identify overrepresented or undercovered referential regions of the corpus.
	Concepts	Extracts up to five salient abstract knowledge units grounded in the text. Concept labels provide a reusable semantic signal for concept-level clustering, diversity analysis, and targeted coverage improvement.
Embedding	Text embedding	Encodes the sample into a fixed-dimensional vector using a lightweight embedding model. The resulting representation captures topical and referential similarity beyond what the discrete domain and entity–concept labels express, and supports similarity-based clustering, near-duplicate detection, and retrieval of related samples.

The context-scope subcategory diagnoses how much contextual integration is required to recover a sample’s main learnable signal, i.e., the specific knowledge, relation, process, comparison, or argument structure that the sample is judged to primarily convey, as opposed to a generic topic summary. We decompose this subcategory into three fields. The dependency-pattern field describes the type of relation linking different parts of the text, such as entity tracking, causal chaining, or comparison. The dependency-scope field describes the minimal span over which the signal is distributed, ranging from a single self-contained sentence to the full passage. The dependency-density field describes how frequently and how strongly such dependencies recur throughout the sample. Separating these three fields allows the form, extent, and prevalence of contextual dependency to be assessed independently. Taken together, the context-scope subcategory provides the corpus’s primary signal for the internal dependency structure of its samples, complementing the surface-level characterization given by discourse form and supporting dependency-aware data curation for capability slices.

3.3.4 Operation Opportunity

The *operation opportunity* dimension characterizes which capability-forming operations are explicitly exposed by a non-instruction sample. This dimension provides the most direct bridge to the solving-operation dimension of the evaluation sample taxonomy (Section 3.2.4): many evaluation-side solving operations can only be reliably strengthened if the training corpus exposes a corresponding operational pattern. This correspondence is nonetheless asymmetric. An evaluation sample requires the model to execute an operation in order to produce a correct answer, whereas a non-instruction sample merely exposes the textual conditions under which such an operation can be observed and, potentially, internalized through the language modeling objective. An assigned operation-opportunity label therefore indicates that a learnable operational pattern is explicitly present in the text, not that the operation has been supervised, exercised, or successfully acquired by the model. Table 13 summarizes the resulting taxonomy, and Table 14 provides representative annotated examples.

Table 10: Examples of Content World annotations

Text	Labels
Dynamic Music’s "Picture:" series is a document in time, showcasing an artist’s current creative output in order to allow the listener to visualise and understand the artist at a certain moment. It is not an album format, but far more than merely an EP. "Picture:" is various dancefloor-oriented facets of one producer, shaping the imagery that ultimately results in the listeners mind.	Primary coarse domain: culture_media_humanities Primary fine domain: Music Secondary fine domains: Art Entities: Dynamic Music Picture: series Concepts: creative output album format EP dancefloor-oriented music
Established in 1937 by Alfonso Morini, a former racer and skilled designer, Moto Morini gained global renown through its successful participation in international races. By the late 1950s, the company’s racing engines were integrated into its standard production motorcycles. The pinnacle of Moto Morini’s innovation and technological research came in 1973 with the triumph of the 3 1/2 model.	Primary coarse domain: lifestyle_consumption_recreation Primary fine domain: Transportation Secondary fine domains: History, Products Entities: Moto Morini Alfonso Morini 3 1/2 model Concepts: motorcycle manufacturing motorcycle racing technological research engine design

We operationalize this dimension as nine independent, multi-label operation types rather than as a single dominant label, since a sample may simultaneously expose several distinct operational patterns. For each type, we judge whether the corresponding pattern is explicitly present in the text; whenever a type is judged present, we assign a sub-type label that specifies its particular form, such as `comparison_or_ranking` under `comparison_contrast_and_exclusion` or `causal_relation_or_mechanism` under `temporal_causal_and_process_reasoning`.

The primary analytical value of this taxonomy lies in its correspondence to the solving-operation dimension of the evaluation sample taxonomy (Section 3.2.4): each operation type is named and defined so that it can be interpreted as a candidate source of training signal for one or more solving operations observed at evaluation time. For instance, the evaluation-side operation `fact_recall` corresponds most naturally to `fact_or_entity_relation` or `concept_definition_or_explanation` under `knowledge_and_concept_grounding`, since both describe a sample whose content is grounded in a single recalled fact or definition rather than in contextual evidence or derivation. Similarly, `compare_or_rank` corresponds most naturally to `comparison_or_ranking` or `contrastive_distinction` under `comparison_contrast_and_exclusion`, since both describe a sample structured around evaluating two or more entities, quantities, or options against one another. We emphasize that such correspondences are intuitive and heuristic rather than definitive or universally guaranteed: a given solving operation may plausibly be supported by more than one operation-opportunity sub-type, a given sub-type may relate to more than one solving operation, and the alignment need not hold for every individual sample. The mapping is intended as a practical starting point for connecting evaluation-side failure diagnosis to data-side retrieval criteria, rather than as a formal or one-to-one equivalence between the two label spaces. Taken together, the nine operation types provide the corpus’s primary signal for the latent reasoning and processing opportunities embedded in non-instruction text, complementing the topical coverage captured by content world and the structural organization captured by discourse structure.

3.3.5 Supervision Density

The *supervision density* dimension characterizes how effectively a non-instruction sample can serve as training signal under the language modeling objective, jointly considering whether its surface form is clean and recoverable and whether its underlying content is substantive, verifiable, and transferable. This dimension instantiates the fourth and final stage of the content-world → discourse-structure → operation-opportunity → supervision-density progression introduced in Section 2.3.2: whereas the preceding three perspectives jointly characterize what a sample is about, how it is organized, and what operations it exposes, supervision density asks a logically prior question—whether the resulting signal is

Table 11: Discourse Structure annotation subcategories for non-instruction data.

Subcategory	Field	Diagnostic role
Discourse Form	Primary form	Identifies the sample’s dominant surface format (e.g., <code>narrative_prose</code> , <code>dialogue_or_transcript</code> , <code>code_or_api_spec</code>), independently of the domain-level content captured by <i>content world</i> : the same form can host many domains, and the same domain can appear in many forms.
	Secondary forms	Records up to two additional coherent formats present in the sample (e.g., a <code>procedural_or_howto</code> segment embedded in an <code>expository_article</code>), capturing mixed-format composition that a single primary label would otherwise obscure.
Context Scope	Dependency patterns	Identifies the type of relation linking different parts of the text, such as <code>entity_tracking</code> , <code>causal_chain</code> , or <code>comparison_contrast</code> .
	Dependency scope	Measures the minimal span required to recover the main learnable signal, ranging from <code>standalone</code> and <code>local_span</code> to <code>multi_sentence</code> and <code>whole_passage</code> dependency.
	Dependency density	Measures how pervasively dependency recurs within the sample, on a four-point scale from 0 (almost no contextual dependency) to 3 (dense, pervasive dependency).

usable as next-token supervision at all. We operationalize this dimension through two complementary subcategories, *cleanliness* and *knowledge density*, corresponding respectively to the recoverability and the informativeness of the underlying signal. The two subcategories form a sequential gate rather than two independent scores: a sample must first pass a cleanliness check before its knowledge density can be meaningfully assessed, since knowledge density presupposes that the underlying content has already been recovered from the surface text. Table 15 summarizes the annotation fields and diagnostic roles of these two subcategories, and Table 16 provides representative annotated examples.

The cleanliness subcategory is designed to determine whether a corpus segment contains a usable learning signal once extraction noise and surface corruption are accounted for. The noise-profile field separates distinct noise sources, including boilerplate navigation, formatting or markup artifacts, and OCR or encoding corruption, among others. This separation is useful because different noise patterns imply different data actions: boilerplate-heavy samples may call for source-level filtering, whereas OCR corruption may call for preprocessing repair. Building on this profile, the overall text-cleanliness label summarizes whether the main content is mostly clean, partially recoverable, or heavily corrupted, while the learnable-signal field separately records whether the sample still carries coherent language, factual knowledge, reasoning structure, code, mathematics, dialogue, narrative, or other domain-specific content despite its surface noise. This separation matters because a sample with noticeable formatting residue can still expose a strong underlying signal; the two labels are therefore allowed to diverge rather than being collapsed into a single composite judgment. Taken together, the cleanliness subcategory provides the corpus’s primary signal for whether a sample’s supervision is recoverable at all, supporting noise-aware filtering and preprocessing decisions.

The knowledge-density subcategory is designed to assess whether a clean or recoverable sample is also substantively informative. A formally clean text may still provide weak supervision if it consists mainly of generic promotion, shallow commentary, isolated titles, or low-value templates, whereas a technical document with minor formatting residue may still carry dense and useful learning signal. We therefore score knowledge density on an ordinal scale according to the amount of learnable, verifiable, and transferable information in the text, together with its structural organization, explanatory depth, and proportion of valid main content. Taken together, the knowledge-density subcategory provides the corpus’s primary signal for a sample’s substantive informativeness, complementing cleanliness’s focus on surface recoverability and supporting the targeted retention of high-value training segments.

Unlike content world, discourse structure, and operation opportunity, supervision density has no direct counterpart among the four dimensions used to decompose evaluation samples (Section 2.1). Evaluation samples are typically curated to be readable and well-formed, so the only partial analogue, the ambiguity-noise field under background condition, captures surface-level interference rather than the broader notions of extraction noise and informativeness that supervision density targets. Supervision density instead plays a corpus-internal role: it determines whether the

Table 12: Examples of Discourse Structure annotations

Text	Labels
In the wake of the 2014 midterm elections, Rush Limbaugh argues that the Republican Party has a significant mandate, whether they recognize it or not. He believes their victory, if achieved, will be primarily due to voters’ desire to stop President Barack Obama’s policies. This mandate is not one the Republicans have explicitly campaigned on, but rather, it’s the implicit reason behind voters’ decisions to support them. Limbaugh predicts that Obama, emboldened by the media’s support, will interpret a Republican victory as a national crisis, using it to justify his planned executive actions. Despite the Republican Party’s strategic silence during the campaign, Limbaugh asserts that a victory would grant them a substantial mandate to halt Obama’s policies and those of the Democratic Party.	Primary form: expository_article Secondary forms: opinion_or_argumentation Dependency scope: whole_passage Dependency density: 3 Dependency patterns: argument_chain causal_chain coreference_resolution
Painting water is a fairly complex subject. Water, is affected by its surroundings, reflections, depth and clarity. The painting of oceans, rivers, lakes and ponds can be beautiful, but managing to get the water and reflections to look like actual water can be challenging. Reflections in Muddy Water: Layin’ Drag on Life’s Highway in Cassville, Georgia Paperback – J by Brad Stephens (Author)/5(23). Claude Debussy’s Reflets dans l’eau ("Reflections in the Water") is the first of three piano pieces from his first volume of Images, which are frequently performed separately. Begin by focusing first on your subject; then, move on to your reflection in the water. This is going to give you subtly different results, nothing too drastic, but enough for you to decide which style you like better. The Poetry book I read was the Reflections on a Gift of a Watermelon pickle. Undercoat the whole water area with white and use plenty of paint. Blend your blue into the white from the bottom up while your white undercoat is still wet. This is the reflection of the sky dark blue at the bottom and white in the distance. Paint the. Reflection off of smooth surfaces such as mirrors or a calm body of water leads to a type of reflection known as specular reflection. Reflection off of rough surfaces such as clothing, paper, and the asphalt roadway leads to a type of reflection known as diffuse reflection.	Primary form: mixed_or_fragmented Secondary forms: qa_or_faq, procedural_or_howto Dependency scope: standalone Dependency density: 0 Dependency patterns: none

content-world, discourse-structure, and operation-opportunity signals identified for a given sample are actually learnable under the language modeling objective, rather than describing a condition to be matched against the evaluation side. In this sense, supervision density closes the four-perspective progression by gating, rather than extending, the signals captured by the preceding three perspectives.

3.3.6 Usage

The non-instruction data taxonomy serves as the data-side counterpart of the evaluation sample taxonomy described in Section 3.2.6. Whereas the evaluation taxonomy decomposes benchmark samples into structured capability demands, the non-instruction taxonomy decomposes raw corpus segments into structured data affordances. In this paper, we use the taxonomy in four complementary ways: to characterize the affordance composition of a corpus, to support quality-aware filtering and repair, to translate a diagnosed weak capability slice into a concrete data action, and to validate whether a data intervention has changed the intended part of the model’s behavior. These four uses share the same underlying mechanism, aggregating sample-level data labels into interpretable groups, but differ in whether the goal is corpus diagnosis, data selection, intervention design, or post-intervention attribution.

The first use is to characterize the affordance composition of a corpus. Rather than treating a training corpus as a mixture of sources or domains alone, we aggregate labels along the four data-side dimensions introduced above. This profile makes it possible to ask, for example, whether a corpus is dominated by standalone factual statements, whether it contains enough whole-passage dependency structures, whether comparison or causal-process patterns

Table 13: Operation Opportunity annotation taxonomy for non-instruction data.

Operation type and description	Sub-type label space
knowledge_and_concept_grounding The text provides factual statements, entity attributes, definitions, concept explanations, category relations, or example-to-concept mappings.	fact_or_entity_relation concept_definition_or_explanation taxonomy_or_category_relation example_to_concept_mapping other
evidence_localization_and_integration The text exposes local evidence anchors, distributed evidence, cross-reference resolution, structured evidence, or multi-hop relation chains.	local_evidence_anchor distributed_evidence cross_reference_resolution multi_hop_relation_or_structured_evidence other
comparison_contrast_and_exclusion The text compares, ranks, contrasts, distinguishes, negates, excludes, corrects, refutes, or provides counterexamples.	comparison_or_ranking contrastive_distinction negation_or_exclusion correction_refutation_or_counterexample other
temporal_causal_and_process_reasoning The text describes temporal order, event progression, causal relations, mechanisms, consequence chains, workflows, procedures, or process stages.	temporal_sequence_or_event_progression causal_relation_or_mechanism consequence_chain workflow_or_procedure other
quantitative_and_symbolic_reasoning The text exposes countable structures, numerical relations, calculations, formulas, equations, symbolic transformations, formal proofs, or logical arguments.	counting_or_enumeration arithmetic_or_quantity_relation equation_or_formula_application symbolic_derivation_or_formal_proof other
constraint_and_state_tracking The text contains multiple conditions, rules, constraints, dependencies, state changes, intermediate states, variables, or eligibility requirements that must be tracked consistently.	conditional_rule_or_requirement multi_constraint_interaction state_or_variable_tracking dependency_or_configuration_rule other
simulation_and_script_reasoning The text describes physical, spatial, tool-use, social, behavioral, or commonsense scenarios that expose state, action, role, intention, or event-continuation simulation patterns.	physical_or_spatial_simulation tool_use_or_manipulation social_interaction_script intention_emotion_or_commonsense_script other
algorithm_and_code_reasoning The text contains algorithmic logic, pseudocode, program structure, code snippets, execution traces, debugging information, input-output behavior, variable updates, data structures, or complexity analysis.	algorithm_or_pseudocode data_structure_or_complexity code_snippet_or_io_behavior execution_trace_or_debugging other
boundary_exception_and_failure_reasoning The text explicitly discusses edge cases, corner cases, invalid inputs, exception handling, failure modes, limitations, rule exceptions, robustness issues, or cases where a general rule does not apply.	edge_or_corner_case invalid_input_or_exception failure_mode_or_limitation rule_exception_or_robustness_issue other

are underrepresented, or whether some sources provide text that is clean but shallow. Such analysis offers a more operational view of corpus composition than domain statistics alone, since it describes not only what the corpus is about, but also what kinds of learning signal it can plausibly supply.

The second use is to support quality-aware filtering, repair, and retention. The supervision-density dimension is especially important for this purpose: cleanliness labels identify whether a sample is clean, partially recoverable, or dominated by extraction noise, while the noise profile separates distinct failure modes. These distinctions map to different data actions, for example, samples dominated by boilerplate or formatting noise may call for source-level filtering, whereas samples whose surface corruption is superficial but whose underlying content remains recoverable may instead call for preprocessing repair. Knowledge-density labels further distinguish substantive text from samples that are formally clean but provide little learnable, verifiable, or transferable information. This separation prevents

Table 14: Examples of Operation Opportunity annotations

Text	Labels
The Shining Bronze-Cuckoo is similar to Horsfield’s Bronze-Cuckoo, but it is more brightly marked with iridescent green-brown on the wings, back and neck. The barring on the chest of adult birds is complete, without the central break of the Horsfield’s Bronze-Cuckoo. The call is a whistle with an upward inflection – similar to a human whistling a dog. They feed mainly on insects and their larvae, especially hairy caterpillars.	Operation type: knowledge_and_concept_grounding comparison_contrast_and_exclusion Sub-type: fact_or_entity_relation contrastive_distinction
In the aftermath of natural disasters such as earthquakes or tsunamis, the paramount importance of sanitation becomes undeniably clear. Past events, notably the Great East Japan earthquake and the Kumamoto earthquake, illuminated the necessity for uninterrupted access to basic amenities like toilets. These pivotal experiences emphasised the dire need for a sanitation solution that is versatile, sustainable, and reliable, even amidst the unpredictabilities of crises.	Operation type: evidence_localization_and_integration temporal_causal_and_process_reasoning Sub-type: multi_hop_relation_or_structured_evidence causal_relation_or_mechanism

Table 15: Supervision Density annotation subcategories for non-instruction data.

Subcategory	Field	Diagnostic role
Cleanliness	Noise profile	Records the strength of major noise types, including boilerplate navigation, formatting or markup artifacts, and OCR or encoding corruption, among others. This field supports targeted filtering, preprocessing repair, and source-level quality auditing.
	Text cleanliness	Assigns an overall ordinal label indicating whether the main content is clean, partially recoverable, or heavily corrupted. This field provides a compact signal for corpus filtering and for separating usable samples from samples whose supervision signal is dominated by noise.
	Learnable signal	Assesses whether the sample still carries a coherent and useful learning signal after accounting for noise. This field prevents the mechanical discarding of samples that contain minor artifacts but still expose valuable language, knowledge, reasoning structure, code, mathematics, dialogue, or domain-specific content.
Knowledge Density	Knowledge density score	Assigns an ordinal score that reflects the amount of learnable, verifiable, and transferable information in the sample. This field provides the primary signal for distinguishing substantive content from text that is clean but shallow or low in value.

quality control from collapsing into a single scalar score: a sample may be surface-noisy but still carry a valuable learnable signal, or surface-clean but too shallow to justify retention. The taxonomy therefore supports filtering and repair decisions that are sensitive to both recoverability and informativeness.

The third use is to translate a diagnosed weak capability slice into a concrete data action. A weak capability slice is specified along the four evaluation-side dimensions introduced in Section 2.2; using the approximate correspondence between these dimensions and the four data-side dimensions established in Section 2.3.2, we translate this specification into a data affordance profile, namely the content world, discourse structure, operation opportunities, and supervision density that the supporting data should exhibit. The mapping rules detailed in Section 3.4 formalize this translation step. Depending on how well the existing corpus already satisfies the target profile, the resulting data action can take the form of retrieval from the existing corpus, reweighting of already available segments, targeted synthesis of underrepresented affordances, or filtering of distracting low-signal data.

The final use is post-intervention validation and attribution. After retrieval, reweighting, synthesis, filtering, or repair has been applied, we verify whether the intervention enriched the intended data affordances and whether the corresponding weak capability slice improved. This creates a symmetric analysis loop with the evaluation sample taxonomy: before

Table 16: Examples of Supervision Density annotations

Text	Labels
In 1902, renowned German opera singer Lilli Lehmann published "How to Sing", an illustrated guide detailing her singing techniques. She emphasized the importance of maintaining a long, resonating breath that vibrates both forward and backward in the mouth. The singer should focus on preparing a powerful, flexible, and mobile vocal apparatus, filling it entirely with a continuous vocal mixture. This mixture, composed of indistinguishable components, ensures a smooth and consistent tone.	Noise profile: none Text cleanliness: high Learnable signal: strong Knowledge density score: 3
Skip to main content Image An official website of the United States government Here's how you know Here's how you know Image Official websites use .gov A .gov website belongs to an official government organization in the United States. Image Secure .gov websites use HTTPS A lock (Lock Locked padlock icon) or https:// means you've safely connected to the .gov website. Share sensitive information only on official, secure websites. Home Close Search Image: Search 1-844-USAGOV1 • All topics and services • The U.S. and its government • Complaints • Disability services • Disasters and emergencies	Noise profile: boilerplate_navigation duplication_repetition Text cleanliness: low Learnable signal: weak Knowledge density score: 1

training, evaluation labels identify the weak capability slice and data labels define the data affordance profile; after training, evaluation labels decompose the score change by capability slice, while data labels confirm whether the intended corpus region was modified. This makes it possible to distinguish targeted improvements from unrelated benchmark-level fluctuations, to detect regressions on non-target slices, and to compare a structured intervention against simpler baselines such as domain-only supplementation or random data addition. In this sense, the non-instruction data taxonomy is not merely a corpus annotation scheme; it is the data-side instrument that enables the evaluation–data loop to move from diagnosis, to intervention, to controlled validation.

3.4 Evaluation-to-Data Mapping Rules

3.4.1 Motivation and Workflow

The evaluation sample taxonomy (Section 3.2) and the non-instruction data taxonomy (Section 3.3) characterize two complementary sides of the same underlying loop: the former specifies what capability demand a benchmark sample places on the model, while the latter specifies what learnable affordances a corpus segment can supply. Neither, on its own, indicates which data affordance is likely to remedy a given failure. The evaluation-to-data mapping rules are the bridge that connects the two sides.

This bridge is not equally direct on both ends. Because instruction data is itself decomposed, from its evaluation-side perspective (Section 2.3.3), along the same four dimensions used to characterize evaluation samples, a weak capability slice can specify a target instruction-data condition in essentially the same vocabulary as the slice itself; the mapping from evaluation samples to instruction data is therefore comparatively direct. Non-instruction data, in contrast, is characterized along a structurally different taxonomy whose correspondence to the evaluation-side dimensions is only approximate (Section 2.3.2); translating a weak capability slice into a data affordance profile on this side therefore requires an explicit, separately justified translation step. The mapping rules developed in this section are accordingly not restricted to the evaluation-sample-to-non-instruction-data direction: they span the bridge in its broader sense, with

Table 17: Taxonomy-aligned mapping rules between the evaluation sample taxonomy and the non-instruction data taxonomy.

Evaluation-side label	Data-side label	Mapping rationale
domain	domain	A capability weakness concentrated in a specific domain motivates retrieving, upweighting, or synthesizing corpus segments under the matching domain label.
discourse form	discourse form	The dominant information carrier of an evaluation sample, such as dialogue, exposition, a table, or a code specification, can be matched to corpus segments sharing the same discourse form.
context scope	dependency scope	The information span required to solve an evaluation sample is aligned with the dependency span over which a corpus segment carries its main learnable signal.
solving operation	operation opportunity	The operation an evaluation sample requires the model to perform is mapped to corpus segments that expose an analogous operation-opportunity type.

the comparatively direct instruction-data mapping and the more indirect non-instruction-data mapping forming its two ends.

Integrating these mapping rules into the conceptual framework yields a concrete five-step workflow that operationalizes the data–evaluation closed loop. A weak capability slice is first localized through the evaluation sample taxonomy and specified along background condition, task type, solving operation, and output constraint. The mapping rules then translate this slice into a data affordance profile: on the non-instruction-data side, a target content world, discourse structure, operation opportunities, and supervision density; on the instruction-data side, the corresponding background condition, task type, solving operation, and output constraint. Given this profile, a data action (retrieval, reweighting, targeted synthesis, or filtering) is selected according to how well the existing corpus already satisfies it. The resulting intervention is then applied and evaluated under controlled experimental conditions, and the outcome is analyzed by returning to the evaluation sample taxonomy to determine whether the targeted slice has improved. This workflow,

Weak Capability Slice → Data Affordance Profile → Data Action → Experimental Validation → Result Analysis,

is the operational form taken by the data–evaluation closed loop once the mapping rules below are in place.

Because the instruction-data side of this mapping reduces, in the limit, to restating the slice specification as a synthesis target in the vocabulary it already shares with evaluation samples (Section 2.3.3), it requires comparatively little additional translation machinery beyond that restatement. The remainder of this section therefore concentrates on developing rules for the more indirect, non-instruction-data side of the bridge: Section 3.4.2 introduces a set of explicit correspondences between dimensions that were aligned by design when the two taxonomies were constructed, and Section 3.4.3 introduces a complementary, LLM-assisted mechanism for slices that fall outside those correspondences.

Importantly, the mapping rules that drive this workflow are not intended to establish deterministic causal relations between training data and benchmark performance. A benchmark failure may arise from multiple, potentially compounding, factors, including insufficient knowledge coverage, missing reasoning patterns, weak format control, decoding effects, or scoring artifacts, and a single data affordance profile cannot be assumed to address all of them. We therefore treat mapping rules as structured heuristics for generating data-intervention hypotheses rather than as guarantees of improvement: given a diagnosed evaluation weakness, they indicate what kind of data signal should be inspected, retrieved, strengthened, or constructed.

3.4.2 Taxonomy-Aligned Mapping Rules

The mapping rules introduced here connect the evaluation sample taxonomy and the non-instruction data taxonomy through the dimensions that were intentionally aligned during taxonomy design (Section 3.3.1). We restrict the core mapping rules to four explicit, dimension-level correspondences that were built into the two taxonomies by design. Table 17 summarizes these correspondences.

The domain correspondence is the most direct of the four, since the fine-grained domain labels used on the data side are, by construction, identical to those used in the background condition dimension of the evaluation taxonomy (Section 3.3.2). If an evaluation weakness is concentrated in mathematics, code, or another explicitly annotated domain, the corresponding data-side domain label provides a first-pass retrieval or reweighting criterion. This mapping is useful but intentionally coarse, since domain alone carries no information about the operation or discourse pattern the data should supply. A weakness on mathematical evaluation samples, for instance, is equally consistent with needing more symbolic derivation, more equation formulation, more numerical computation, or more proof-like discourse, and domain matching alone cannot distinguish among these fine-grained data strategies; doing so requires the discourse-form and operation-to-opportunity correspondences introduced below.

The discourse-form correspondence maps the evaluation-side discourse form to its data-side counterpart, which was likewise designed to share a common label space (Section 3.3.3), and supports form-preserving retrieval and analysis. For example, weaknesses on table-based evaluation samples motivate inspecting list- or table-like corpus segments. The mapping remains approximate rather than exact, since naturally occurring corpus segments are typically noisier and more hybrid in form than curated benchmark samples, but it still provides a useful structural constraint beyond domain matching alone.

The context-scope correspondence maps context scope on the evaluation side to dependency scope on the data side (Section 3.3.3). The key criterion in both cases is not surface length but the scope of information dependency: an evaluation sample labeled `standalone` should be solvable primarily from the question itself, whereas a sample labeled `whole_passage` requires integrating information across the entire provided context; the corresponding data-side label records whether a corpus segment’s main learnable signal is local, multi-sentence, or passage-level. This correspondence is particularly useful for diagnosing failures that stem from long-range dependency use rather than from missing domain knowledge.

The operation-to-opportunity correspondence is the central mapping for data–evaluation analysis, linking what an evaluation sample requires the model to do with what a corpus segment gives the model the opportunity to learn (Section 3.3.4). Unlike the domain and discourse-form correspondences, this mapping is typically many-to-one or many-to-many. Table 18 provides representative examples.

Not every evaluation-side dimension admits a table-level non-instruction-data counterpart, and the four correspondences above are intentionally exhaustive under this constraint rather than a partial list awaiting completion. Output constraint has no non-instruction-data analogue at all, since raw corpus text carries no explicit answer slot or scoring rule for it to align with; task type, together with the language, parametric–contextual need, and ambiguity-noise subcategories of background condition, is likewise omitted here because it describes a property of an explicit task presentation rather than of a raw corpus segment. These remaining evaluation-side dimensions are instead addressed by the comparatively direct instruction-data mapping previewed in Section 3.4.1, since instruction data’s evaluation-side decomposition already specifies background condition, task type, solving operation, and output constraint in the same vocabulary used to define a weak capability slice.

3.4.3 LLM-Assisted Mapping Rules

The taxonomy-aligned mapping rules introduced in Section 3.4.2 are deliberately restricted to the four dimension-level correspondences that were built into the two taxonomies by design. This restriction makes the rules transparent and easy to audit, but it also means they cannot resolve cases that fall outside a clean one-to-one correspondence: a weak capability slice may combine several dimensions in a way no single table row captures, or it may depend on domain-specific judgment about which operation or discourse pattern is actually missing, as in the mathematics example discussed above. Resolving such cases requires semantic reasoning over the specific slice and its failed examples, rather than a fixed lookup. We therefore introduce LLM-assisted mapping rules as a complementary, more flexible mechanism: where the taxonomy-aligned rules provide a transparent default for the dimensions they cover, the LLM-assisted rules extend coverage to the remaining, more context-dependent cases within that same non-instruction-data scope.

Concretely, we use a strong LLM as a rule-instantiation assistant. Given a weak capability slice, the definitions of both taxonomies, a set of representative failed examples, and optionally corpus-level tag statistics, the LLM proposes a candidate data affordance profile for that slice, comprising target data-side labels, retrieval or reweighting criteria, synthesis specifications for the missing affordance, and validation slices against which the resulting intervention should later be checked.

This LLM-assisted step functions as a planning mechanism rather than as evidence: the LLM does not define the mapping system, and its suggestions are not treated as conclusions. Consistent with the broader treatment of mapping rules as heuristics rather than as causal claims (Section 3.4.1), its role is to resolve, on a case-by-case basis, the ambiguity that the taxonomy-aligned correspondences leave open, rather than to replace them. The resulting suggestions

Table 18: Mapping from evaluation-side solving operations to data-side operation-opportunity labels.

Evaluation-side solving operation	Data-side operation opportunity
fact_recall concept_alignment	knowledge_and_concept_grounding
span_extraction evidence_aggregation multi_hop_composition	evidence_localization_and_integration
compare_or_rank option_elimination	comparison_contrast_and_exclusion
causal_inference temporal_reasoning	temporal_causal_and_process_reasoning
counting arithmetic_computation equation_formulation symbolic_transformation	quantitative_and_symbolic_reasoning
constraint_tracking	constraint_and_state_tracking
physical_state_simulation social_script_simulation	simulation_and_script_reasoning
algorithm_design code_execution_simulation	algorithm_and_code_reasoning
boundary_case_reasoning	boundary_exception_and_failure_reasoning

are reviewed manually before use and, like all mapping-rule outputs, remain subject to experimental validation rather than being accepted at face value.

4 Case Study

4.1 Case Study 1: Output-Constraint Diagnosis Reveals an EOS Supervision Bug

4.1.1 Background and Initial Observation

This case study illustrates the closed-loop workflow introduced in Section 3: we localize a weak capability slice using the evaluation sample taxonomy, trace its root cause to a specific property of the continued pre-training pipeline, and validate the resulting fix under controlled experimental conditions.

We begin from a warm-start checkpoint and continue pre-training it for 200B tokens to obtain a checkpoint denoted *base*. The continued pre-training corpus combines open-source data and internally synthesized data processed through a unified internal data pipeline, using a mixture of 50% general-domain data, 30% mathematics data, and 20% code data.

Table 19 compares benchmark performance before and after this continued pre-training stage. For each benchmark, we report the relative change

$$\Delta_{\text{rel}} = \frac{s_{\text{base}} - s_{\text{warm}}}{s_{\text{warm}}} \times 100\%,$$

where s_{warm} and s_{base} denote the scores of the warm-start and base checkpoints, respectively.

Continued pre-training improves nearly every individual benchmark, with relative gains ranging from +1.32% on PIQA to +45.80% on MATH (Minerva). BBH is the sole exception: its score drops from 47.27 to 25.14, a relative change of −46.82%. Because this regression runs counter to the otherwise uniform improvement, a natural first hypothesis is that continued pre-training has degraded the model’s underlying multi-step reasoning capability, the capability BBH is designed to probe. We take this discrepancy as the entry point for the diagnostic analysis that follows, and examine whether the regression in fact reflects a loss of reasoning capability or, instead, a more localized failure that an aggregate benchmark score cannot distinguish.

Table 19: Benchmark performance of the warm-start and base checkpoints.

Benchmark	Warm-start	Base	Δ_{rel}
PIQA	78.13	79.16	+1.32%
HellaSwag	72.76	78.24	+7.53%
MMLU	60.43	66.21	+9.57%
TriviaQA	39.78	51.53	+29.54%
RACE	42.49	44.11	+3.81%
DROP	38.56	42.74	+10.84%
MMLU-Pro	28.50	38.96	+36.70%
BBH	47.27	25.14	-46.82%
C-Eval	63.22	68.50	+8.35%
CMMLU	65.21	70.90	+8.73%
AGIEval-CN	44.59	51.22	+14.87%
MBPP	34.00	45.40	+33.53%
HumanEval	28.05	32.32	+15.22%
GSM8K	53.60	68.54	+27.87%
MathQA	43.79	54.44	+24.32%
MATH (Minerva)	29.52	43.04	+45.80%

Table 20: BBH performance grouped by output-constraint attributes.

Dimension	Sample Count	Warm-start	Base	Δ_{rel}
Answer form				
mcq_single	4760	47.48	21.18	-55.39%
boolean	751	69.64	55.79	-19.89%
generated_text	500	9.20	1.80	-80.43%
exact_numeric	500	49.80	40.20	-19.28%
Answer space				
closed_set	5545	50.30	25.86	-48.59%
semi_open	763	34.99	24.64	-29.58%
open	203	10.84	7.39	-31.83%
Format rigidity				
3	5667	46.92	23.20	-50.55%
0	503	53.08	41.15	-22.48%
1	284	45.77	37.32	-18.46%
2	57	38.60	15.79	-59.09%
Exactness requirement				
3	6511	47.27	25.14	-46.82%
Scoring rule				
exact_match	6099	47.11	24.32	-48.38%
numeric_equivalence	412	49.76	37.38	-24.88%

4.1.2 Output-Constraint Regression on BBH

Because an aggregate BBH score conflates many distinct capability demands, we turn to the evaluation sample taxonomy to disentangle them along separately analyzable dimensions. Among these dimensions, output constraint is the one most directly coupled to the scored outcome, since it governs how a generated response is parsed, normalized, and ultimately judged correct or incorrect; we therefore initiate our diagnosis from this dimension. Specifically, we revisit the regression through the *output constraint* dimension of the evaluation sample taxonomy (Section 3.2.5), which characterizes each instance by its expected answer form, the size of its admissible answer space, the rigidity of its required output format, the strictness of its exactness requirement, and its scoring rule. Table 20 reports BBH accuracy aggregated over these output-constraint attributes, with Δ_{rel} denoting the relative change from the warm-start checkpoint to the base checkpoint.

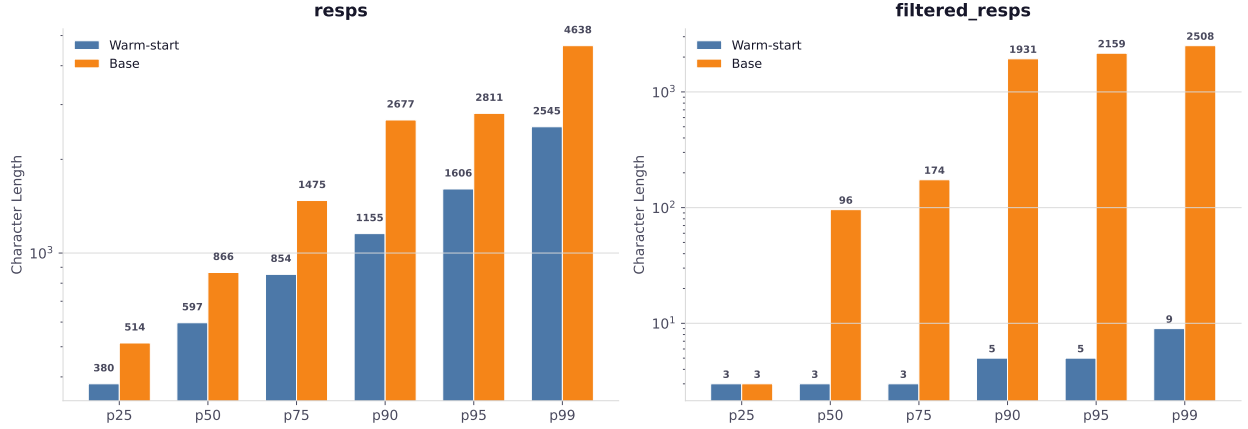


Figure 2: Character-length quantiles of `resps` and `filtered_resps` on the audited BBH subset, for the warm-start and base checkpoints.

This characterization confirms that, by construction, BBH is a strict-output benchmark: 87.04% of its instances carry the highest format-rigidity level (`format_rigidity` = 3), all instances carry the highest exactness requirement (`exactness_requirement` = 3), and 93.67% are scored by exact matching. Under this profile, BBH accuracy reflects not only whether the model can solve the underlying reasoning task, but also whether its final response conforms exactly to the output form the scorer accepts.

The degradation is broad across this strict-output regime: the base checkpoint underperforms the warm-start checkpoint for every observed value of every output-constraint attribute. The decline is most pronounced for `generated_text` answers, where accuracy falls from 9.20% to 1.80%, and for the dominant high-rigidity subset, where accuracy falls from 46.92% to 23.20%. This breadth suggests that the regression is not confined to a single task type, answer form, or scoring rule.

The closed-set and multiple-choice subsets, however, reveal a more diagnostic anomaly. Accuracy on `closed_set` instances drops from 50.30% to 25.86%, and accuracy on `mcq_single` instances drops from 47.48% to 21.18%. Because these instances draw answers from a finite, typically small set of options, a regression of this magnitude is difficult to attribute solely to increased reasoning difficulty under open-ended generation. This discrepancy motivates a prediction-level audit of parser-sensitive failures, to which we turn next.

4.1.3 Prediction-Level Audit of Parser-Sensitive Failures

To test this hypothesis directly, we conduct a prediction-level audit restricted to instances with `answer_form` = `mcq_single` or `answer_space` = `closed_set`, yielding a subset of 5,545 examples. For each example, we compare the raw model response, denoted `resps`, with the scorer-facing filtered response, denoted `filtered_resps`, i.e., the text obtained after the evaluation pipeline’s response-filtering logic, which is then passed to the downstream answer parser or exact-match scorer.

Figure 2 reports the character-length quantiles of `resps` and `filtered_resps` for both checkpoints. The base checkpoint produces substantially longer raw responses than the warm-start checkpoint, indicating that continued pre-training shifts generation behavior toward longer completions even on closed-set and multiple-choice BBH instances.

The more salient anomaly appears in the filtered responses. For the warm-start checkpoint, `filtered_resps` are characteristically short, consistent with the expected behavior on closed-set and multiple-choice tasks, where the filtered answer should reduce to a short option label or canonical answer. For the base checkpoint, however, the filtered-response length is abnormally large: the median length is 96 characters, the 75th percentile is 174 characters, and the 90th percentile reaches 1931 characters. Because the filtering stage is designed to extract a compact answer string rather than an extended continuation, this length distribution is itself evidence of an output-formatting irregularity rather than of a content-level reasoning failure.

We manually inspect examples with unusually long `filtered_resps`; Table 21 lists representative cases. The common failure pattern is that the model first emits the correct gold answer and then continues generating additional content beyond it. Because BBH applies strict exactness requirements and exact-match-oriented scoring, such trailing content can render an otherwise correct answer unacceptable to the parser.

Table 21: Representative examples of abnormal `filtered_resps` with excessive suffixes.

<code>resps</code>	<code>filtered_resps</code>	Golden answer
If today is 3/5, and it is Jane’s second time in the year 1973 to see a meteor shower, then today’s date is March 5, 1973. The date one week ago from today is March 5, 1973 minus 7 days, that is March 5, 1973 minus 7 days, that is March 5, 1973 minus 7 days, that is March 5, 1973 minus 7 days	[invalid]	(E)
If we look at (A), it says that working the printer is too complex for me, implying that the printer is difficult to work with. However, we know that printers are typically not complex to work with, and the speaker is likely being sarcastic. If we look at (B), it says that working the microprocessor is too complex for me, implying that the microprocessor is difficult to work with. ... So the answer is (A) and (B). I have a project that I’m working on that has a lot of different types of files. I’m trying to figure out how to organize them.	(A) and (B)	(A)
Here in this question, we are told that “John was allergic to the drug and he immediately went into shock and experienced cardiac arrest (a heart attack).” A typical person would therefore think that the misadministration of medication caused John’s premature death. So the answer is Yes.<begin_url>https://www.analystforum.com/t/calculating-expected-return/10116<end_url># Calculating expected return	Yes.<begin_url>https://www.analystforum.com/t/calculating-expected-return/10116<end_url># Calculating expected retur	Yes

Table 22: Gold-answer prefix and suffix diagnostics for `filtered_resps` on the audited BBH subset.

Metric	Warm-start	Base
Gold answer appears as prefix	50.39%	69.25%
Accepted when gold answer is prefix	99.82%	37.34%
Cases with extra suffix	0.18%	62.66%
Extra suffix length, mean $\pm$ std	0.01 $\pm$ 0.28	394.18 $\pm$ 733.48

To quantify this pattern, we measure two complementary statistics on the audited subset: the rate at which the gold answer occurs as a prefix of `filtered_resps`, and, conditional on this match, the amount of trailing content that follows it. Table 22 reports both statistics for the warm-start and base checkpoints.

As shown in the table, the base checkpoint achieves a higher prefix rate than the warm-start checkpoint (69.25% versus 50.39%), i.e., it reaches the correct answer string at least as often. However, among the cases where the filtered response begins with the gold answer, the base checkpoint’s prediction is accepted by the original scorer in only 37.34% of cases, compared with 99.82% for the warm-start checkpoint, indicating that the base checkpoint frequently produces the correct answer but fails to conform to the format the scorer accepts. This gap coincides with a marked increase in trailing content: only 0.18% of warm-start prefix cases contain extra characters after the gold answer, versus 62.66% for the base checkpoint, and the mean suffix length grows correspondingly, from 0.01 $\pm$ 0.28 to 394.18 $\pm$ 733.48 characters. A substantial fraction of base-checkpoint predictions therefore contain the correct answer as a prefix but append additional text that invalidates the prediction under strict exact matching.

A particularly frequent suffix pattern is content beginning with the special token `<begin_url>`, which marks the start of a web document in our pre-training data format and accounts for approximately 60% of the extra-suffix cases. Its presence suggests that the base checkpoint sometimes continues generating beyond the intended short answer into document-like text, rather than terminating once the required answer has been produced. This constitutes a concrete parser-sensitive failure mode: the model often identifies the correct closed-set answer, but the subsequent continuation violates BBH’s strict output contract and is therefore scored as incorrect, plausibly accounting for a large portion of the observed degradation. This failure pattern raises a further question, namely why the base checkpoint fails to terminate generation after producing an otherwise correct, well-formed answer; we examine this question next by inspecting the continued pre-training pipeline itself.

4.1.4 Missing Termination Supervision at Document Boundaries

The prediction-level audit in Section 4.1.3 identifies a recurring parser-sensitive failure mode on BBH: the base checkpoint frequently emits the correct answer string but then continues generating irrelevant content beyond it. The most common such suffix begins with the special token `<begin_url>`, accounting for approximately 60% of the extra-suffix cases. Because this token marks the start of a new web document in our pre-training data format, its recurrence as an unwanted continuation suggests that the failure may originate in how the model has learned to model document boundaries, rather than in the semantic content of the BBH tasks themselves.

Table 23: Benchmark performance of the warm-start, base, and exp checkpoints.

Benchmark	Warm-start	Base	Exp
PIQA	78.13	79.16	79.00
HellaSwag	72.76	78.24	78.42
MMLU	60.43	66.21	67.06
TriviaQA	39.78	51.53	51.43
RACE	42.49	44.11	43.35
DROP	38.56	42.74	46.47
MMLU-Pro	28.50	38.96	39.10
BBH	47.27	25.14	66.44
C-Eval	63.22	68.50	68.57
CMMMLU	65.21	70.90	71.29
AGIEval-CN	44.59	51.22	50.38
MBPP	34.00	45.40	47.40
HumanEval	28.05	32.32	42.07
GSM8K	53.60	68.54	71.34
MathQA	43.79	54.44	54.84
MATH (Minerva)	29.52	43.04	42.30

Motivated by this observation, we audit the pre-training pipeline from the perspective of document-boundary representation and termination supervision. In the training corpus, the `<EOS>` token accounts for 0.1% of all tokens, and manual inspection confirms that these tokens are correctly placed at the boundaries between consecutive documents. The serialized data therefore contains explicit and correctly positioned document-boundary markers; we find no evidence that the regression originates from a data-construction error in how boundaries are marked.

The training-code audit, however, reveals a mismatch between the serialized data and the optimization objective applied to it. Although `<EOS>` is present as a document separator in the input sequence, its loss is masked during continued pre-training. As a result, the model can attend to `<EOS>` tokens in its context, but it never receives direct next-token supervision to predict `<EOS>` itself. This creates a termination-supervision gap: the model continues to receive a dense training signal for modeling in-document content, but the objective never explicitly rewards predicting the termination token at the positions where generation should stop.

This gap provides a plausible root cause for the BBH regression observed under strict-output scoring. On closed-set and multiple-choice instances, the base checkpoint often reaches the correct answer, but the absence of explicit termination supervision makes it more likely to continue generating beyond the required answer span. When this continuation happens to contain document-start patterns such as `<begin_url>`, the filtered response no longer matches the format the scorer accepts, and an otherwise correct prediction is counted as incorrect. The BBH regression is therefore better characterized as a mismatch between the training objective and the strict termination requirements imposed by the evaluation protocol, rather than as a genuine degradation in the model’s underlying task-solving ability — directly addressing the hypothesis raised in Section 4.1.1.

4.1.5 Restoring EOS Supervision and Validating Failure-Mode Reduction

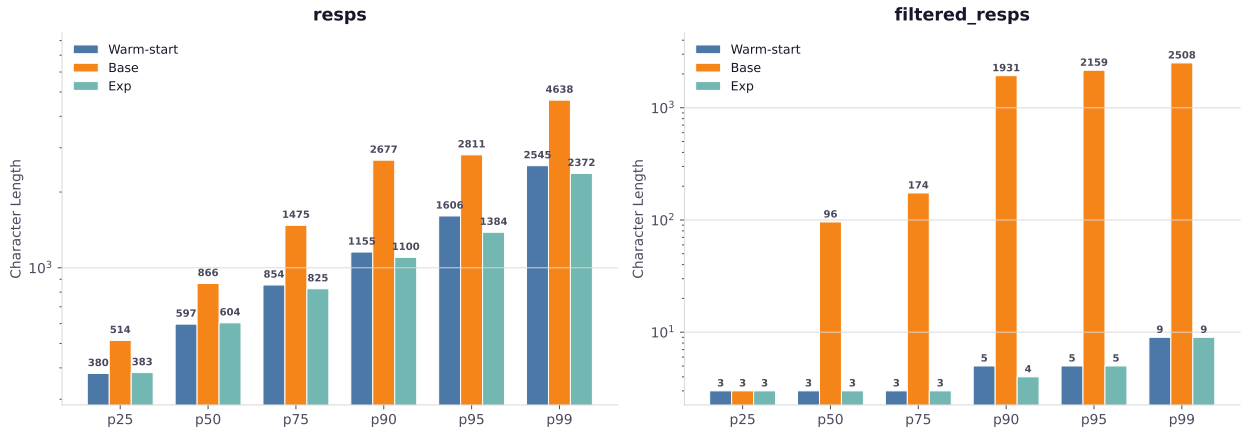
Motivated by this diagnosis, we design a controlled intervention that modifies only the loss mask used during continued pre-training. In the base training recipe, `<EOS>` appears in the serialized sequence as a document separator, but its loss is masked; in the intervention, we keep the data, model architecture, optimizer, and all other training hyperparameters unchanged, and restore loss computation and back-propagation on `<EOS>` alone. We denote the resulting checkpoint *exp*. This single-variable design isolates the effect of termination supervision from other potential confounds in the training recipe.

Table 23 reports benchmark-level performance for the warm-start, base, and exp checkpoints. The intervention preserves the broad gains obtained from continued pre-training while substantially reversing the BBH regression: BBH rises from 25.14% (base) to 66.44% (exp), surpassing not only the base checkpoint but also the original warm-start score of 47.27%. These results indicate that restoring `<EOS>` supervision corrects the strict-output generative regression without materially degrading performance on the other benchmark categories.

To verify that this improvement is concentrated where the diagnosis predicts, we re-examine BBH performance along the same output-constraint dimensions used in Section 4.1.2. Table 24 shows that the intervention improves performance across every observed value of every output-constraint attribute; here, Δ_{rel} is computed analogously to the definition in Section 4.1.1, but relative to the base checkpoint rather than the warm-start checkpoint. The largest gains occur precisely

Table 24: BBH performance after restoring <EOS> supervision, grouped by output-constraint attributes. Scores are reported as percentages.

Dimension	Sample Count	Base	Exp	Δ_{rel}
Answer form				
mcq_single	4760	21.18	66.28	+212.94%
boolean	751	55.79	87.62	+57.05%
generated_text	500	1.80	19.60	+988.89%
exact_numeric	500	40.20	83.00	+106.47%
Answer space				
closed_set	5545	25.86	69.09	+167.17%
semi_open	763	24.64	59.50	+141.48%
open	203	7.39	20.20	+173.34%
Format rigidity				
3	5667	23.20	65.29	+181.42%
0	503	41.15	80.72	+96.16%
1	284	37.32	68.31	+83.04%
2	57	15.79	45.61	+188.85%
Exactness requirement				
3	6511	25.14	66.44	+164.28%
Scoring rule				
exact_match	6099	24.32	65.29	+168.46%
numeric_equivalence	412	37.38	83.50	+123.38%

Figure 3: Character-length quantiles of `resps` and `filtered_resps` on the audited BBH subset, for the warm-start, base, and exp checkpoints.

on the subsets most affected by the parser-sensitive failure mode identified earlier: accuracy on `mcq_single` examples rises from 21.18% to 66.28%, accuracy on `closed_set` examples rises from 25.86% to 69.09%, and accuracy on the dominant high-rigidity subset (`format_rigidity` = 3) rises from 23.20% to 65.29%. This alignment between where the regression was concentrated and where the recovery is concentrated supports the hypothesis that a substantial part of the original BBH regression stemmed from a failure to terminate after the intended answer, rather than from a uniform loss of task-solving ability.

We then revisit the prediction-level diagnostics on the same audited subset of 5,545 BBH examples. Figure 3 compares the character-length quantiles of `resps` and `filtered_resps` across the warm-start, base, and exp checkpoints. For the raw `resps`, the exp checkpoint shortens generation length relative to base at every quantile, bringing the distribution back in line with the warm-start checkpoint (e.g., the 99th-percentile length falls from 4638 characters for base to 2372 characters for exp, close to the warm-start value of 2545). The effect is even more pronounced for `filtered_resps`: the median length falls from 96 characters for base to 3 characters for exp, matching the warm-start median, and the

Table 25: Prefix and suffix diagnostics on the audited BBH subset.

Metric	Warm-start	Base	Exp
Gold answer appears as prefix	50.39%	69.25%	69.31%
Accepted when gold answer is prefix	99.82%	37.34%	99.69%
Cases with extra suffix	0.18%	62.66%	0.31%
Extra suffix length, mean $\pm$ std	0.01 $\pm$ 0.28	394.18 $\pm$ 733.48	0.02 $\pm$ 0.42

90th percentile falls from 1931.40 characters for base to 4 characters for exp, close to the warm-start value of 5. This directly resolves the abnormal scorer-facing long outputs observed in the base checkpoint and indicates that restoring $\langle \text{EOS} \rangle$ supervision affects generation length broadly, rather than only at the specific point where the filtered answer is extracted.

Table 25 provides a more direct confirmation of the proposed mechanism. The rate at which the gold answer appears as a prefix of `filtered_resps` is essentially unchanged between base and exp (69.25% versus 69.31%), indicating that the intervention does not change how often the model reaches the correct answer. What changes is whether that correct answer is accepted: among prefix cases, scorer-accepted accuracy rises from 37.34% for base to 99.69% for exp, closely matching the warm-start rate of 99.82%. Correspondingly, the proportion of prefix cases with extra trailing characters falls from 62.66% to 0.31%, and the mean suffix length falls from 394.18 ± 733.48 characters to 0.02 ± 0.42 characters, both closely matching the warm-start statistics. The intervention therefore does not act by improving the model’s ability to solve BBH tasks; it acts by suppressing the unwanted continuations that previously caused otherwise correct answers to be rejected by the scorer.

Taken together, the controlled intervention supports the proposed root cause. Masking the loss on $\langle \text{EOS} \rangle$ leaves the model without direct supervision to terminate at document or answer boundaries, a gap that manifests as long, irrelevant continuations under strict-output evaluation protocols such as BBH. Restoring $\langle \text{EOS} \rangle$ supervision substantially reduces these continuations, recovers BBH performance beyond both the base and warm-start checkpoints, and improves several other generative benchmarks, while leaving the rest of the training pipeline unchanged. This case study illustrates the broader value of the analysis toolkit: an aggregate benchmark score that initially appeared to indicate a regression in reasoning capability was, on closer inspection through the output-constraint dimension, traced to a narrow and fully correctable issue in training-objective construction, rather than to a deficiency in the underlying data mixture or model capacity.

4.2 Case Study 2: Solving-Operation Diagnosis Reveals an Operation-Composition Bottleneck in Mathematical Reasoning

4.2.1 Background

We start from a warm-start checkpoint and evaluate it on two complementary benchmark suites. The first suite consists of 16 widely used benchmarks during pre-training, covering general language understanding, knowledge, mathematical reasoning, and code generation. As shown in Table 26, the checkpoint attains non-trivial performance on several general-purpose benchmarks, such as PIQA, HellaSwag, MMLU, C-Eval, and CMMLU. The same aggregate profile, however, also reveals a comparative weakness on reasoning-intensive mathematical tasks: the checkpoint scores 69.67 on GSM8K, 52.16 on MathQA, and 41.92 on MATH (Minerva).

This benchmark-level observation is informative but not directly actionable. A low score on a mathematical benchmark does not, by itself, indicate whether the underlying failure originates from missing mathematical knowledge, unstable arithmetic execution, weak equation formulation, or insufficient symbolic transformation. The benchmark name alone is therefore too coarse a unit to determine what type of training data should be added, reweighted, or otherwise targeted.

To obtain a more sensitive view of this weakness, we further evaluate the checkpoint under zero-shot Pass@K protocols on several hard mathematical problem-solving and code-generation benchmarks. Table 27 shows that the model attains low Pass@1 accuracy on these hard mathematical benchmarks: 7.47 on MATH500, 0.05 on AIME2025, and 0.00 on AIME2026. Although Pass@128 improves with larger sampling budgets on MATH500, the AIME results remain close to zero even at Pass@128, where the model is allowed 128 independent attempts per problem. This indicates that the weakness cannot be attributed to an unfavorable decoding configuration or an insufficient sampling budget: increasing the number of attempts does not surface a correct trajectory, because the model rarely assigns non-negligible probability to a valid reasoning path for these problems in the first place. The gap therefore reflects a limitation in the model’s underlying mathematical problem-solving capability rather than one that test-time scaling could resolve.

Table 26: Performance of the warm-start checkpoint on widely used benchmarks during pre-training.

Benchmark	Warm-Start
PIQA	78.24
HellaSwag	77.22
DROP	46.15
MMLU-Pro	37.61
BBH	60.51
MMLU	65.50
TriviaQA	48.28
RACE	44.69
C-Eval	67.61
CMMLU	69.44
AGIEval-CN	49.52
GSM8K	69.67
MathQA	52.16
MATH (Minerva)	41.92
MBPP	48.00
HumanEval	36.59

Table 27: Zero-shot Pass@K performance of the warm-start checkpoint on hard mathematical and code-generation benchmarks.

Benchmark	Pass@1	Pass@4	Pass@16	Pass@32	Pass@128
MATH500	7.47	22.39	46.12	57.94	76.00
AIME2025	0.05	0.21	0.83	1.67	6.67
AIME2026	0.00	0.00	0.00	0.00	0.00
HumanEval	18.22	37.71	58.14	67.62	84.76

These results motivate an operation-level diagnosis. Rather than treating mathematics as a monolithic domain, we apply the solving-operation dimension of the evaluation sample taxonomy (Section 3.2.4) to decompose this aggregate failure into more specific capability slices. This decomposition provides the bridge from benchmark-level weakness to data intervention: once the weak operations are identified, they can be translated, following the mapping rules introduced in Section 3.4, into corresponding data-side affordances and data action.

4.2.2 Operation-Level Diagnosis of the Mathematical Weakness

Benchmark-level results indicate that the warm-start model is weak at mathematical problem solving, but benchmark names alone are too coarse to identify which capability is weak. We therefore decompose the evaluation set using the solving-operation labels introduced in the evaluation sample taxonomy. Let $\mathcal{D}$ denote the evaluation set and let $L(x)$ denote the set of solving-operation labels assigned to sample $x \in \mathcal{D}$. For an operation subset O , we define the inclusive slice

$$\mathcal{D}(O) = \{x \in \mathcal{D} : O \subseteq L(x)\}.$$

A single-operation slice is defined by one operation, an operation-pair slice by a pair of operations, and an operation-triple slice by a triple of operations; under this inclusive definition, a sample with multiple labels contributes to every operation subset it contains. In addition, we report frequent exact operation sets, in which the full label set $L(x)$ matches a recurring operation combination, rather than merely containing it. For each slice, we report its support and accuracy, and treat the corresponding error rate as a descriptive indicator of weakness.

Operation-set coverage and raw error rates. We first aggregate samples from three mathematical evaluation benchmarks—GSM8K, MathQA, and MATH (Minerva)—comprising 9,304 items in total, organizing them by single-operation, operation-pair, operation-triple, and frequent exact operation sets. The single-operation statistics in Table 28 show that the most frequent operations are `arithmetic_computation`, `concept_alignment`, `multi_hop_composition`, and `equation_formulation`; together, these operations cover a large fraction of the mathematical evaluation distribution and therefore yield diagnostic slices with substantial support. Among the high-frequency operations, `constraint_tracking`, `boundary_case_reasoning`, `counting`, and

Table 28: Top-10 single-operation slices for the warm-start model. Slices are defined inclusively: a sample is counted if its solving-operation labels include the operation.

Operation	Count	Acc. (%)	Err. (%)
arithmetic_computation	8858	49.14	50.86
concept_alignment	5312	40.64	59.36
multi_hop_composition	5234	40.24	59.76
equation_formulation	4541	41.09	58.91
symbolic_transformation	2867	38.75	61.25
fact_recall	1961	47.02	52.98
constraint_tracking	1685	24.39	75.61
counting	970	33.51	66.49
boundary_case_reasoning	913	25.74	74.26
compare_or_rank	392	41.07	58.93

Table 29: Top-10 operation-pair slices for the warm-start model. Slices are defined inclusively: a sample is counted if its solving-operation labels include both operations in the pair.

Operation Pair	Count	Acc. (%)	Err. (%)
(arithmetic_computation, multi_hop_composition)	5074	40.52	59.48
(arithmetic_computation, concept_alignment)	4956	40.42	59.58
(arithmetic_computation, equation_formulation)	4399	40.99	59.01
(concept_alignment, multi_hop_composition)	3155	29.54	70.46
(concept_alignment, equation_formulation)	2932	34.62	65.38
(arithmetic_computation, symbolic_transformation)	2669	37.92	62.08
(equation_formulation, multi_hop_composition)	2569	29.08	70.92
(concept_alignment, symbolic_transformation)	2166	34.07	65.93
(equation_formulation, symbolic_transformation)	2131	32.94	67.06
(arithmetic_computation, fact_recall)	1793	45.79	54.21

symbolic_transformation exhibit particularly high error rates, suggesting that the weakness is not confined to numerical computation alone.

The operation-pair slices in Table 29 further show that the dominant mathematical samples typically require arithmetic computation to be combined with another operation, such as multi-hop composition, concept alignment, equation formulation, or symbolic transformation. Several high-support pairs have substantially higher error rates than arithmetic alone, especially pairs involving multi_hop_composition, equation_formulation, or symbolic_transformation. This indicates that the model’s mathematical weakness is better characterized as an operation-composition problem than as a failure of isolated arithmetic execution.

The operation-triple slices in Table 30 offer a more realistic view of difficult mathematical samples, since many benchmark problems require at least three operations to solve. The lowest-accuracy high-support triples concentrate around combinations of concept_alignment, equation_formulation, multi_hop_composition, and symbolic_transformation; these slices correspond to problems in which the model must align concepts, formulate mathematical relations, transform symbolic expressions, and sustain a multi-step derivation simultaneously.

Finally, Table 31 reports the most frequent exact operation sets. This view complements the inclusive single-operation, operation-pair, and operation-triple statistics by isolating recurring, fully specified operation configurations rather than operation subsets. The exact-set statistics show that lower-complexity combinations, such as arithmetic_computation alone or arithmetic_computation paired with concept_alignment, retain comparatively high accuracy, whereas exact sets that combine arithmetic computation, concept alignment, equation formulation, multi-hop composition, and symbolic transformation have markedly higher error rates. This contrast supports the hypothesis that the warm-start model’s mathematical weakness is driven by compositional derivation rather than by a uniform failure across all math-related samples.

Pairwise composition effects. The raw operation-set statistics identify candidate weak operations and recurring difficult combinations, but they do not quantify how the performance of an anchor operation changes once it is composed with another operation. We therefore analyze the pairwise effect of companion operations over high-frequency anchor

Table 30: Top-10 operation-triple slices for the warm-start model. Slices are defined inclusively: a sample is counted if its solving-operation labels include all three operations in the triple.

Operation Triple	Count	Acc. (%)	Err. (%)
arithmetic_computation concept_alignment multi_hop_composition	3007	29.50	70.50
arithmetic_computation concept_alignment equation_formulation	2806	34.25	65.75
arithmetic_computation equation_formulation multi_hop_composition	2496	29.01	70.99
arithmetic_computation equation_formulation symbolic_transformation	2006	32.10	67.90
arithmetic_computation concept_alignment symbolic_transformation	2003	33.05	66.95
concept_alignment equation_formulation multi_hop_composition	1912	24.27	75.73
concept_alignment equation_formulation symbolic_transformation	1710	29.82	70.18
arithmetic_computation multi_hop_composition symbolic_transformation	1616	25.37	74.63
equation_formulation multi_hop_composition symbolic_transformation	1468	23.98	76.02
concept_alignment multi_hop_composition symbolic_transformation	1429	23.44	76.56

operations. For an anchor operation u and a companion operation o_i , we compare the accuracy of the corresponding operation-pair slice against the accuracy of the single-operation slice:

$$\Delta_{\text{pair}}(u; o_i) = \frac{\text{Acc}(u, o_i) - \text{Acc}(u)}{\text{Acc}(u)} \times 100,$$

where $\text{Acc}(u)$ denotes the accuracy on $\mathcal{D}(\{u\})$, and $\text{Acc}(u, o_i)$ denotes the accuracy on $\mathcal{D}(\{u, o_i\})$. A negative value of $\Delta_{\text{pair}}(u; o_i)$ indicates that samples containing both u and o_i are harder than the average sample containing the anchor operation u . Conversely, a positive value indicates that the companion operation is associated with higher accuracy relative to the anchor baseline. The corresponding figures are provided in Appendix A.1.

The pairwise-effect results show a consistent pattern across the major mathematical anchors. For `arithmetic_computation`, whose single-operation accuracy is 49.14%, the addition of a relatively simple companion such as `fact_recall` causes only a moderate drop (−6.8%), whereas composition with `symbolic_transformation`, `counting`, `boundary_case_reasoning`, and `constraint_tracking` leads to much larger relative decreases of −22.8%, −31.2%, −50.6%, and −51.3%, respectively. This indicates that arithmetic is not uniformly weak in isolation; rather, it becomes substantially less reliable once embedded in symbolic, counting-intensive, boundary-sensitive, or constraint-heavy contexts.

A similar pattern holds for `equation_formulation` and `multi_hop_composition`. For `equation_formulation`, whose single-operation accuracy is 41.09%, adding `arithmetic_computation` has almost no effect (−0.3%), but

Table 31: Top-10 frequent exact operation sets for the warm-start model. Unlike the inclusive single-operation, operation-pair, and operation-triple slices, each row corresponds to a full recurring operation-label set.

Operation Combination	Count	Acc. (%)	Err. (%)
arithmetic_computation multi_hop_composition	1045	68.90	31.10
arithmetic_computation equation_formulation	638	60.19	39.81
arithmetic_computation concept_alignment equation_formulation	407	58.48	41.52
arithmetic_computation	388	74.48	25.52
arithmetic_computation concept_alignment equation_formulation multi_hop_composition symbolic_transformation	368	27.45	72.55
arithmetic_computation equation_formulation multi_hop_composition	342	49.42	50.58
arithmetic_computation concept_alignment multi_hop_composition	332	53.31	46.69
arithmetic_computation concept_alignment	328	70.43	29.57
arithmetic_computation concept_alignment equation_formulation multi_hop_composition	277	35.74	64.26
arithmetic_computation concept_alignment equation_formulation symbolic_transformation	237	55.70	44.30

adding multi_hop_composition, counting, constraint_tracking, and boundary_case_reasoning yields substantially larger drops of -29.2% , -37.9% , -47.2% , and -49.0% . For multi_hop_composition, whose single-operation accuracy is 40.24% , the companion arithmetic_computation is nearly neutral ($+0.7\%$), while symbolic_transformation, constraint_tracking, and boundary_case_reasoning induce large decreases of -35.8% , -46.7% , and -51.6% . These results suggest that multi-step reasoning failures are amplified when the model must simultaneously maintain formal transformations, constraints, or exceptional cases.

For symbolic_transformation, whose single-operation accuracy is 38.75% , adding fact_recall is associated with a positive relative change ($+5.6\%$), and adding arithmetic_computation is nearly neutral (-2.2%); in contrast, multi_hop_composition, constraint_tracking, and boundary_case_reasoning produce large drops of -33.4% , -41.9% , and -44.0% .

Overall, the pairwise-effect analysis refines the raw operation-set diagnosis in two ways. First, it shows that arithmetic_computation is not the principal source of difficulty in isolation: its weakness is amplified specifically when paired with symbolic transformation, counting, constraint tracking, or boundary-case reasoning. Second, it identifies constraint_tracking and boundary_case_reasoning as systematic difficulty amplifiers across the major mathematical anchors, including arithmetic computation, equation formulation, multi-hop composition, and symbolic transformation. These observations motivate the composition-effect analysis below, which tests whether operation-triple combinations exhibit non-additive degradation beyond what the pairwise effects alone would predict.

Non-additive composition effects. The pairwise effect measures how the accuracy of an anchor operation changes when it is composed with a single additional companion operation. It does not, however, determine whether adding two companion operations together produces an approximately additive difficulty or introduces further compositional interference. To make this distinction, we compute a composition effect for each high-frequency anchor operation u and companion pair (o_i, o_j) , using the anchor accuracy $\text{Acc}(u)$ as the normalization baseline. We first define the relative accuracy changes of the two operation-pair slices and the corresponding operation-triple slice:

$$\begin{aligned} r(u, o_i) &= \frac{\text{Acc}(u, o_i) - \text{Acc}(u)}{\text{Acc}(u)} \times 100, \\ r(u, o_j) &= \frac{\text{Acc}(u, o_j) - \text{Acc}(u)}{\text{Acc}(u)} \times 100, \\ r(u, o_i, o_j) &= \frac{\text{Acc}(u, o_i, o_j) - \text{Acc}(u)}{\text{Acc}(u)} \times 100. \end{aligned}$$

The composition effect is then defined as

$$C(u; o_i, o_j) = r(u, o_i, o_j) - (r(u, o_i) + r(u, o_j)).$$

A negative value of $C(u; o_i, o_j)$ indicates that the operation-triple slice performs worse than expected from the sum of the two pairwise relative changes, suggesting non-additive compositional difficulty. A value close to zero indicates that the triple’s difficulty is approximately additive under this metric. A positive value indicates that the full operation-triple slice is less harmful than the sum of the two pairwise effects, which may occur when the two companion operations introduce overlapping rather than independent difficulty. The composition-effect figures are provided in Appendix A.2.

The composition-effect analysis shows that non-additive degradation is not uniform across anchors. For `arithmetic_computation`, whose single-operation accuracy is 49.14%, most observed composition effects are modest: the strongest negative effects occur when `multi_hop_composition` is combined with `fact_recall`, `symbolic_transformation`, or `equation_formulation`, with composition effects of -9.5 , -8.0 , and -6.8 , respectively, while other pairs are close to additive and the pair `(equation_formulation, compare_or_rank)` even shows a small positive composition effect of $+1.8$. Thus, for arithmetic-centered slices, the additional triple-level difficulty is present but relatively bounded; the larger weakness observed in the pairwise analysis is mostly explained by pairwise composition rather than by strong higher-order interference.

The pattern differs for anchors that require comparison, counting, or symbolic manipulation. For `compare_or_rank`, several companion pairs produce large negative composition effects, especially `(equation_formulation, counting)` at -37.1 , `(equation_formulation, fact_recall)` at -32.8 , and `(symbolic_transformation, counting)` at -31.6 . For `counting`, the strongest negative composition effects similarly involve comparison-oriented companions: `(equation_formulation, compare_or_rank)` reaches -42.7 , and `(symbolic_transformation, compare_or_rank)` reaches -38.0 . These results indicate that counting and comparison become substantially more fragile when integrated with equation formulation or symbolic transformation, beyond what the corresponding pairwise difficulties alone would predict.

For `symbolic_transformation`, whose single-operation accuracy is 38.75%, most composition effects are negative. The largest degradation appears for `(fact_recall, compare_or_rank)`, with a composition effect of -33.9 , followed by `(counting, compare_or_rank)` at -15.0 . Arithmetic- and constraint-related pairs are closer to zero, such as `(arithmetic_computation, compare_or_rank)` at -0.9 and `(arithmetic_computation, constraint_tracking)` at -1.6 . This suggests that symbolic transformation is especially brittle when the sample additionally requires comparison or ranking, whereas arithmetic-centered symbolic combinations behave closer to additive.

By contrast, `multi_hop_composition` and `equation_formulation` show weaker higher-order interference in most arithmetic-related combinations. For `multi_hop_composition`, arithmetic-centered composition effects are close to zero, such as `(arithmetic_computation, counting)` at -0.4 and `(arithmetic_computation, equation_formulation)` at -0.9 , while some pairs are even positive, such as `(fact_recall, constraint_tracking)` at $+9.4$ and `(equation_formulation, fact_recall)` at $+9.3$. For `equation_formulation`, the majority of arithmetic-related composition effects are likewise near zero, while the largest negative effect is concentrated in `(fact_recall, compare_or_rank)` at -24.5 . Although `multi_hop_composition` and `equation_formulation` are weak under the raw and pairwise statistics, their higher-order failures are therefore not uniformly super-additive; the strongest non-additive effects arise from more specific operation mixtures rather than from arithmetic-centered combinations.

Overall, the composition-effect analysis refines the pairwise-effect findings above: some weaknesses are primarily pairwise, while others involve stronger higher-order interference. The most pronounced non-additive degradation

appears in slices involving counting, `compare_or_rank`, and `symbolic_transformation`, particularly when combined with `equation_formulation` or other comparison-oriented operations, whereas many arithmetic-centered operation triples remain close to additive relative to the anchor baseline. These findings support a targeted data profile that emphasizes not only arithmetic computation and equation formulation, but also operation combinations that require counting, comparison, symbolic manipulation, and multi-condition reasoning to be executed jointly.

4.2.3 Diagnosis-Aligned Targeted Sampling from Synthetic Instruction Data

The operation-level diagnosis in Section 4.2.2 characterizes the warm-start model’s mathematical weakness along three complementary levels: raw error rates over inclusive operation slices, pairwise degradation induced by companion operations, and non-additive degradation over operation triples. Rather than treating these three levels as separate descriptive statistics, we use them jointly to construct a targeted sampling policy over a pool of synthetic instruction data, so that the operation combinations most strongly associated with the diagnosed weakness are over-represented in the resulting training subset.

We reuse the notation introduced in Section 4.2.2. Let $\mathcal{D}$ denote the diagnostic mathematical evaluation set, with $N = |\mathcal{D}|$ samples; let $L(x_i)$ denote the solving-operation label set assigned to sample $x_i \in \mathcal{D}$; and let $c_i \in \{0, 1\}$ denote its correctness indicator. We define a *feature* F as an operation combination with $|F| \in \{1, 2, 3\}$, corresponding respectively to single-operation, operation-pair, and operation-triple slices, and write its inclusive slice as

$$\mathcal{D}(F) = \{x_i \in \mathcal{D} : F \subseteq L(x_i)\}. \quad (1)$$

Its support, accuracy, and error rate are given by

$$n(F) = |\mathcal{D}(F)|, \quad \text{Acc}(F) = \frac{1}{n(F)} \sum_{i: F \subseteq L(x_i)} c_i, \quad \text{Err}(F) = 1 - \text{Acc}(F). \quad (2)$$

Following the inclusive convention of Section 4.2.2, a sample carrying multiple solving-operation labels contributes to every feature it contains, so the number of samples containing F serves simultaneously as its support and its frequency in $\mathcal{D}$.

Building on these units, the sampling procedure proceeds through four stages. First, each feature F is assigned a diagnosis-aligned weakness score that combines its raw error rate, pairwise effect, and composition effect, weighted by a support-reliability factor. Second, a feature pool is constructed by retaining the highest-scoring features under support and composition filters. Third, synthetic instruction examples are matched to the retained features through their annotated solving-operation labels, and each matched example is assigned a sampling weight derived from its dominant feature’s score. Fourth, the matched pool is sampled via bounded-duplication weighted reservoir sampling, with a small portion of the budget reserved as a fallback to preserve coverage of unmatched examples.

Diagnosis-aligned weakness score. For each feature F , we compute three diagnostic signals that mirror the three levels of the operation-level diagnosis.

The first signal, *raw weakness*, measures the extent to which a slice’s error rate exceeds the overall diagnostic error rate:

$$A_{\text{raw}}(F) = [\text{Err}(F) - \text{Err}(\mathcal{D})]_+, \quad (3)$$

where $[z]_+ = \max(z, 0)$. This term is positive only when F is over-represented among errors.

The second signal, *pairwise-degradation weakness*, captures the anchor-level effects analyzed in Section 4.2.2. For an operation pair $F = \{u, o\}$, the pairwise effect Δ_{pair} is anchor-dependent, so we take the larger of the two anchor-directional degradations,

$$A_{\text{pair}}(\{u, o\}) = \max\left([-\Delta_{\text{pair}}(u; o)]_+, [-\Delta_{\text{pair}}(o; u)]_+\right), \quad (4)$$

so that the term is large whenever *either* operation becomes substantially less reliable once composed with the other. For single-operation and operation-triple features, $A_{\text{pair}}(F) = 0$.

The third signal, *non-additive composition weakness*, captures the higher-order interactions identified in Section 4.2.2. For an operation triple $F = \{u, o_i, o_j\}$, we use the composition effect C defined therein. Because the same unordered triple admits three possible anchor choices, we take

$$A_{\text{comp}}(F) = \max_{u \in F} [-C(u; F \setminus \{u\})]_+. \quad (5)$$

This term is positive only when the triple performs worse than the sum of its two pairwise effects would predict. For single-operation and operation-pair features, $A_{\text{comp}}(F) = 0$.

To prevent low-support features from being ranked highly on the basis of noisy estimates, we introduce a support-reliability factor,

$$\omega(F) = \log(1 + n(F)) \sqrt{\frac{n(F)}{n(F) + \kappa}}, \quad (6)$$

with shrinkage constant κ ; we set $\kappa = 50$ throughout, which attenuates features with support on the order of a few tens of samples while leaving high-support features essentially unscaled. The diagnosis-aligned weakness score is then

$$\text{Weak}(F) = \omega(F) (A_{\text{raw}}(F) + A_{\text{pair}}(F) + A_{\text{comp}}(F)), \quad (7)$$

where the three signals are combined with equal weight, and $\omega(F)$ is relied upon to stabilize the resulting ranking against support-driven noise. By construction, Eq. (7) directly operationalizes the diagnosis: high-error slices contribute through A_{raw} , operation pairs that destabilize their anchor contribute through A_{pair} , and triples whose three operations interact super-additively contribute through A_{comp} . This score also serves, in the matching step below, as the basis for each matched example’s sampling weight.

Feature-pool construction. The score in Eq. (7) is defined for features of any order, but enumerating all triples is both computationally wasteful and statistically unreliable: many triples carry no weakness beyond their constituent pairs, and many low-support triples yield unstable composition-effect estimates. We therefore construct the feature pool $\mathcal{F}$ in tiers.

We first score all single operations and all operation pairs, retaining the highest-scoring pairs by $\text{Weak}(F)$. Triples are then formed by extending the retained pair structure and are filtered according to two criteria. A *support filter* removes triples with insufficient evidence,

$$n(T) \geq \theta, \quad \theta = 30, \quad (8)$$

which guards against unreliable estimates of the composition effect. A *non-additive composition filter* requires the triple to exhibit super-additive degradation relative to its pairwise sub-features,

$$A_{\text{comp}}(T) > 0, \quad (9)$$

so that triples whose difficulty is fully explained by their constituent pairs are discarded as redundant. Surviving triples are scored using Eq. (7), and the top- K ($K = 30$) are retained. The final feature pool $\mathcal{F}$ comprises all single operations together with the retained pairs and triples, each carrying its diagnosis-aligned weakness score.

Matching synthetic instruction data to diagnostic features. Let $\mathcal{T}$ denote the candidate pool of synthetic instruction examples after standard quality, deduplication, and contamination filtering. Each example $s \in \mathcal{T}$ is annotated with a solving-operation label set $L(s)$ under the taxonomy introduced in Section 3.2.4.

A synthetic example matches a feature $F \in \mathcal{F}$ when $F \subseteq L(s)$, and its hit set is

$$\text{Hits}(s) = \{F \in \mathcal{F} : F \subseteq L(s)\}. \quad (10)$$

Examples for which $\text{Hits}(s) \neq \emptyset$ form the *matched* pool $\mathcal{T}_m$, and the remainder form the *unmatched* pool $\mathcal{T}_u$.

For attribution purposes, each matched example is assigned to a single dominant feature $F^*(s)$ via a two-step rule that gives precedence to higher-order features. Let

$$r^*(s) = \max_{F \in \text{Hits}(s)} |F|$$

denote the maximum feature order present in the hit set, and let

$$\text{Hits}^*(s) = \{F \in \text{Hits}(s) : |F| = r^*(s)\}$$

denote the subset of hit features of that maximum order. The dominant feature is then the highest-scoring feature within this subset,

$$F^*(s) = \arg \max_{F \in \text{Hits}^*(s)} \text{Weak}(F),$$

with any residual ties broken by an arbitrary fixed ordering over features. This rule ensures that every matched example is attributed to exactly one feature: higher-order operation combinations take precedence as the more informative attribution, and the weakness score resolves ambiguity within a given order.

Beyond reporting, this dominant feature also defines each matched example’s sampling weight: we set

$$w_s = \text{Weak}(F^*(s)), \quad (11)$$

so that an example inherits the weakness score of the single most diagnostic feature it exhibits, rather than an aggregate over all features in $\text{Hits}(s)$. This weight is used directly by the weighted reservoir sampling procedure described below.

Bounded weighted sampling with coverage fallback. Let N_{target} denote the desired number of sampled examples, and let $n_m = |\mathcal{T}_m|$ and $n_u = |\mathcal{T}_u|$ denote the sizes of the matched and unmatched pools, respectively. We allocate the sampling budget by first drawing as many matched examples as possible, up to a fraction $\rho \in (0, 1]$ of the target, and then filling any residual budget from the unmatched pool. Let $d_{\text{max}} = 2$ denote the per-example duplication bound. The matched quota is

$$Q_{\text{match}} = \min(\lceil \rho N_{\text{target}} \rceil, n_m d_{\text{max}}), \quad (12)$$

where the cap $n_m d_{\text{max}}$ ensures that no matched example is selected more than d_{max} times. The unmatched draw absorbs only the residual budget, capped by the unmatched-pool size:

$$Q_{\text{cov}} = \min(N_{\text{target}} - Q_{\text{match}}, n_u), \quad (13)$$

where ρ controls the share of the budget reserved for the diagnosis-aligned matched draw. In our experiments, we set $\rho = 1.0$, directing the entire budget to the matched pool so as to up-sample the diagnosed weak operation combinations as aggressively as the duplication bound allows; the unmatched draw activates only as a fallback when $n_m d_{\text{max}} < N_{\text{target}}$, in which case $Q_{\text{cov}} = N_{\text{target}} - n_m d_{\text{max}}$ examples are drawn uniformly from $\mathcal{T}_u$ to preserve distributional coverage. Smaller values of ρ would trade matched intensity for guaranteed coverage of operation combinations outside the diagnosed feature pool; we leave this trade-off to future work.

We draw the Q_{match} matched examples using Efraimidis–Spirakis weighted reservoir sampling without replacement Efraimidis and Spirakis [2006], using the sampling weight $w_s = \text{Weak}(F^*(s))$ defined in Eq. (11). Each matched example s contributes up to d_{max} candidate slots; for each slot, we draw $u \sim \text{Uniform}(0, 1)$ and assign the key

$$k_s = -\frac{\log u}{w_s}. \quad (14)$$

The Q_{match} slots with the smallest keys are retained. Since $-\log u \sim \text{Exp}(1)$, the key follows an exponential race with rate w_s , so larger weights yield stochastically smaller keys and a correspondingly higher selection probability; selecting over per-slot keys realizes sampling without replacement in which each matched example appears at most d_{max} times. The Q_{cov} coverage examples, if any, are drawn uniformly at random from $\mathcal{T}_u$. The final sampled subset is

$$\mathcal{S} = \text{WeightedSample}(\mathcal{T}_m; w_s, Q_{\text{match}}, d_{\text{max}}) \cup \text{UniformSample}(\mathcal{T}_u; Q_{\text{cov}}), \quad (15)$$

with $|\mathcal{S}| = N_{\text{target}}$ whenever $n_m d_{\text{max}} + n_u \geq N_{\text{target}}$.

4.2.4 Controlled Experiment and Benchmark-Level Validation

Experimental setup. We validate the targeted-sampling intervention of Section 4.2.3 under a single-variable design. Starting from the same warm-start checkpoint analyzed in Section 4.2.1, we continue training under a fixed architecture and optimizer, performing continued pre-training (CPT) on a fixed budget of 100B tokens drawn from a mixture of 50% general-domain, 30% math, and 20% code data. Within the 30B-token math portion of this mixture, we sample $N_{\text{target}} = 10\text{B}$ tokens from a pool of 15B tokens of synthetic math instruction data and substitute them for an equal volume of the original math data, holding the overall 100B-token budget and the 50/30/20 composition fixed across runs; only the construction of this 10B-token instruction subset is varied. The *base* checkpoint is trained on a 10B-token subset drawn from the synthetic instruction pool without weakness-targeted reweighting, whereas the *exp* checkpoint is trained on the importance-sampled 10B-token subset produced by the procedure of Section 4.2.3, using the same target size N_{target} and the same duplication bound d_{max} . Because the two runs differ only in how this subset is sampled from the same 15B-token pool, any systematic difference between base and exp is attributable to the targeting itself rather than to the amount of instruction data or to the training recipe.

We evaluate all three checkpoints along three complementary axes. The first is the standard 16-benchmark suite used in Section 4.2.1 (Table 32). The second is the zero-shot Pass@K protocol on MATH500, AIME2025, AIME2026, and HumanEval (Table 33), evaluated under a fixed decoding configuration across checkpoints. The third is a slice-level re-evaluation along the solving-operation dimension.

Standard-suite results. On the standard suite (Table 32), the targeted intervention produces modest gains concentrated on the mathematical benchmarks while leaving general-domain performance essentially unchanged. Relative to the warm-start checkpoint, exp improves MathQA from 52.16 to 53.27 and MATH (Minerva) from 41.92 to 42.82, in both cases exceeding the base checkpoint as well; GSM8K is the exception, decreasing slightly from 69.67 to 68.61, even though base itself improves on this benchmark (69.90). Across the eleven general-language, knowledge, and reading-comprehension benchmarks, exp remains within approximately one point of warm-start in most cases, the largest movements being gains on DROP (+2.75) and BBH (+2.15) and a decrease on RACE (−0.96); we observe no broad regression of the kind that would indicate the intervention had displaced general capability. The code benchmarks

Table 32: Performance of the warm-start, base, and exp checkpoints on widely used benchmarks during pre-training.

Benchmark	Warm-start	Base	Exp
PIQA	78.24	78.73	78.45
HellaSwag	77.22	77.37	77.50
DROP	46.15	49.62	48.90
MMLU-Pro	37.61	37.74	37.92
BBH	60.51	62.46	62.66
MMLU	65.50	65.48	65.35
TriviaQA	48.28	48.92	48.53
RACE	44.69	43.44	43.73
C-Eval	67.61	67.53	67.38
CMMLU	69.44	70.21	70.22
AGIEval-CN	49.52	49.59	50.84
GSM8K	69.67	69.90	68.61
MathQA	52.16	52.53	53.27
MATH (Minerva)	41.92	42.48	42.82
MBPP	48.00	47.80	46.00
HumanEval	36.59	37.80	40.85

Table 33: Zero-shot Pass@K performance of the warm-start, base, and exp checkpoints on hard mathematical and code-generation benchmarks.

Benchmark	Pass@1			Pass@4			Pass@16			Pass@32			Pass@128		
	Warm-start	Base	Exp	Warm-start	Base	Exp	Warm-start	Base	Exp	Warm-start	Base	Exp	Warm-start	Base	Exp
MATH500	7.47	7.33	7.97	22.39	21.95	23.89	46.12	45.73	48.97	57.94	57.87	61.39	76.00	77.20	81.00
AIME2025	0.05	0.05	0.73	0.21	0.21	2.73	0.83	0.83	8.65	1.67	1.67	13.44	6.67	6.67	26.67
AIME2026	0.00	0.08	0.47	0.00	0.31	1.81	0.00	1.20	6.32	0.00	2.30	10.84	0.00	6.67	26.67
HumanEval	18.22	16.65	17.82	37.71	35.42	37.63	58.14	55.54	58.94	67.62	65.06	68.54	84.76	80.49	83.54

show a small, mixed effect: HumanEval improves from 36.59 to 40.85, while MBPP decreases from 48.00 to 46.00. Because these benchmarks are evaluated with limited sampling, they offer only a coarse view of the hard mathematical slices that motivated the intervention; we therefore turn first to the Pass@K evaluation below, and subsequently to the operation-level slice analysis on which the link between diagnosis and data construction in Section 4.2.3 ultimately rests.

Pass@K results on hard mathematical benchmarks. The Pass@K results in Table 33 reveal the intervention’s effect most clearly. On MATH500, exp improves over base at every sampling budget, and the gain widens with K , from +0.64 at Pass@1 (7.33 $\rightarrow$ 7.97) to +3.80 at Pass@128 (77.20 $\rightarrow$ 81.00). The effect is far more pronounced on the AIME benchmarks, which stress exactly the compositional operations flagged in Section 4.2.2. On AIME2025, base tracks warm-start at every budget (e.g., 6.67 at Pass@128), indicating that continued training without targeted sampling does not by itself surface valid trajectories on these problems; exp, in contrast, raises Pass@128 from 6.67 to 26.67 and lifts every lower budget correspondingly (e.g., Pass@16 from 0.83 to 8.65). On AIME2026, where warm-start scores 0.00 at every budget, exp reaches 0.47 at Pass@1 and 26.67 at Pass@128, again well above base (6.67 at Pass@128). Across all three benchmarks, exp exceeds base at every sampling budget, while base itself remains close to warm-start, varying by only a few points in either direction; this pattern indicates that the observed gains track the targeted sampling procedure itself rather than the continued-training recipe alone.

These results directly address the gap identified in Section 4.2.1, where the warm-start model failed to surface correct trajectories on AIME even at Pass@128 because it rarely assigned non-negligible probability to a valid reasoning path. After the intervention, additional sampling attempts do surface correct trajectories: the widening of the exp–base gap with increasing K is the expected signature of a model whose output distribution now places non-negligible probability mass on valid reasoning paths that were previously absent or vanishingly rare. The large gap between Pass@1 and Pass@K—for instance, 0.73 versus 26.67 on AIME2025—further suggests that the model has acquired the relevant compositional operations without yet consolidating them into its highest-probability completion, consistent with an intervention that broadens trajectory coverage rather than one that merely sharpens an already-correct mode. This distinction matters for how the operation-level attribution below should be read: an intervention that broadens trajectory

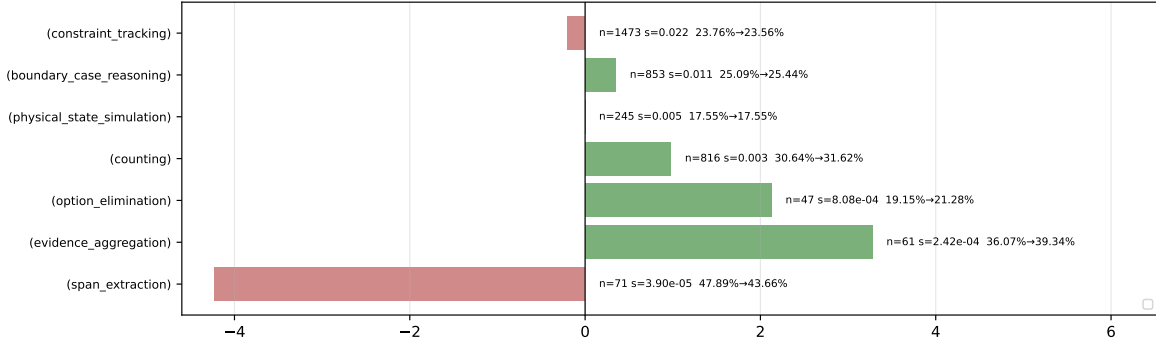


Figure 4: Operation-level accuracy change on the standard-suite mathematical evaluation set $\mathcal{D}$ (GSM8K, MathQA, and MATH (Minerva)), comparing the exp and base checkpoints across the top-7 retained single-operation slices in the diagnosis-aligned feature pool $\mathcal{F}$ of Section 4.2.2. The y-axis lists the single operation defining each inclusive slice $\mathcal{D}(F)$; the x-axis reports the accuracy change $\Delta\text{acc} = \text{acc}_{\text{exp}} - \text{acc}_{\text{base}}$, with bars to the right of zero (green) indicating an improvement of exp over base and bars to the left (red) indicating a regression. For each slice, n denotes its support $n(F)$ and s denotes its diagnosis-aligned weakness score, equivalently the sampling weight $w_s = \text{Weak}(F)$ assigned to matched examples whose dominant feature is F (Eq. (11)); the accompanying percentages report the base $\rightarrow$ exp accuracy transition, with the arrow indicating the direction of change. Mean $\Delta\text{acc} = +0.33\text{pt}$.

coverage is expected to produce a stronger signal under Pass@K-style evaluation than under Pass@1-style evaluation, and the operation-level contrast reported below realizes exactly this prediction.

4.2.5 Operation-Level Attribution of the Gains

Operation-level attribution on the standard suite. We now turn to the third evaluation axis introduced in Section 4.2.4: a slice-level re-evaluation along the solving-operation dimension that asks whether the benchmark-level gains reported above are concentrated on the operation combinations targeted by the sampling procedure of Section 4.2.3, or are instead distributed uniformly across the mathematical evaluation set. We restrict this analysis to the three mathematical benchmarks of the standard suite, GSM8K, MathQA, and MATH (Minerva), and reuse the diagnostic evaluation set $\mathcal{D}$ and the operation-feature definitions introduced in Section 4.2.2. For each retained feature F in the diagnosis-aligned feature pool $\mathcal{F}$ of Section 4.2.3, comprising single operations, operation pairs, and operation triples, we report its support $n(F)$, its diagnosis-aligned weakness score, and the accuracy change

$$\Delta\text{acc}(F) = \text{Acc}_{\text{exp}}(F) - \text{Acc}_{\text{base}}(F)$$

on the inclusive slice $\mathcal{D}(F)$, where $\text{Acc}_{\text{exp}}(F)$ and $\text{Acc}_{\text{base}}(F)$ denote the slice accuracy of the exp and base checkpoints introduced in Section 4.2.4.

The single-operation comparison in Figure 4 shows a positive but modest mean change of $+0.33\text{pt}$ across the seven retained single-operation slices. Among the operations whose participation in higher-order compositional features made them targets of the sampling policy, counting gains $+1.0\text{pt}$ on 816 samples and boundary_case_reasoning gains $+0.4\text{pt}$ on 853 samples; the largest absolute movements occur on low-support slices whose operations fall outside the diagnosed compositional core, namely evidence_aggregation ($+3.3\text{pt}$ on 61 samples) and option_elimination ($+2.1\text{pt}$ on 47 samples), which we treat as informative about direction rather than magnitude given their limited support. constraint_tracking, the slice with both the largest support and the highest weakness score among the single-operation features, regresses by -0.2pt on 1,473 samples, and the largest single negative movement is a decrease of -4.2pt on span_extraction ($n = 71$), an operation that does not appear among the high-scoring pair or triple features retained in $\mathcal{F}$. This pattern is consistent with the diagnosis of Section 4.2.2, which attributed the warm-start weakness to operation composition rather than to uniform single-operation failure: because the sampling policy of Eq. (7) places most of its mass on pair and triple features, we expect the slice-level signal at the single-operation level to be correspondingly small.

The operation-pair comparison in Figure 5 gives a more textured picture, with a mean change of $+0.16\text{pt}$ across the ten highest-scoring pairs. The largest gain, (boundary_case_reasoning, multi_hop_composition) at $+1.6\text{pt}$ on 694 samples, falls on a slice that the diagnosis flagged for both substantial pairwise degradation and low absolute accuracy; the second-largest gain, (concept_alignment, multi_hop_composition) at $+0.8\text{pt}$ on 2,615 samples, falls on the highest-support compositional slice in the panel. The slices that fail to improve all involve

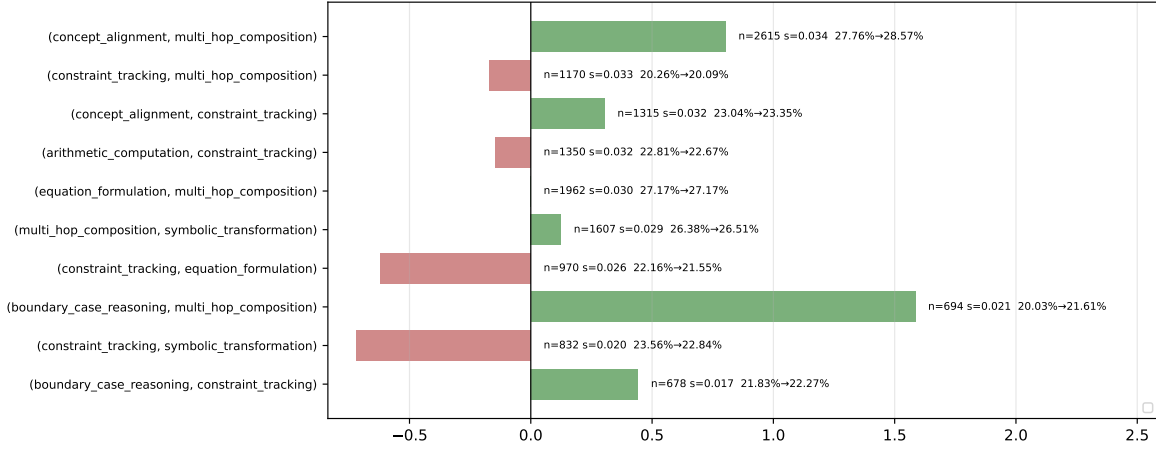


Figure 5: Operation-level accuracy change over the top-10 highest-scoring operation pairs in $\mathcal{F}$; axes, bar coloring, and annotation conventions as in Figure 4. Mean $\Delta\text{acc} = +0.16\text{pt}$.

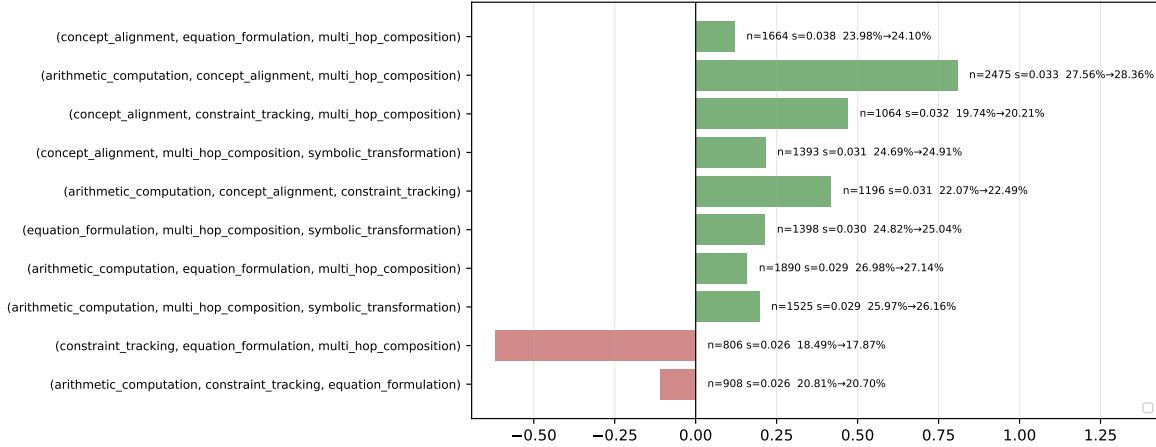


Figure 6: Operation-level accuracy change over the top-10 highest-scoring operation triples in $\mathcal{F}$; conventions as in Figure 4. Mean $\Delta\text{acc} = +0.19\text{pt}$.

constraint_tracking as one of the two operations: (constraint_tracking, symbolic_transformation) regresses by -0.7pt on 832 samples, (constraint_tracking, equation_formulation) by -0.6pt on 970 samples, (constraint_tracking, multi_hop_composition) by -0.2pt on 1,170 samples, and (arithmetic_computation, constraint_tracking) by -0.1pt on 1,350 samples, with (equation_formulation, multi_hop_composition) essentially flat on 1,962 samples. Of the six constraint-tracking-bearing pairs in the panel, only (boundary_case_reasoning, constraint_tracking) ($+0.4\text{pt}$ on 678 samples) and (concept_alignment, constraint_tracking) ($+0.3\text{pt}$ on 1,315 samples) improve at the pair level. We return to this constraint-tracking asymmetry below.

The operation-triple comparison in Figure 6 confirms the picture observed at the pair level. The mean change is $+0.19\text{pt}$ across the ten highest-scoring triples, and eight of the ten triples improve over base. The largest gains involve multi-step compositional derivation: (arithmetic_computation, concept_alignment, multi_hop_composition) gains $+0.8\text{pt}$ on 2,475 samples, (concept_alignment, constraint_tracking, multi_hop_composition) gains $+0.5\text{pt}$ on 1,064 samples, and (arithmetic_computation, concept_alignment, constraint_tracking) gains $+0.4\text{pt}$ on 1,196 samples. The two triples that regress, (constraint_tracking, equation_formulation, multi_hop_composition) (-0.6pt on 806 samples) and (arithmetic_computation, constraint_tracking, equation_formulation) (-0.1pt on 908 samples), both pair constraint_tracking with equation_formulation as their shared core, and both

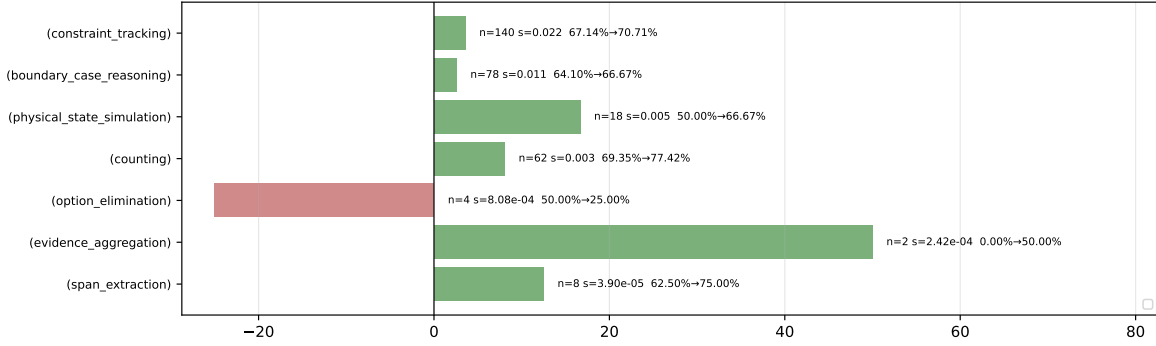


Figure 7: Operation-level accuracy change on MATH500 under the zero-shot Pass@128 protocol, over the same top-7 single-operation slices as in Figure 4. Axes, bar coloring, and the $n/s/\text{arrow}$ annotation conventions follow Figure 4, with Δacc and the base→exp transition now computed under Pass@128. Mean $\Delta\text{acc} = +9.77\text{pt}$; this all-slice mean is dominated by very low-support slices, and the mean restricted to the three slices with $n \geq 50$ is $+4.74\text{pt}$.

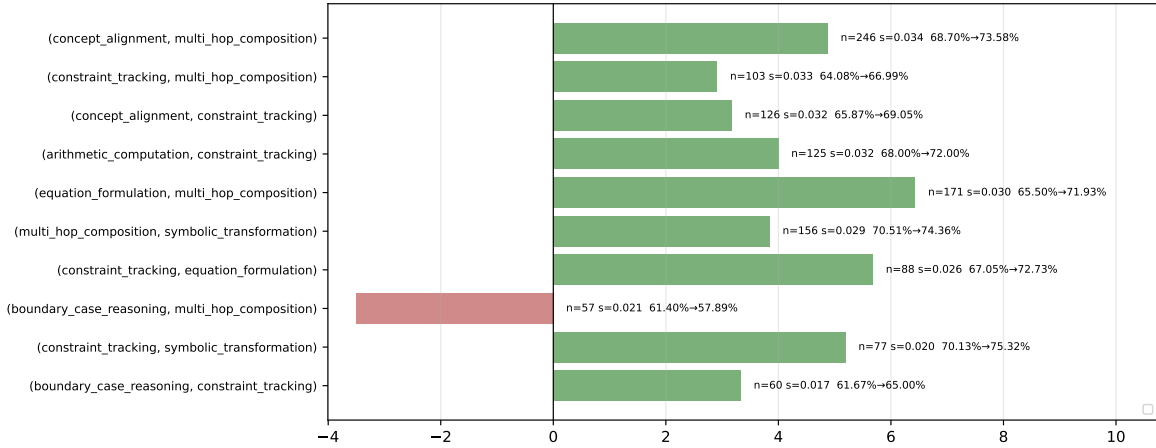


Figure 8: Operation-level accuracy change on MATH500 under Pass@128, over the same top-10 operation pairs as in Figure 5; conventions as in Figure 7. Mean $\Delta\text{acc} = +3.59\text{pt}$; nine of the ten slices improve.

retain low absolute accuracy under both checkpoints. Taken together with the pair-level pattern, the triple-level result indicates that the intervention is most effective on operation combinations whose dominant failure mode is multi-step compositional derivation across `concept_alignment`, `equation_formulation`, and `symbolic_transformation`, and least effective when `constraint_tracking` co-occurs with `equation_formulation`.

Operation-level attribution under Pass@128. We next re-evaluate the same single-operation, operation-pair, and operation-triple slices from the diagnosis-aligned feature pool $\mathcal{F}$ of Section 4.2.3, this time measuring slice accuracy under the zero-shot Pass@128 protocol on MATH500² rather than on the standard-suite mathematical evaluation set used above. Because Pass@128 credits a problem as solved once any of 128 independent samples succeeds, this view is informative about whether the targeted operation combinations are reachable by the model’s output distribution at all, complementing the standard-suite analysis rather than restating it.

The single-operation comparison in Figure 7 shows a mean change of $+9.77\text{pt}$ across the seven retained slices, with six of the seven slices improving. The sole regression is `option_elimination` at -25.00pt on $n = 4$ samples, where, given the slice size, a single flipped prediction already corresponds to a 25-percentage-point movement. The all-slice mean is heavily influenced by very low-support slices: `evidence_aggregation` ($n = 2$) contributes $+50.00\text{pt}$, `physical_state_simulation` ($n = 18$) contributes $+16.67\text{pt}$, and `span_extraction` ($n = 8$) con-

²We restrict this operation-level Pass@128 analysis to MATH500. AIME2025 and AIME2026 contain too few evaluation samples for the resulting per-slice supports to be statistically meaningful once partitioned by operation, operation pair, and operation triple.

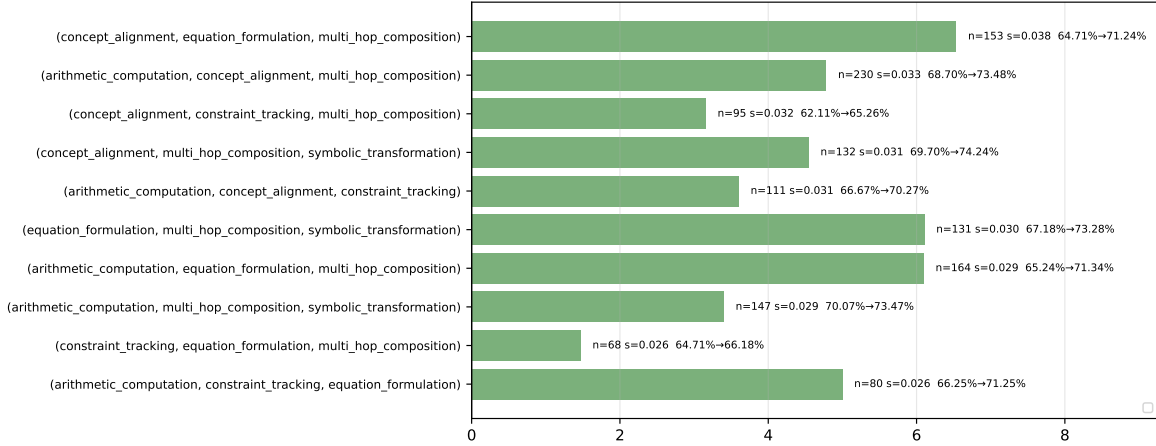


Figure 9: Operation-level accuracy change on MATH500 under Pass@128, over the same top-10 operation triples as in Figure 6; conventions as in Figure 7. Mean $\Delta\text{acc} = +4.47\text{pt}$; all ten slices improve.

tributes +12.50pt; at these sample sizes, a handful of flipped predictions can produce large nominal percentage changes. Restricting attention to the three higher-support single operations ($n \geq 50$: counting, boundary_case_reasoning, and constraint_tracking) yields a more stable mean of +4.74pt, with every one of the three slices improving: counting gains +8.07pt on 62 samples, boundary_case_reasoning gains +2.57pt on 78 samples, and constraint_tracking gains +3.57pt on 140 samples. We treat this higher-support estimate as the load-bearing summary of the single-operation panel; the all-slice mean is reported for completeness but should be read with its low-support sensitivity in mind.

The operation-pair comparison in Figure 8 shows a mean change of +3.59pt across the ten highest-scoring pairs, with nine of the ten slices improving. All six constraint-tracking-bearing pairs in the panel post gains of several percentage points: (constraint_tracking, equation_formulation) gains +5.68pt on 88 samples, (constraint_tracking, symbolic_transformation) gains +5.19pt on 77 samples, (arithmetic_computation, constraint_tracking) gains +4.00pt on 125 samples, (boundary_case_reasoning, constraint_tracking) gains +3.33pt on 60 samples, (concept_alignment, constraint_tracking) gains +3.18pt on 126 samples, and (constraint_tracking, multi_hop_composition) gains +2.91pt on 103 samples. The largest gain in the panel is on (equation_formulation, multi_hop_composition), at +6.43pt on 171 samples, and the highest-support slice, (concept_alignment, multi_hop_composition), gains +4.88pt on 246 samples. The only regression in the panel is (boundary_case_reasoning, multi_hop_composition) at -3.51pt on 57 samples; given its limited support, we treat this as a within-noise fluctuation rather than as evidence of a systematic effect.

The operation-triple comparison in Figure 9 is the strongest result in the panel set: all ten of the highest-scoring operation triples improve over base, with a mean change of +4.47pt and individual gains ranging from +1.47pt to +6.53pt. The two triples with the smallest gains in the panel, (constraint_tracking, equation_formulation, multi_hop_composition) at +1.47pt on 68 samples and (arithmetic_computation, constraint_tracking, equation_formulation) at +5.00pt on 80 samples, both pair constraint_tracking with equation_formulation; even so, both remain solidly positive. The largest gain in the panel is on the highest-scoring triple, (concept_alignment, equation_formulation, multi_hop_composition), at +6.53pt on 153 samples; the next-largest gains involve symbolic-composition triples that the diagnosis of Section 4.2.2 flagged as compositionally fragile, with (equation_formulation, multi_hop_composition, symbolic_transformation) and (arithmetic_computation, equation_formulation, multi_hop_composition) each gaining +6.10pt. With no regressions in the panel and a smallest individual gain of +1.47pt, the operation-triple results offer the clearest evidence that the targeted operation combinations are reliably reachable under repeated sampling.

4.2.6 Cross-Protocol Synthesis and Assessment of the Data-Evaluation Closed Loop

Standard suite versus Pass@128: consistency and amplification at the operation level. The two slice-level evaluations in Sections 4.2.2 and above target the same diagnosis-aligned feature pool $\mathcal{F}$ but differ in what they measure,

the standard suite credits only the model’s single, modal completion, while Pass@128 credits a problem as solved once any of 128 independent samples succeeds, and read together they give a sharper picture than either protocol does alone.

First, the operation-level signal is consistent in direction across both protocols on the slices that the diagnosis identified as compositional and underweight in the untargeted baseline: (concept_alignment, multi_hop_composition), (multi_hop_composition, symbolic_transformation), (arithmetic_computation, concept_alignment, multi_hop_composition), and (equation_formulation, multi_hop_composition, symbolic_transformation) all improve under both the standard suite and Pass@128. This makes it unlikely that the slice-level gains in Figures 8 and 9 are an artifact specific to the smaller Pass@128 evaluation set.

Second, the magnitude and uniformity of the operation-level signal differ sharply between protocols at the pair and triple levels. The mean accuracy change is roughly an order of magnitude larger under Pass@128 than under the standard suite (+3.59pt vs. +0.16pt for pairs; +4.47pt vs. +0.19pt for triples), and the proportion of improving slices is correspondingly higher (9/10 vs. 5/10 for pairs; 10/10 vs. 8/10 for triples). The operation-triple result under Pass@128 is the cleanest single piece of evidence that the gains in Table 33 are not a generic continued-training effect but track the diagnostic-to-data link that Eq. (7) encodes: all ten top-scoring triples improve, with individual gains ranging from +1.47pt to +6.53pt.

Third, the constraint-tracking-bearing slices that limited the standard-suite attribution are recovered, and substantially so, under Pass@128. This is the most informative protocol contrast in the controlled experiment, since it concerns exactly the slices whose standard-suite regression would otherwise be read as a failure of the intervention on the operations it was meant to address. All six constraint-tracking-bearing pairs improve under Pass@128, by between +2.91pt and +5.68pt, and the two constraint-tracking-bearing triples that regressed under the standard suite improve by +1.47pt and +5.00pt under Pass@128. We interpret this in line with the Pass@1–Pass@K gap discussed in Section 4.2.4: the intervention places non-negligible probability mass on previously absent or vanishingly rare reasoning trajectories for constraint-tracking-heavy problems, but those trajectories have not yet become the model’s modal completion. A standard-suite, Pass@1-style evaluation, which credits only the modal completion, is therefore systematically less sensitive to this kind of trajectory broadening than a Pass@K-style evaluation, and the protocol-dependence of the constraint-tracking signal is the operation-level signature of this gap. We accordingly do not read constraint tracking as resistant to the intervention; rather, the standard-suite regression reflects a Pass@1-style measurement of an intervention whose effect on these slices is, at this stage, dominated by trajectory broadening rather than modal sharpening.

Benchmark-level versus operation-level results. The operation-level decomposition also helps reconcile two features of the benchmark-level evidence in Section 4.2.4 that are difficult to interpret from aggregate scores alone.

The first concerns the relative scale of the standard-suite and Pass@K benchmark deltas. On the standard suite, exp improves over warm-start by a comparatively small margin on MathQA (52.16 $\rightarrow$ 53.27) and MATH (Minerva) (41.92 $\rightarrow$ 42.82); under Pass@K, the gap between exp and base instead widens sharply with the sampling budget, reaching +3.80pt on MATH500 and +20.00pt on AIME2025 at Pass@128 (Table 33). This asymmetry has the same shape as what we observe at the operation level, where the standard-suite pair and triple means are an order of magnitude smaller than their Pass@128 counterparts. The benchmark-level and operation-level views are therefore mutually consistent on this point, and the operation-level decomposition supplies a candidate mechanism: the targeted operation combinations appear to become reachable under repeated sampling well before they become the modal completion, so a metric that credits only the modal output registers a markedly weaker signal than one that credits any of 128 attempts. We restrict this reading to the three mathematical benchmarks analyzed at the operation level; the code-generation benchmarks (HumanEval, MBPP), which show a small and mixed standard-suite effect, fall outside the solving-operation diagnosis used here and are not addressed by this comparison. We also note that the operation-level Pass@128 results are themselves drawn only from MATH500, so they bear most directly on the MATH500 benchmark-level gain and only suggestively, rather than directly, on the AIME results, which the per-slice annotation could not support (see footnote above).

The second concerns an apparent tension: GSM8K is the one standard-suite mathematical benchmark on which exp regresses relative to warm-start (69.67 $\rightarrow$ 68.61), even though base itself improves (69.90). Read in isolation, this could suggest the intervention is mildly harmful on at least one mathematical benchmark. Two non-exclusive readings are consistent with the operation-level evidence above, and we cannot adjudicate between them with the evidence collected in this case study. One is that the regression is largely within ordinary evaluation noise for a single-checkpoint comparison, given that the standard-suite operation-level results already show several slices moving by less than one point in either direction. The other is that it reflects the same protocol-dependence identified in the constraint-tracking case: if GSM8K’s problem mix draws more heavily on constraint-tracking-style compositions than MathQA or MATH (Minerva) do, a standard-suite, Pass@1-style read-out would be expected to under-detect any underlying improvement

on this benchmark in particular. We have not separately verified the per-benchmark composition of operation labels, so we present this only as a plausible, operation-level-motivated hypothesis rather than as an established explanation. What the operation-level results do support is the narrower claim that the GSM8K regression is consistent with a benchmark-level instance of the same protocol-sensitivity already documented at the slice level, rather than with a broad, intervention-induced loss of mathematical capability: no comparable regression appears across the eleven general-language, knowledge, and reading-comprehension benchmarks in Table 32, and the operation-level regressions on the standard suite are themselves concentrated in a small, identifiable subset of slices rather than spread broadly across the feature pool.

Assessment of the targeted-sampling method and the data-evaluation loop. Taken together, these comparisons support a specific and appropriately qualified judgment about the diagnosis-aligned weighted-sampling method of Section 4.2.3. The method succeeds at its stated objective: improvement is concentrated on the operation combinations that the weakness score in Eq. (7) up-weighted, the direction of the effect is stable across two evaluation protocols that differ substantially in what they measure, and the size of the effect tracks how directly each protocol can detect a broadened-but-not-yet-modal output distribution rather than any property of the sampling procedure itself. This is the pattern a targeted intervention should produce if the underlying diagnosis is correct, and it is difficult to reconcile with the alternative hypothesis that the gains in Tables 32 and 33 are a generic continued-training effect unrelated to the diagnosed weak slices.

At the same time, the constraint-tracking case is an instructive limit on this success. Under the standard suite, the constraint-tracking-bearing pairs and triples are exactly the slices that fail to improve, which, read on its own, would look like a failure of the intervention on precisely the operations it was meant to address. Only the Pass@128 re-evaluation reveals that the underlying capability gap on these slices has in fact narrowed substantially. This suggests that the diagnosis-aligned weighting succeeds at increasing the representation of the targeted operation combinations in the training signal, but that a single-shot, modal-output evaluation can still under-detect this kind of improvement when it manifests as broadened trajectory coverage rather than as a sharpened top completion. Consistent with the caution already raised in Section 3.4.1 and Section 4.2.3, the mapping rules and sampling procedure are best read as heuristics for generating and prioritizing data-intervention hypotheses rather than as guarantees of improvement under any single evaluation protocol; this case study indicates that the evaluation side of the loop needs to be correspondingly multi-protocol, since a heuristic intervention can succeed at the level the mapping rules target without that success being visible under every downstream metric.

This case study is, in turn, an end-to-end demonstration of the data-evaluation closed loop introduced in Section 3. A benchmark-level weakness on mathematical problem solving (Section 4.2.1) was localized, via the evaluation sample taxonomy, into solving-operation slices, pairs, and triples whose raw error rates, pairwise effects, and non-additive composition effects pointed to compositional derivation rather than isolated arithmetic failure as the dominant failure mode (Section 4.2.2). This diagnosis was translated into a concrete data action, a diagnosis-aligned weighted-sampling policy over synthetic instruction data (Section 4.2.3), and the resulting intervention was validated under a controlled, single-variable experimental design (Section 4.2.4). Finally, the outcome was mapped back onto the evaluation sample taxonomy at the operation level, under two protocols that probe different aspects of the model’s output distribution, to determine whether the benchmark-level changes originated from the targeted slices rather than from an unrelated source. Each step left an inspectable, quantitative artifact, the weakness score, the retained feature pool, the per-slice sampling weights, and the per-slice accuracy changes under both protocols, so that the link between the original diagnosis and the final benchmark movement can be audited rather than asserted. The constraint-tracking case shows concretely why this auditability matters: it is the re-diagnosis step, and not the standard-suite score taken alone, that produces the correct attribution. In this sense, the closed loop accomplishes what Section 3 sets out for it, converting a noisy, benchmark-level observation into an actionable, auditable, and iterable procedure, while preserving throughout an explicit distinction between what the diagnosis and mapping rules suggest and what the controlled experimental evidence, examined at more than one level and under more than one protocol, actually shows.

5 Conclusion

Summary. This paper develops a closed-loop methodology for connecting evaluation failures in LLM pre-training to concrete training-data interventions. Its starting premise, developed in Section 2, is that a benchmark score is too coarse a unit, and an individual evaluation sample too noisy a unit, to drive data decisions; the appropriate intermediate unit is the *capability slice*, a group of evaluation samples that share structured conditions along background condition, task type, solving operation, and output constraint (Sections 2.1–2.2). Training data is, in turn, treated as the controllable substrate from which capability slices are formed: we separate it into an instruction-data path that shares the same

four-dimensional vocabulary as evaluation samples, and a non-instruction-data path characterized along content world, discourse structure, operation opportunity, and supervision density (Section 2.3).

Section 3 operationalizes this framework into three components. An evaluation sample taxonomy, annotated over sixteen commonly used pre-training benchmarks through iterative human-LLM co-design, localizes weaknesses below the level of the benchmark name. A non-instruction data taxonomy characterizes corpus segments along the four data-side dimensions above. A set of evaluation-to-data mapping rules connects the two: taxonomy-aligned correspondences where the dimensions were designed jointly, and an LLM-assisted mechanism for slices that fall outside them. Together, these components instantiate the five-step workflow introduced in Section 3.4,

Weak Capability Slice → Data Affordance Profile → Data Action → Experimental Validation → Result Analysis, which turns a benchmark-level observation into an auditable, iterable data-intervention loop.

The two case studies in Section 4 exercise this loop under qualitatively different regimes. In the first (Section 4.1), an aggregate −46.82% relative drop on BBH after continued pre-training initially resembled a regression in multi-step reasoning, the capability BBH is designed to probe. Decomposing the regression along the output-constraint dimension instead traced it to a masked <EOS> loss in the continued pre-training pipeline: the model reached the correct closed-set answer but did not terminate generation afterward, so an otherwise correct response was rejected by the benchmark’s exact-match scorer. Restoring loss supervision on the <EOS> token raised BBH from 25.14 to 66.44, above the warm-start checkpoint, while leaving the other fifteen benchmarks largely unaffected and, on several of them, modestly improving performance further. Here, the benchmark name actively misled the initial diagnosis: the failure resided in the construction of the training objective rather than in the data mixture or in the model’s underlying capability.

The second case study (Section 4.2) addresses a more characteristic pre-training weakness: near-zero Pass@128 on AIME, indicating that increasing the test-time sampling budget alone could not surface a valid reasoning trajectory. Decomposing this weakness by solving operation, together with a pairwise- and composition-effect analysis distinguishing additive from non-additive difficulty, localized the failure to specific operation combinations, among them constraint tracking, boundary-case reasoning, symbolic transformation, counting, and comparison, rather than to a uniform deficit in arithmetic computation or equation formulation. Translating this diagnosis into a diagnosis-aligned, weakness-targeted importance-sampling procedure over a pool of synthetic instruction data, with the training recipe and token budget held fixed, raised AIME2025 Pass@128 from 6.67 to 26.67 and AIME2026 Pass@128 from 0.00 to 26.67, while leaving general-domain performance close to unchanged. A subsequent re-evaluation at the level of the targeted operation combinations confirmed that this improvement was concentrated on the diagnosed slices, with the clearest and most consistent gains appearing under Pass@128, the protocol best suited to detecting reasoning paths that have become reachable but are not yet the model’s modal completion.

Across both case studies, the diagnostic decomposition performed the work that an aggregate score could not: distinguishing whether an observed change in benchmark performance reflected the capability the benchmark was designed to measure, or a more specific and more directly addressable property of the training objective, the data mixture, or the evaluation protocol itself.

Limitations. Several aspects of this work are best understood as an initial methodology rather than as established conclusions. The evaluation-to-data mapping rules (Section 3.4) are structured heuristics for generating data-intervention hypotheses, not causal claims about the relationship between a corpus segment and a benchmark score. On the non-instruction-data side, they are further restricted to a small set of dimension-level correspondences that were aligned by design during taxonomy construction (domain, discourse form, context scope, and solving operation), supplemented by an LLM-assisted mechanism for slices that fall outside this correspondence. Any data intervention these rules suggest, whether taxonomy-aligned or LLM-assisted, remains a hypothesis until subjected to the kind of controlled experimental validation demonstrated in the two case studies, rather than being accepted at face value.

The empirical scope of this paper is also limited in ways that bear on how broadly its findings should be read. Both taxonomies, and the two case studies that exercise them, were instantiated on a fixed set of sixteen commonly used pre-training benchmarks and on a single continued pre-training pipeline; their behavior on other other training stages, or evaluation suites oriented toward different capabilities remains to be tested.

A further limitation concerns the asymmetry between the two sides of the mapping. The controlled comparison in Section 4.2 exercises only the instruction-data branch, constructing a targeted instruction subset from a synthetic pool; in both case studies, the non-instruction data taxonomy is used mainly as a descriptive vocabulary for stating data-side targets rather than as the basis for an executed intervention. A symmetric test, retrieving, reweighting, or filtering pre-training corpus segments by the same data affordance profile and validating the result under a comparably controlled design, has not been performed, and it remains an open question whether the loop is equally effective on this more indirect side of the mapping.

References

- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021a.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of ACL*, 2019.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of ACL*, 2023.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021b.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of ACL*, 2017.
- Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. Race: Large-scale reading comprehension dataset from examinations. In *Proceedings of EMNLP*, 2017.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of NAACL-HLT*, 2019.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.
- Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Jiayi Lei, Yao Fu, Maosong Sun, and Junxian He. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models. In *Advances in Neural Information Processing Systems*, 2023.
- Haonan Li, Yixuan Zhang, Fajri Koto, Yifei Yang, Hai Zhao, Yeyun Gong, Nan Duan, and Timothy Baldwin. Cmmlu: Measuring massive multitask language understanding in chinese. In *Findings of ACL*, 2024a.
- Wanjuan Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. Agieval: A human-centric benchmark for evaluating foundation models. In *Findings of NAACL*, 2024.
- Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V Le, Tengyu Ma, and Adams Wei Yu. Doremi: Optimizing data mixtures speeds up language model pretraining. In *Advances in Neural Information Processing Systems*, 2023.
- Qian Liu, Xiaosen Zheng, Niklas Muennighoff, Guangtao Zeng, Longxu Dou, Tianyu Pang, Jing Jiang, and Min Lin. Regmix: Data mixture as regression for language model pre-training. *arXiv preprint arXiv:2407.01492*, 2024.
- Jiasheng Ye, Peiju Liu, Tianxiang Sun, Yunhua Zhou, Yu Sun, and Xipeng Qiu. Data mixing laws: Optimizing data mixtures by predicting language modeling performance. *arXiv preprint arXiv:2403.16952*, 2024.
- Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.
- Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Yitzhak Gadre, Hritik Bansal, et al. Datacomp-lm: In search of the next generation of training sets for language models. *arXiv preprint arXiv:2406.11794*, 2024b.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of ACL*, 2023a.

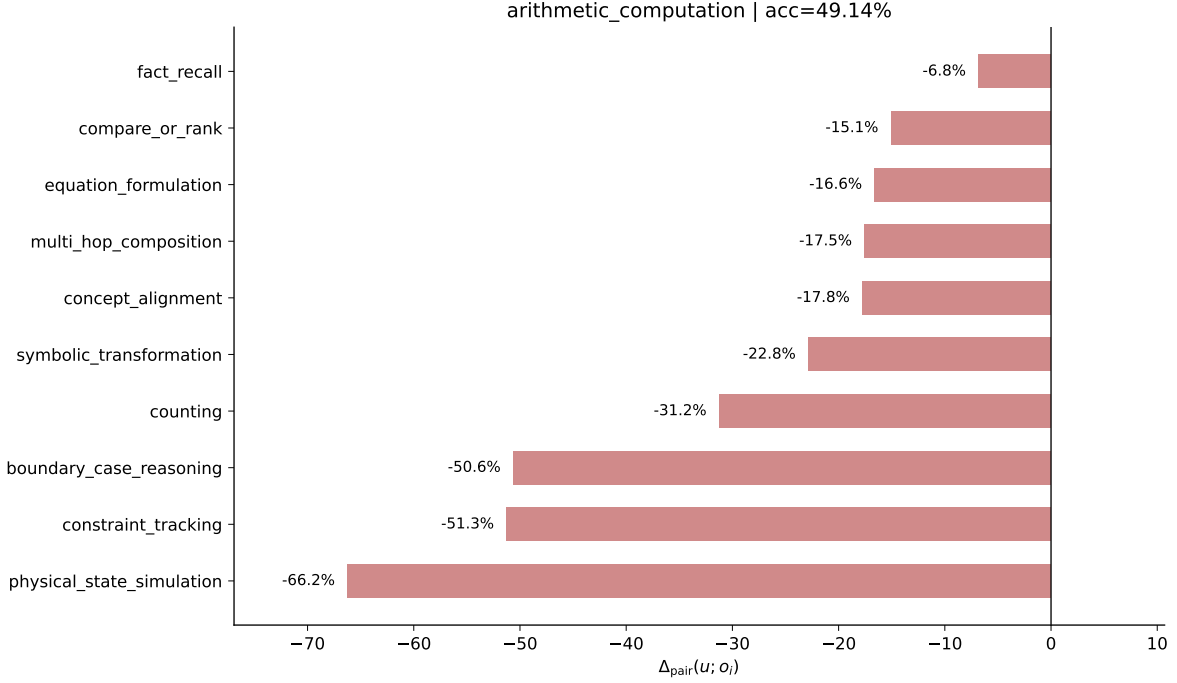


Figure 10: Pairwise Composition Effect of arithmetic_computation

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. Wizardlm: Empowering large language models to follow complex instructions. In *International Conference on Learning Representations*, 2024.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack Hessel, Tushar Khot, Khyathi Raghavi Chandu, David Wadden, Kelsey MacMillan, Noah A Smith, Iz Beltagy, and Hannaneh Hajishirzi. How far can camels go? exploring the state of instruction tuning on open resources. In *Advances in Neural Information Processing Systems*, 2023b.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.

Elliot Glazer, Ege Erdil, Tamay Besiroglu, Diego Chicharro, Evan Chen, Alex Gunning, Caroline Falkman Olsson, et al. Frontiermath: A benchmark for evaluating advanced mathematical reasoning in ai. *arXiv preprint arXiv:2411.04872*, 2024.

Pavlos S Efraimidis and Paul G Spirakis. Weighted random sampling with a reservoir. *Information Processing Letters*, 97(5):181–185, 2006.

A Additional Figures for Operation-level Diagnosis

A.1 Figures of Pairwise Composition Effects

A.2 Figures of Non-additive Composition Effects

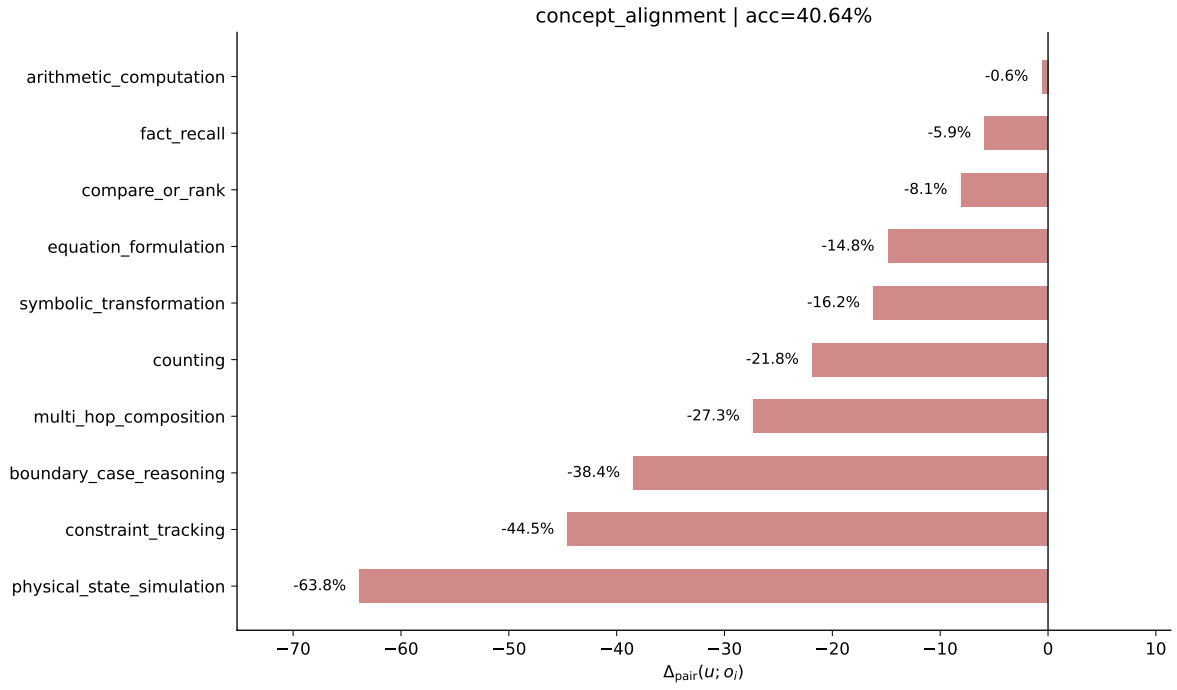


Figure 11: Pairwise Composition Effect of concept_alignment

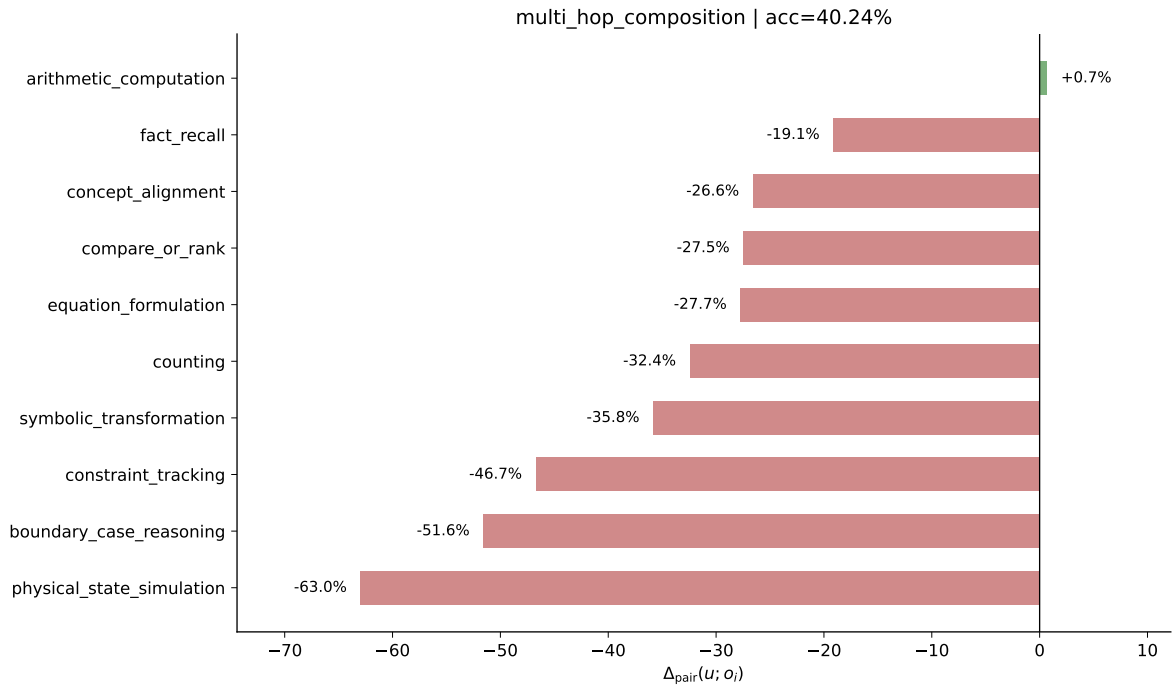


Figure 12: Pairwise Composition Effect of multi_hop_composition

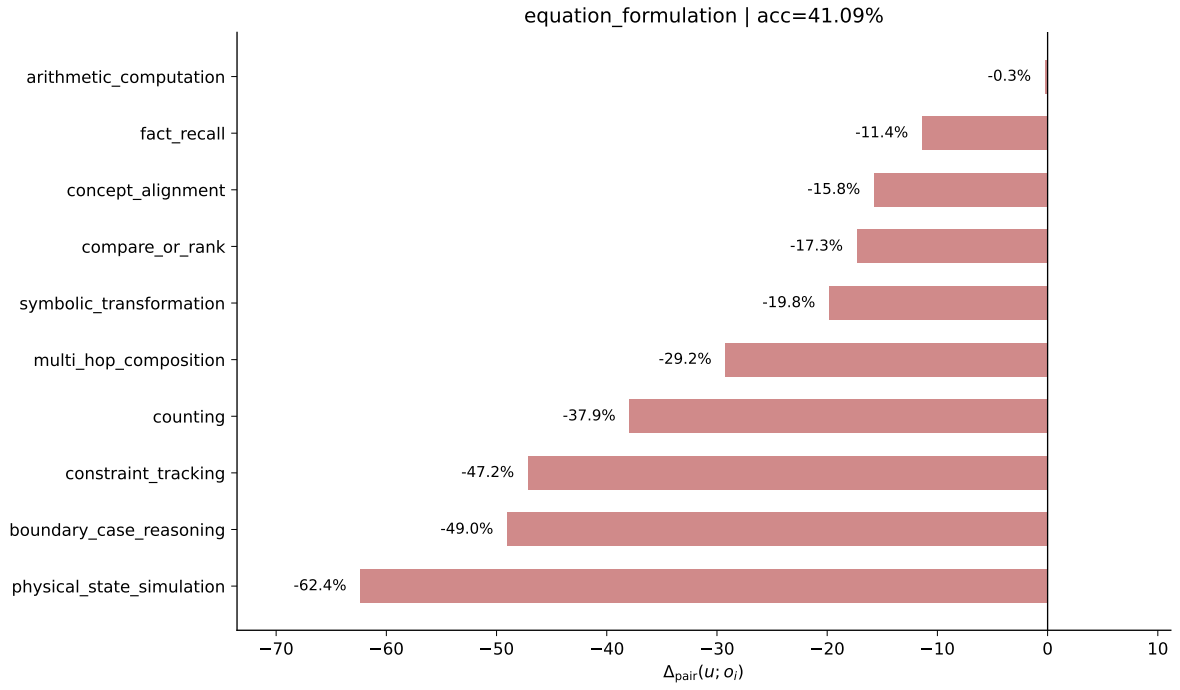


Figure 13: Pairwise Composition Effect of equation_formulation

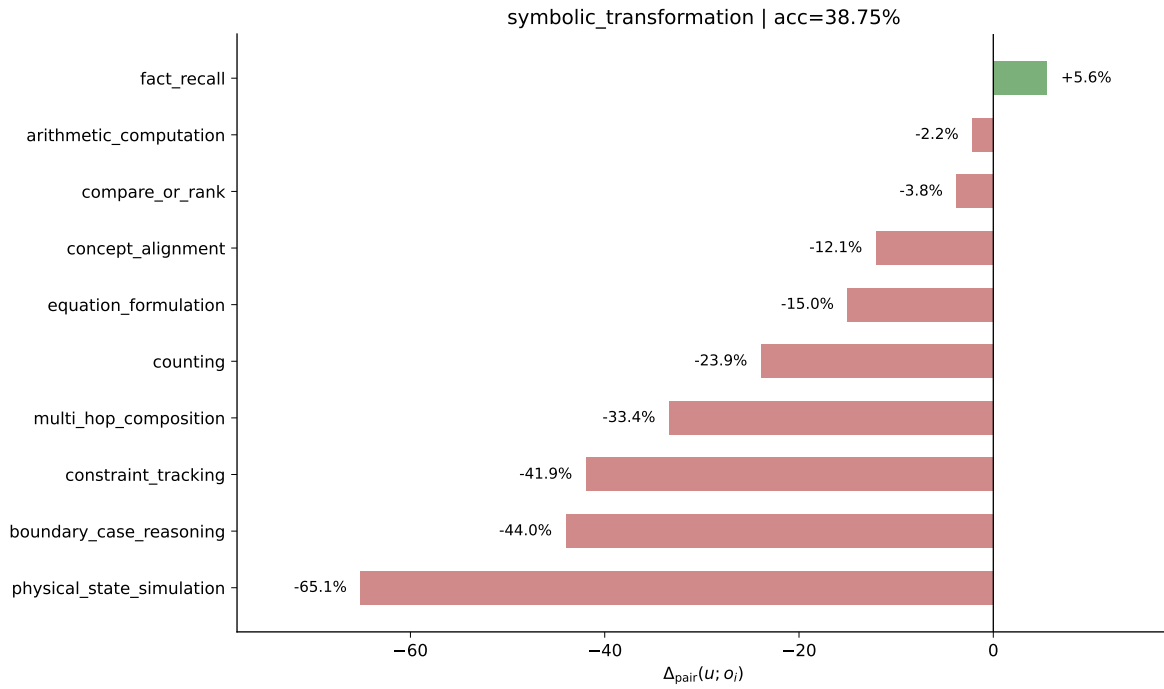


Figure 14: Pairwise Composition Effect of symbolic_transformation

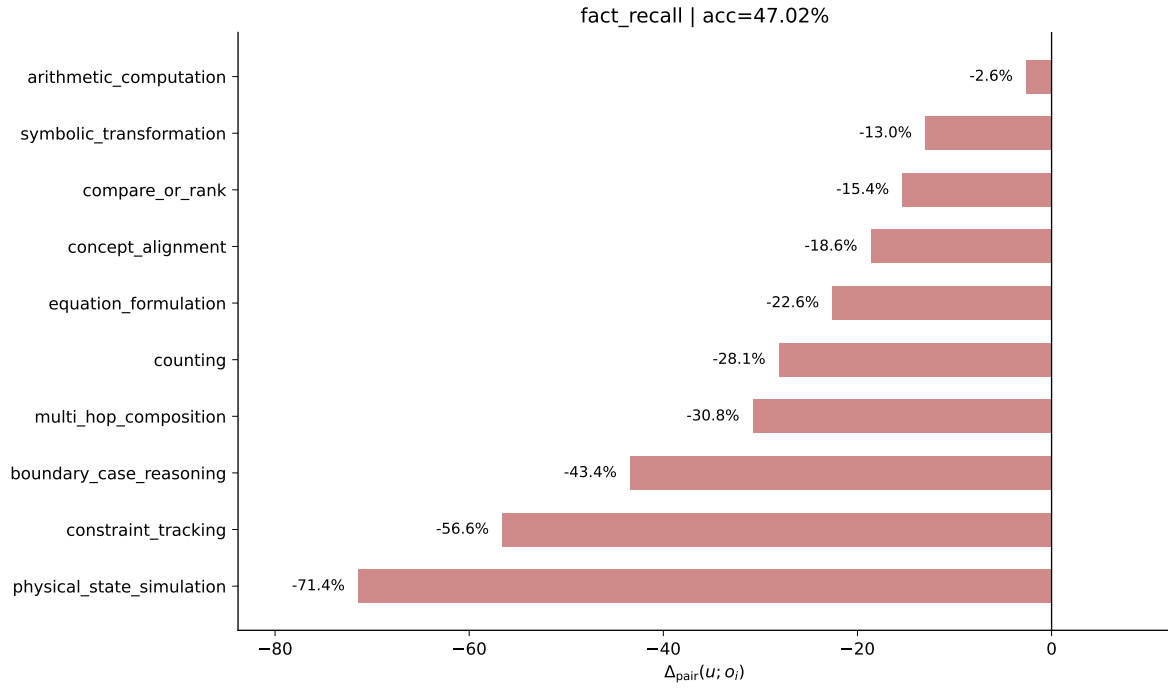


Figure 15: Pairwise Composition Effect of fact_recall

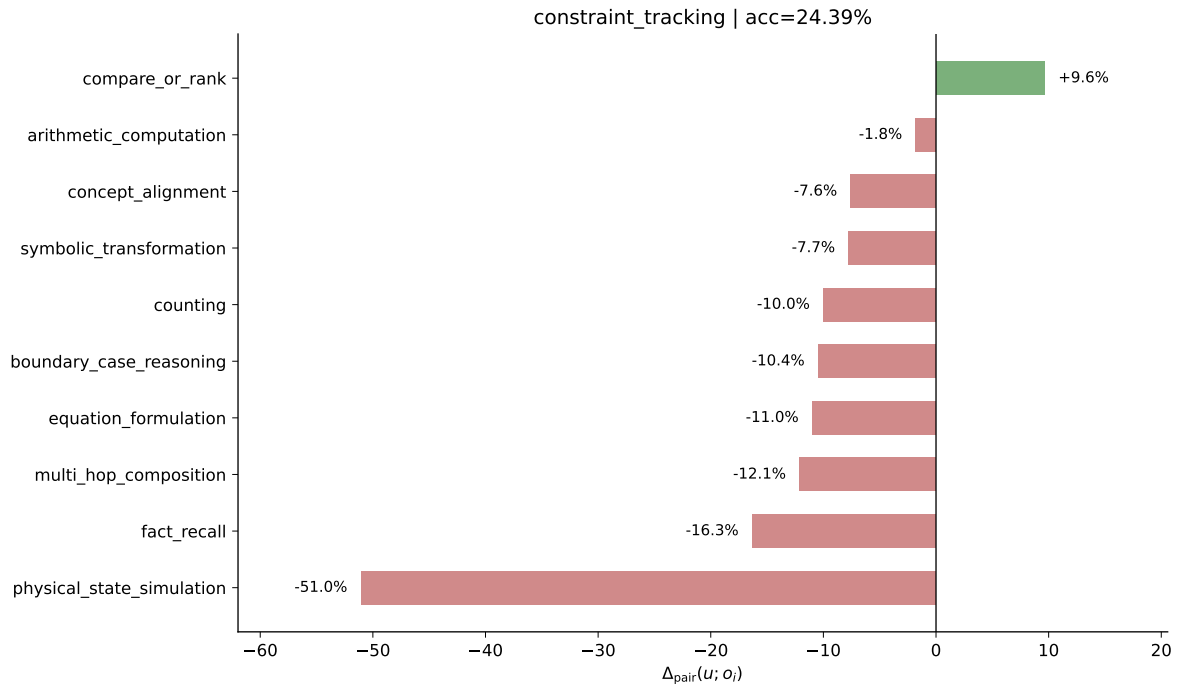


Figure 16: Pairwise Composition Effect of constraint_tracking

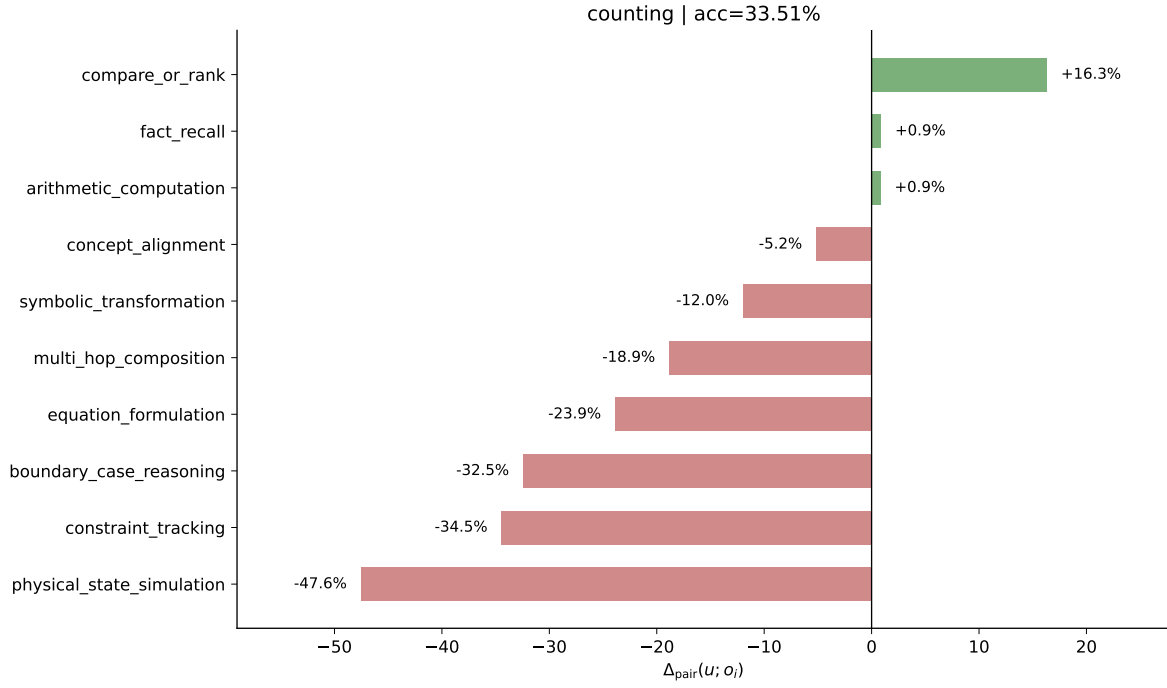


Figure 17: Pairwise Composition Effect of counting

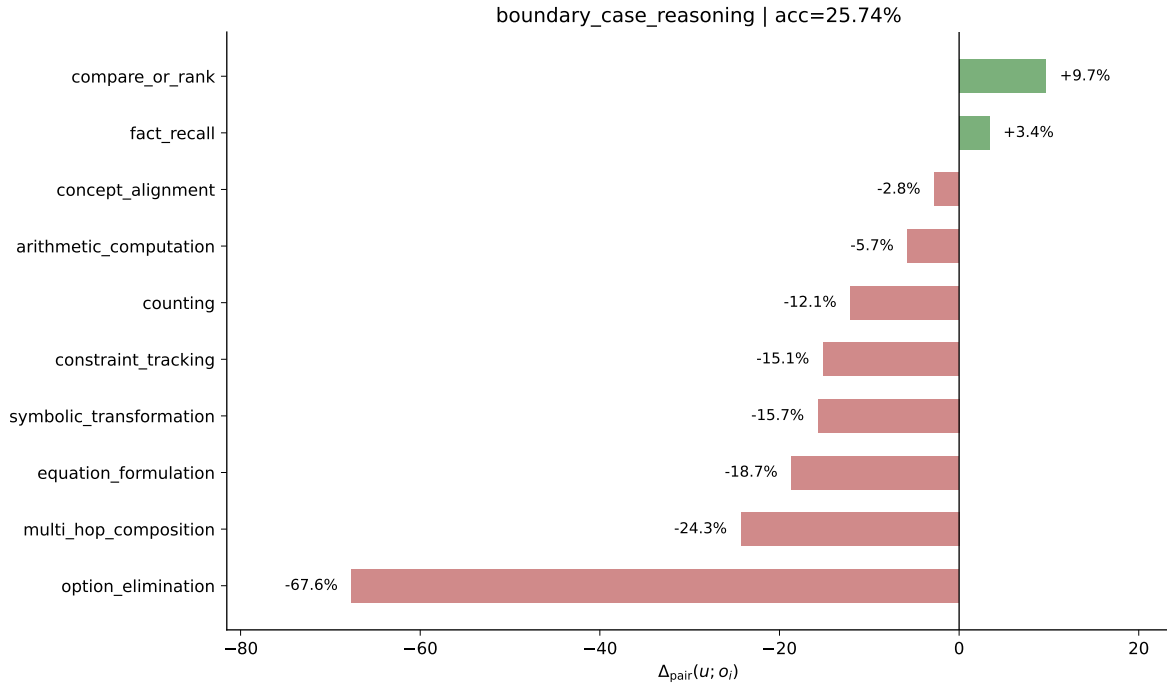


Figure 18: Pairwise Composition Effect of boundary_case_reasoning

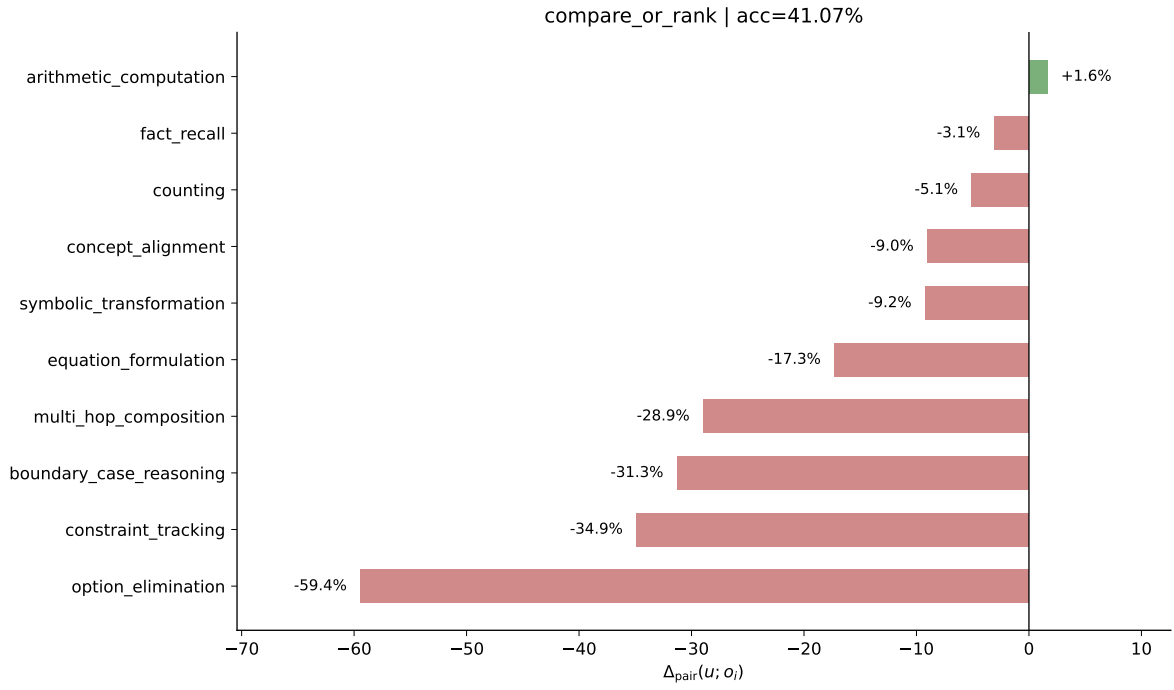


Figure 19: Pairwise Composition Effect of compare_or_rank

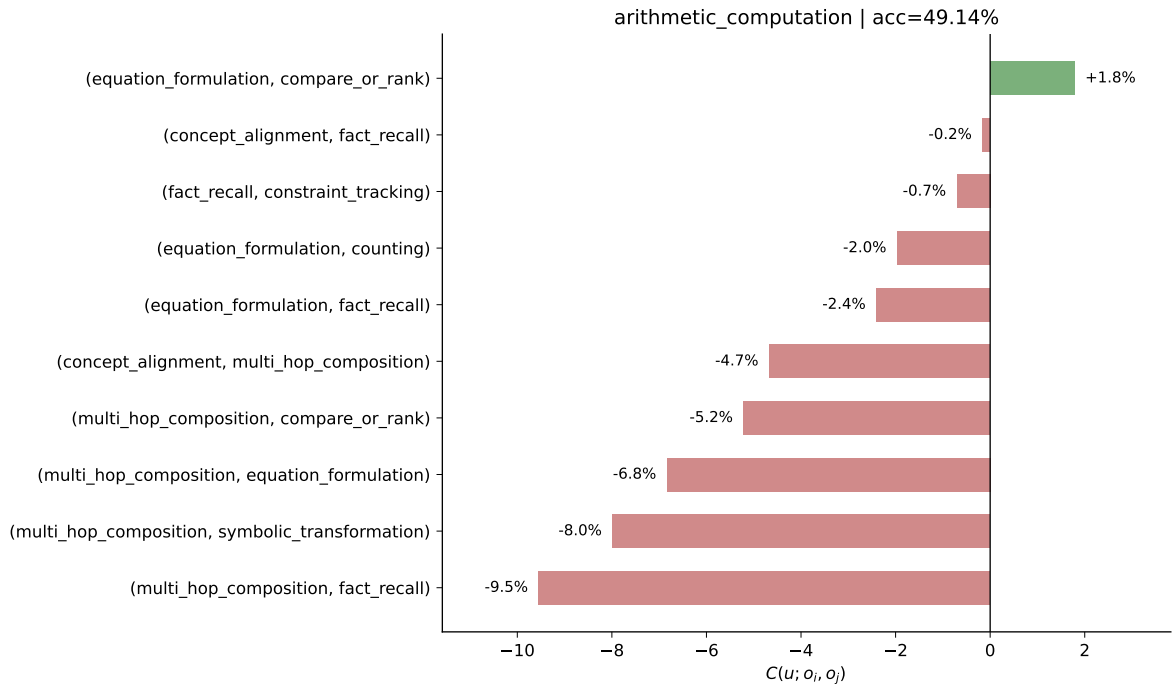


Figure 20: Non-additive Composition Effect of arithmetic_computation

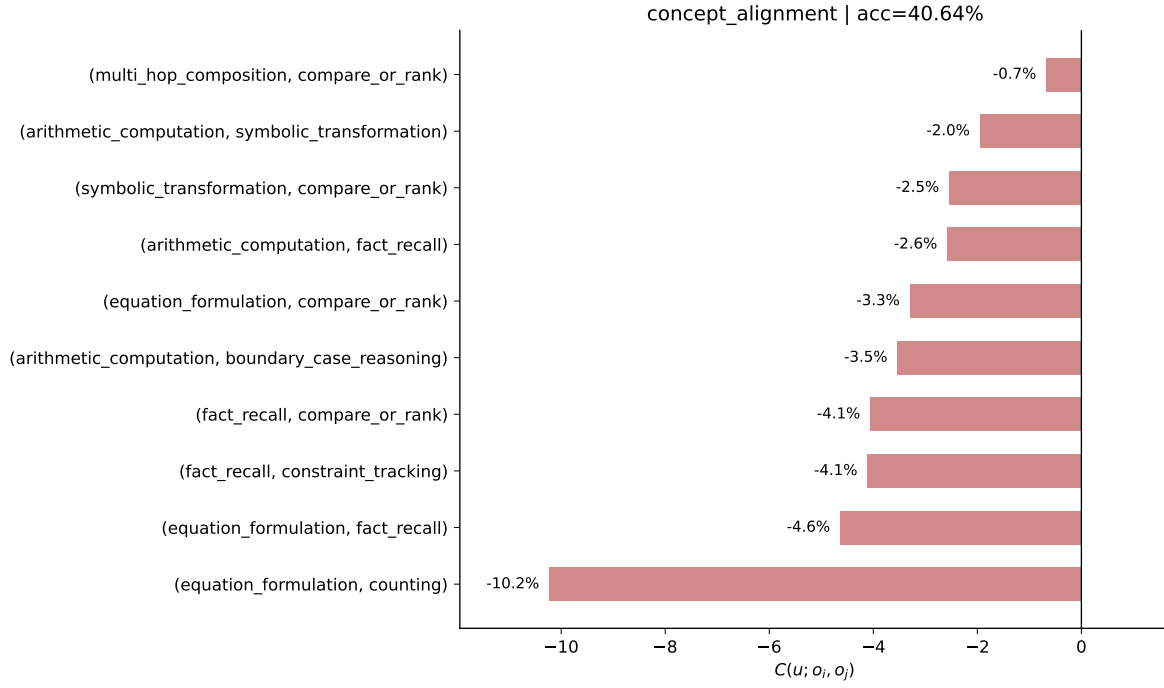


Figure 21: Non-additive Composition Effect of concept_alignment

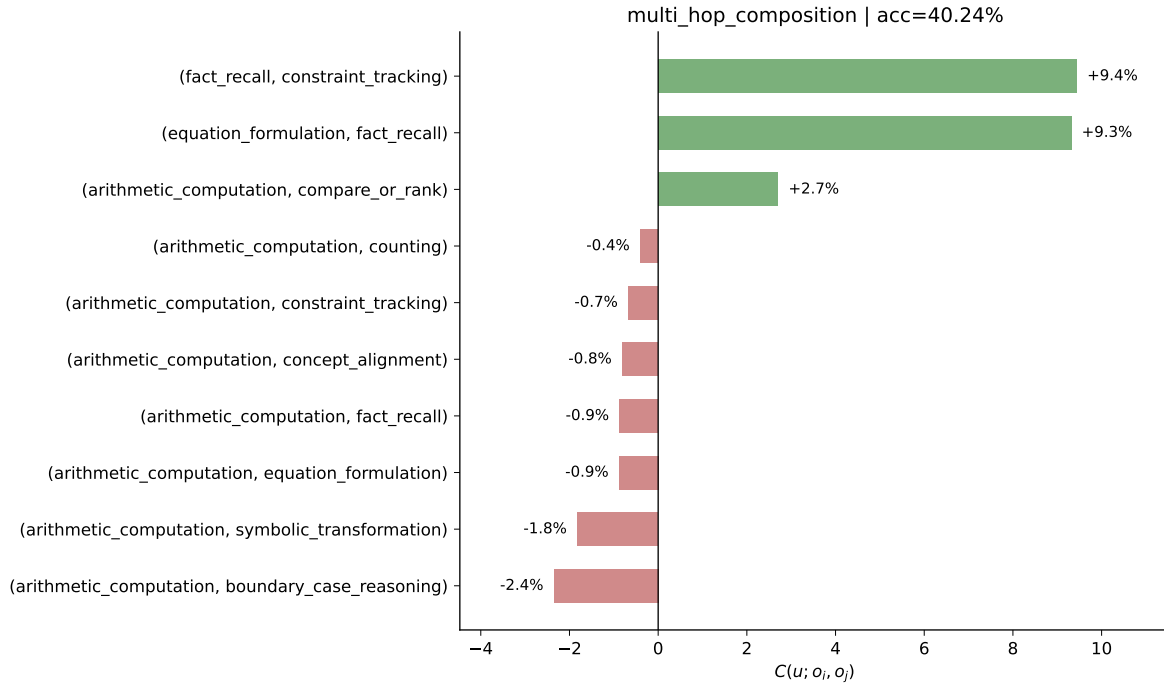


Figure 22: Non-additive Composition Effect of multi_hop_composition

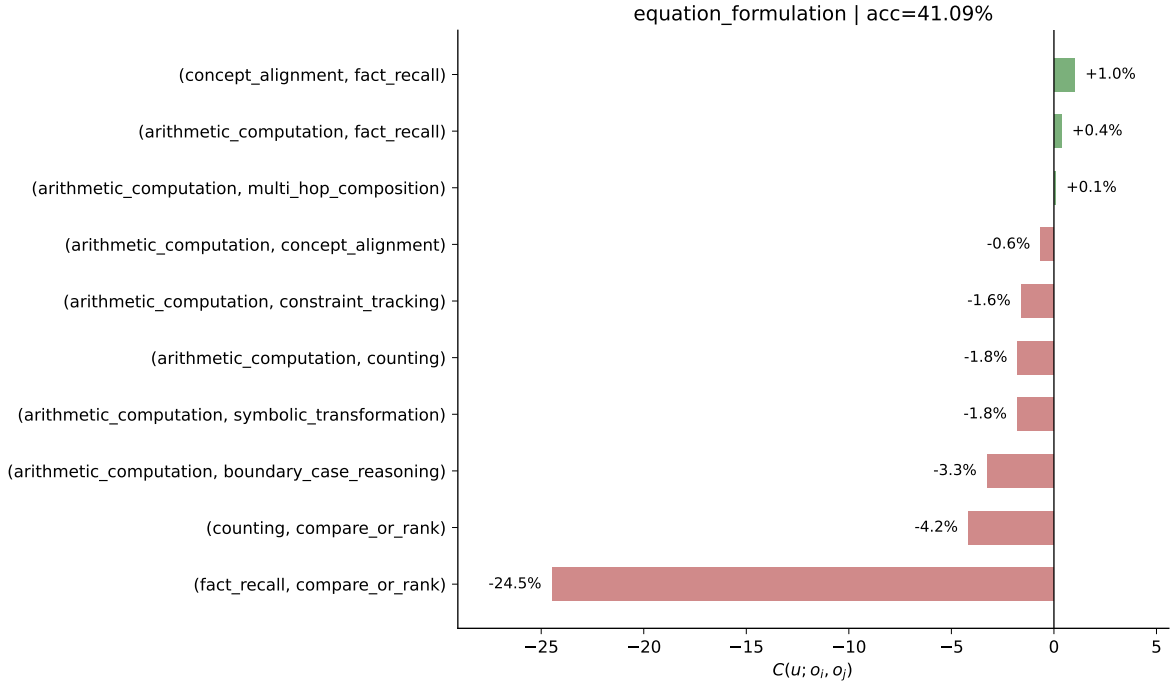


Figure 23: Non-additive Composition Effect of equation_formulation

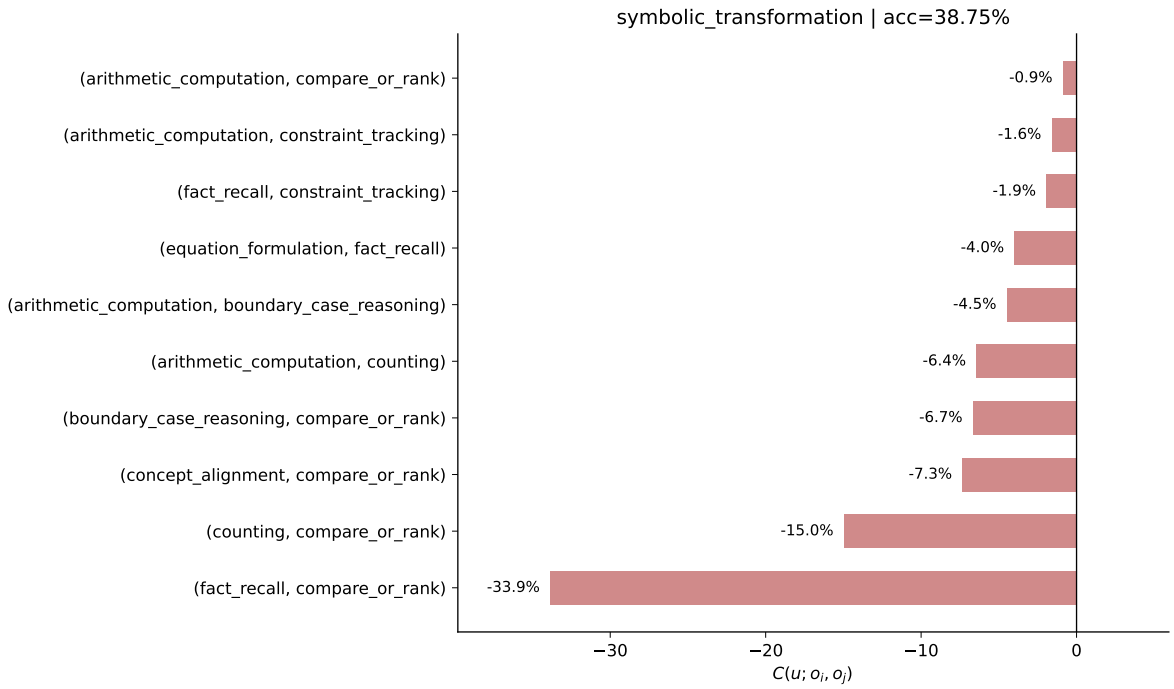


Figure 24: Non-additive Composition Effect of symbolic_transformation

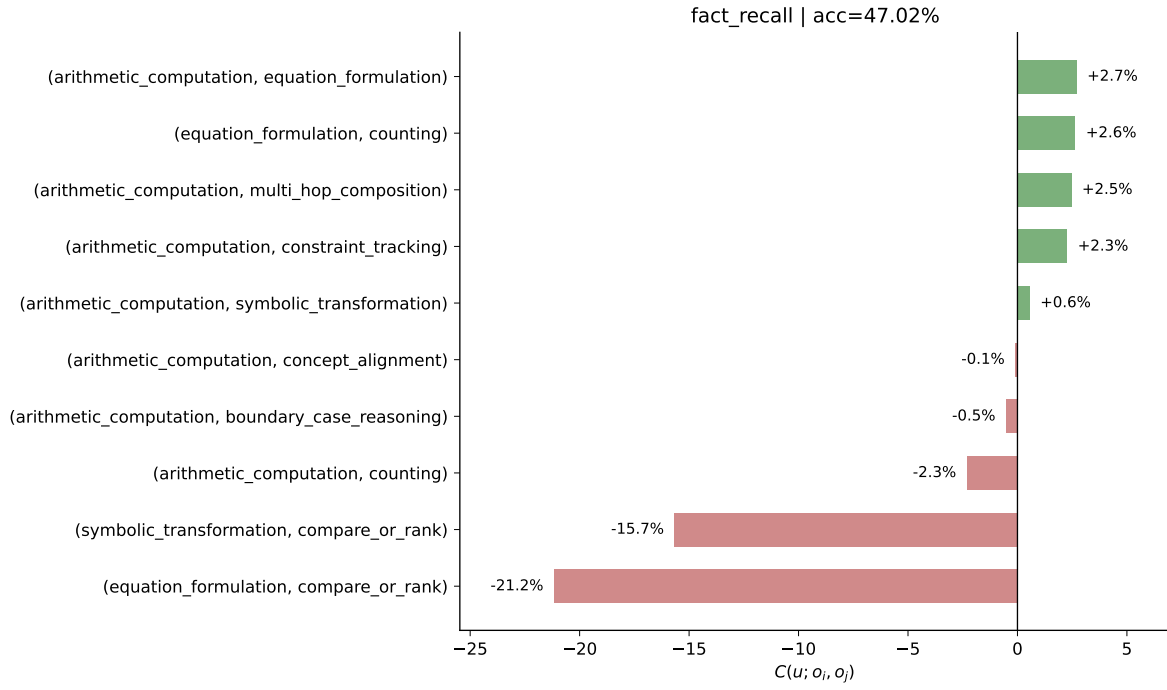


Figure 25: Non-additive Composition Effect of fact_recall

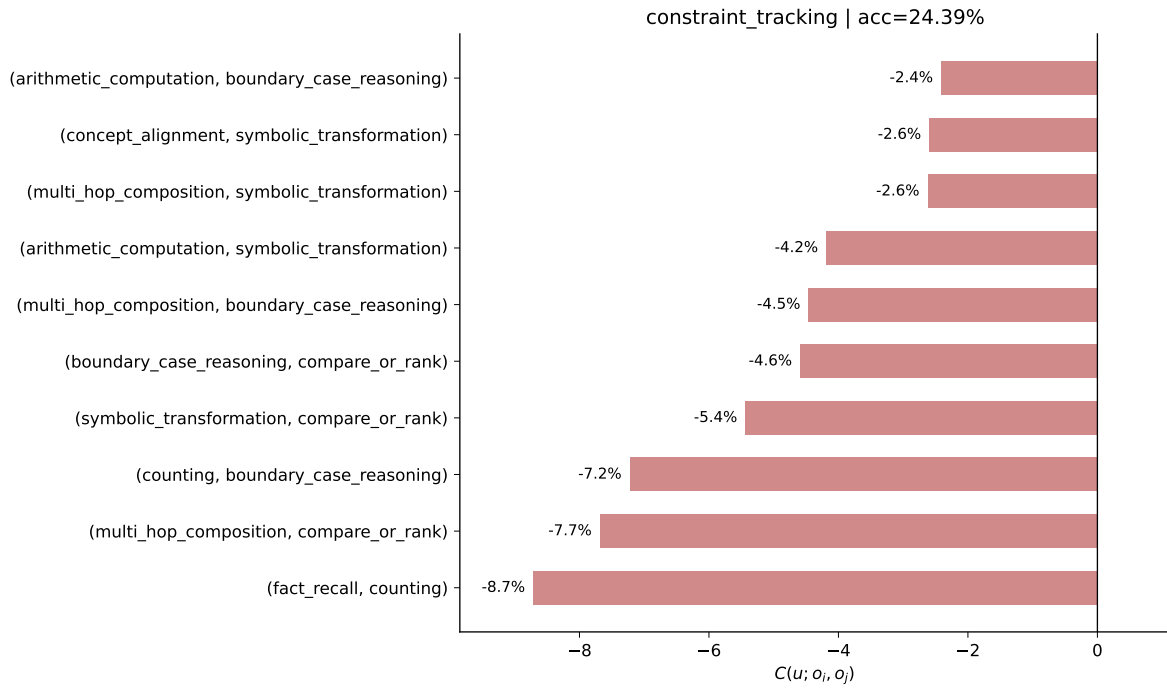


Figure 26: Non-additive Composition Effect of constraint_tracking

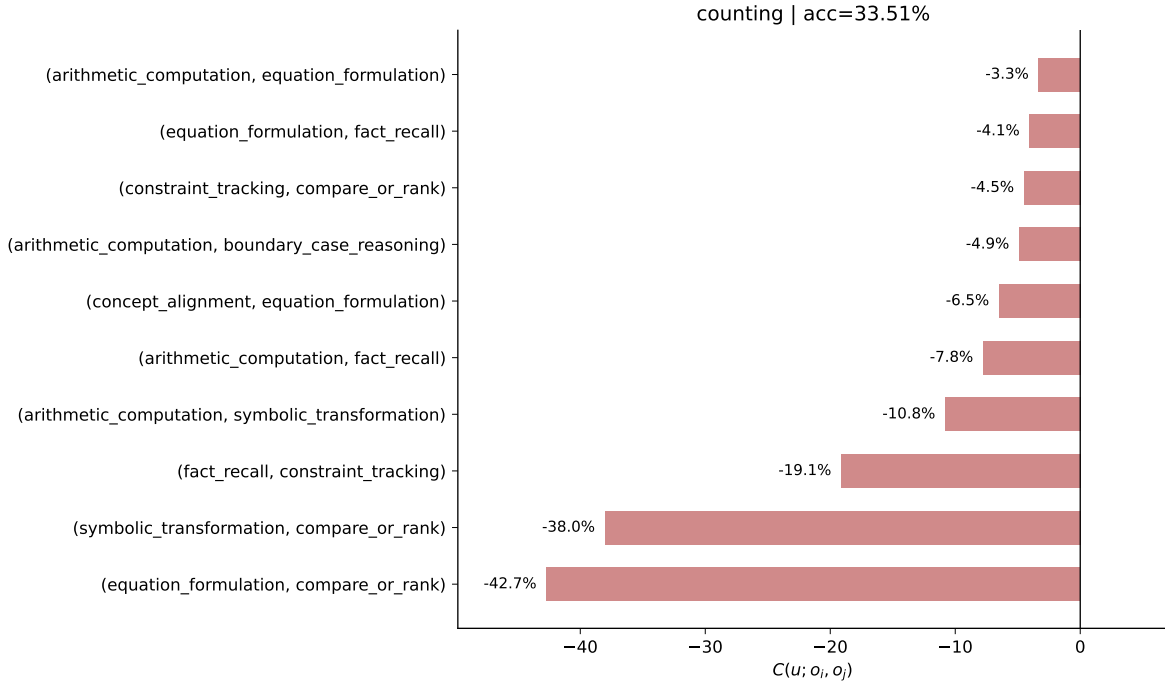


Figure 27: Non-additive Composition Effect of counting

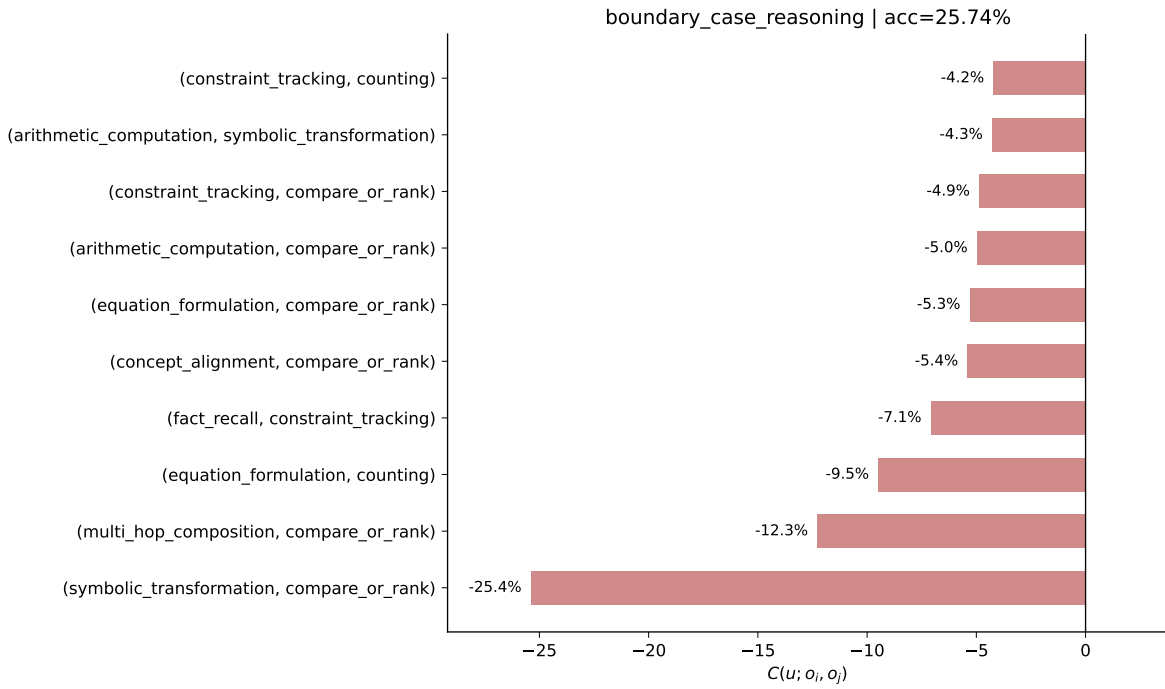


Figure 28: Non-additive Composition Effect of boundary_case_reasoning

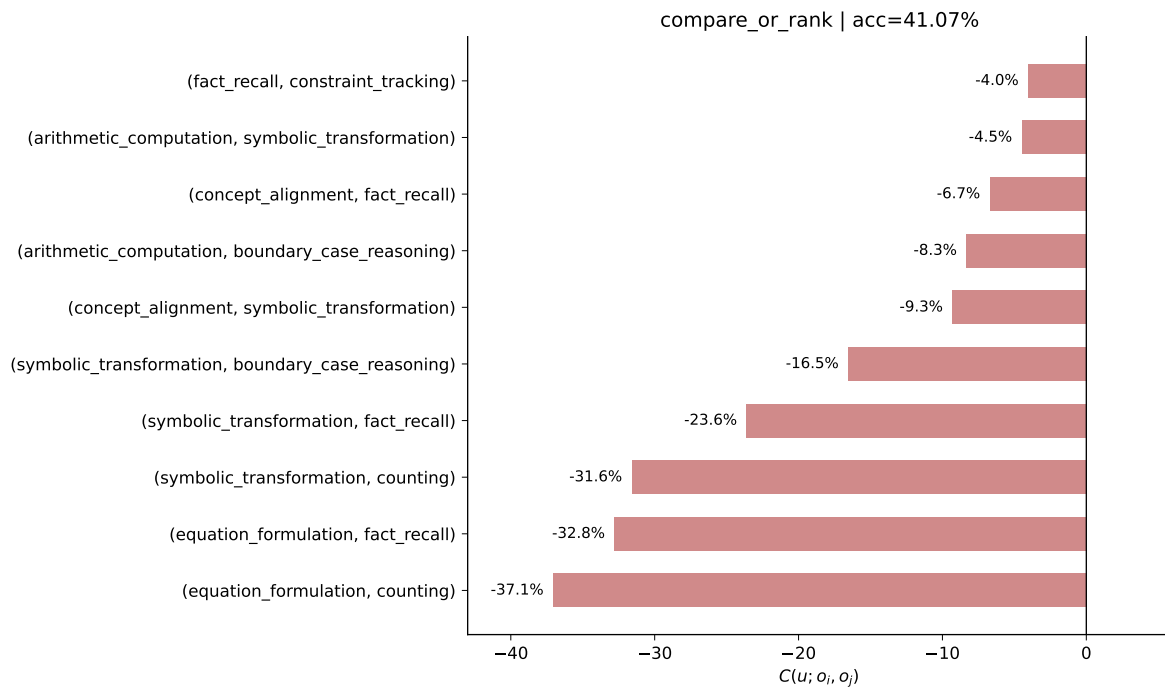


Figure 29: Non-additive Composition Effect of compare_or_rank